Leakage-Resilient Computation via Matrix Refreshing

Stefan Dziembowski
University of Warsaw
and IDEAS Institute
Warsaw, Poland

Sebastian Faust
TU Darmstadt
Darmstadt, Germany

Abstract

Leakage-resilient cryptography protects secrets against side-channel attacks, in which a physical implementation reveals partial information about its internal state. A fundamental primitive in this area is a *leakage-resilient storage*, which allows a secret to be stored in a way that remains secure even if an adversary learns some information about the storage's internal state. A popular approach represents a secret as the inner product of two n-bit vectors, called *shares*. The shares are stored and processed separately by two parts of the implementation, called the *left* and *right parties*. Because leakage accumulates over time, the shares must be *refreshed* repeatedly. We prove that a simple refresh, in which the left party multiplies its share by a uniform invertible matrix M and the right party multiplies its share by M^{-1}, is secure under continual local leakage. After T refreshes, each allowing at most λ bits of leakage per party, the transcripts for two stored secrets are at statistical distance at most $C \cdot T \cdot 2^{6\lambda - n/9}$ for an absolute constant C.

We also obtain a Boolean-circuit compiler with statistical privacy error at most $C \cdot S \cdot 2^{18\lambda - n/27}$ for S operations and at most λ bits of local leakage per party per operation. The guarantee covers repeated stateful evaluations with adaptively chosen public inputs, with S counting all operations across invocations. Thus, for polynomial-size circuits, a constant leakage fraction is tolerated with share length depending only on the security parameter. We formalize the refreshing bound, the composition and compiler guarantees, and the expected-work bounds in Lean 4. Our results also provide efficient simulation: the leakage can be reproduced from the public inputs and the circuit outputs alone, with expected polynomial overhead.

1 Introduction

Physical implementations of cryptographic algorithms leak information about their internal state through side channels such as running time, power consumption, and electromagnetic radiation. This has motivated a large body of work on *leakage-resilient cryptography*, which extends the traditional black-box security model to account for side-channel leakage and designs schemes that provably withstand it [28, 34, 23, 2, 38, 37, 3, 24, 18, 11, 25, 19, 32]. The central goal of leakage-resilient cryptography is to compute arbitrary programs securely in the presence of side-channel leakage. This goal is formalized by *leakage-resilient circuit compilers* [28, 24], which transform any program, modeled as a Boolean or arithmetic circuit, into a protected circuit that is secure against leakage. Security means that the leakage is useless to the adversary: anything it learns from the leakage together with the input/output behavior of the protected circuit could also be learned from the input/output behavior alone.

Our leakage model. Security is impossible if the leakage can be arbitrary, so the admissible leakage functions must be restricted. In this work, we consider *bounded leakage*: the adversary

chooses leakage functions of the internal state with bounded output length and learns their outputs. Moreover, the leakage is *continual*: the protected computation runs for many rounds, each round leaks, and the adversary chooses the leakage function of every round after seeing the answers of the earlier ones, so the total leakage is unbounded.

Considering bounded leakage alone is not enough, however: by the impossibility of program obfuscation [7], there exist programs for which a single bit of leakage, computed by an efficient function of the entire internal state, breaks security. We therefore also restrict which parts of the state a single leakage function can access. Models differ in how finely the state is split. At one extreme is the probing model of Ishai, Sahai, and Wagner [28], in which the adversary learns the values of a bounded number of individual wires. At the other extreme is the *split-state model* [23, 20, 21], in which the state is divided between two parties that leak separately. We keep this two-party structure and combine it with the assumption that only computation leaks [34], as in prior circuit compilers with bounded leakage [25, 31, 21, 26]. We call the participants the *left party* L and the *right party* R. Each stores its own part of the state and performs its part of the computation. Each party is thus a processor with its own memory, and the protected computation is a sequence of *operations* that leak in *rounds*: in each round, each party's leakage function reads the data that the party is currently computing on, namely its part of the state involved in the operation, the randomness drawn so far, and the messages it has received so far, and outputs a bounded number of bits. Memory that is not in use does not leak. The two leakage functions of a round are chosen before either answer is revealed; across rounds, the adversary may adapt its choices to all earlier answers. Communication between the parties is private, and each party may permanently erase data it no longer needs; erased data cannot be read by later leakage.

Leakage-resilient encoding. A first ingredient of a leakage-resilient compiler is an encoding scheme that splits a secret into two parts, so that separate leakage from each part reveals almost nothing about it. This notion of leakage-resilient encoding for split-state computation was introduced by Davì, Dziembowski, and Venturi [17], who also proposed a simple scheme based on the inner-product extractor. For an integer share length $n \geq 2$, their scheme encodes a secret bit $s \in \mathbb{F}_2 = \{0, 1\}$ as two nonzero vectors, a row $\ell \in \mathbb{F}_2^n$ held by the left party L and a column $r \in \mathbb{F}_2^n$ held by the right party R, chosen uniformly at random subject to

$$s = \ell \cdot r = \sum_{k=1}^{n} \ell_k \cdot r_k.$$

Its security rests on the fact that the inner product is a *two-source extractor* [15]: its output is close to uniform whenever its two inputs are independent and each is sufficiently unpredictable. This remains true given sufficiently bounded leakage obtained separately from each share, so the encoding hides s [17, 21].

Towards continual leakage: refreshing. A leakage-resilient encoding protects the secret only when the *total* leakage from each share is less than its length n. Otherwise, an adversary can learn both shares and hence s. A device that operates for a long time, however, keeps leaking, so its total leakage eventually exceeds any such bound. The standard remedy is to *refresh* the encoding periodically, replacing (ℓ, r) by a new encoding (ℓ', r') of the same secret, so that leakage in different periods concerns different shares [20, 4]. Refreshing is a central component of leakage-resilient circuit compilers [28, 24, 26, 21], and designing it is one of their main technical challenges. The difficulty is that a refresh must remain secure under composition: the new shares must be correlated with the old ones to encode the same secret, yet the refresh itself leaks, and the adversary may exploit this leakage to combine information from different periods.

The most natural refresh for inner-product encodings adds to each share a random vector that is orthogonal to the other share. For example, the left party sends a random column x with $\ell \cdot x = 0$ to the right party, which replaces r by $r + x$, and the right party then does the same for the left share. Dziembowski and Faust showed that this protocol is completely insecure under continual leakage [20, full version, App. D]. Both parties see the exchanged vectors, so an adversary that leaks two field elements from each party per refresh can accumulate the inner products of longer and longer prefixes of the shares and recover the secret after n refreshes. One approach to preventing such attacks is to deliver the refreshing randomness *obliviously*: a party learns nothing about the vector added to the other party's share beyond what correctness requires. Prior works [20, 21, 4] achieve this with a leak-free component that samples a random pair of orthogonal vectors and gives one vector to each party. The component itself does not leak, so each vector can leak only through the party that receives it. Leak-free components are a strong assumption since they rely on a piece of hardware that must not leak at all. This raises the following question: can an inner-product encoding be refreshed securely under continual leakage without any leak-free component, even when the leakage functions may read all the randomness of the refresh?

Goldwasser and Rothblum [26, 27] address this question in the model where only computation leaks: they construct a circuit compiler, including a refreshing procedure, based on the inner-product encoding. Apart from gaps in the proof of their main theorem (see our detailed discussion in App. C), their refreshing procedure is considerably more complex than the one we present in this work.

Leakage-resilient circuit compilers. Given a leakage-resilient refresh, the second ingredient of a compiler is a leakage-resilient *gadget* for each gate of the original circuit: a two-party protocol that takes encodings of the gate's inputs and produces an encoding of the gate's output. While the gadget runs, each party leaks on the data it uses: its shares of these encodings, its randomness, and the messages it receives. Composing the gadgets along the circuit yields the protected circuit. The main difficulty lies in nonlinear gates such as multiplication or NAND: the parties must compute an encoding of $s_1 \cdot s_2$ from encodings of s_1 and s_2, and every intermediate value of this computation may leak. Since the total leakage over an evaluation grows with the number of gates, the gadgets must refresh encodings as they go, and security must compose across all gates of the circuit.

The compiler of Dziembowski and Faust [21], which works in the split-state model, relies on leak-free components. The compiler of Goldwasser and Rothblum discussed above avoids them, but its proof has gaps. This raises a second question: can arbitrary circuits be compiled so that they remain secure when every active gadget leaks locally, without any leak-free internal component?

1.1 Our contributions

In this work, we answer both questions affirmatively. We give a refresh for inner-product encodings and a compiler for arbitrary circuits that remain secure under continual bounded leakage from both parties, without any leak-free component. We summarize our contributions in more detail below.

The refreshing protocol. Recall that the left party L holds the row share ℓ, the right party R holds the column share r, and the stored secret is their inner product $\ell \cdot r$. To refresh these shares, the left party samples $M \leftarrow\!\!\$\ \mathsf{InvMatr}_2^n$, where $\mathsf{InvMatr}_2^n$ denotes the set of invertible $n \times n$ matrices over $\mathbb{F}_2$. The left party sends M privately to the right party, and the two parties update

$$\ell := \ell \cdot M, \qquad r := M^{-1} \cdot r,$$

after which both parties erase M and their old shares. Correctness is immediate, since $(\ell \cdot M) \cdot (M^{-1} \cdot r) = \ell \cdot r$. Each refresh communicates the n^2 bits of M and costs an expected $O(n^3)$ bit

operations: sampling an invertible $n \times n$ matrix and inverting it by Gaussian elimination takes cubic time in its dimension. For security, we use the leakage model described above with each refresh as one operation: each party leaks at most λ bits through a function of its old share, its new share, and the entire matrix M. Unlike in [20], the refresh randomness is thus not delivered obliviously: both parties know M, so each knows exactly how the other party's share is transformed. The main challenge in our proof is therefore that the two parties' leakage is correlated through the shared randomness M: the leakage of a refresh cannot be treated as two independent leakages from the two shares, so the extractor argument for a single encoding does not apply directly. We use a hybrid argument: starting from the execution with stored secret 0, we switch the bit encoded by the successive encodings, produced by the refreshes, one at a time until we reach the execution with stored secret 1, and we bound the change caused by each switch. The intermediate experiments, called *hybrids*, are not real executions, since in each of them one refresh connects encodings of different bits. Each switch requires a new estimate that controls the leakage correlation introduced by the shared matrix M (see Sec. 1.2). As in our model, both leakage functions of a refresh are fixed before either answer is revealed; our proof extends to adaptive choices within a refresh, which we sketch in Sec. 6.2.

Secure continual refreshing. As a first result, we obtain that the above simple matrix refresh is secure. Suppose that the parties perform T refreshes, that each share has n bits, and that each party leaks at most λ bits per refresh. Let P_s^T denote the distribution of the complete leakage transcript when the stored secret is s, and let δ denote the statistical distance. Thm. 4.1 shows, for arbitrary leakage that is adaptive across refreshes and nonadaptive within each refresh, that

$$\delta\Big(P_0^T, P_1^T\Big) \le C \cdot T \cdot 2^{6\lambda - n/9}$$

for some constant C. For T polynomial in n, the bound decays exponentially whenever $\lambda \le \alpha n$ for a fixed $\alpha < 1/54$ (see Cor. 4.17). Sec. 4.8 derives the sufficient share length $n = O(\kappa + \lambda + \log(T+2))$ for target distance $2^{-\kappa}$.

Leakage-resilient circuit compilation. We also use our refresh to protect arbitrary Boolean circuits. We first treat stateless circuits and then extend the result to stateful ones. A *stateless* circuit Γ is evaluated once, on a public input $\mathbf{x}$ and a private input $\mathbf{s}$, and outputs $\mathbf{y} = \Gamma(\mathbf{x}, \mathbf{s})$, whereas a *stateful* circuit keeps a secret state, such as a cryptographic key, between invocations and in invocation t computes $(\mathsf{st}_{t+1}, \mathbf{y}_t) = \Gamma(\mathsf{st}_t, \mathbf{x}_t)$ from the current state st_t and a fresh public input $\mathbf{x}_t$ chosen by the adversary [28]. We express the circuit to be protected, which we call the *source circuit*, using NAND gates, whose gate function is $\mathrm{NAND}(s_1, s_2) = 1 + s_1 \cdot s_2$ over $\mathbb{F}_2$. We call the two input bits of a gate its *operands*. A gate of the source circuit may read the same wire twice, as in the negation $\mathrm{NAND}(s, s) = 1 + s$. Every circuit over a fixed AND/OR/NOT basis can be rewritten in the NAND basis with constant-factor overhead (see Sec. 5.1.1).

Each NAND gate of the source circuit is implemented by a two-party *gadget* in three steps, inspired by the multiplication gadget of Dziembowski and Faust [21] but without any leak-free component. First, to multiply the operands, the parties compute the tensor product of their shares locally. For encodings (L_1, R_1) and (L_2, R_2) of the operands s_1 and s_2 (refreshed copies, as explained below), the left party forms a long row a and the right party a long column b,

$$a = (L_1 \otimes L_2, 1), \qquad b = \begin{pmatrix} R_1 \otimes R_2 \\ 1 \end{pmatrix},$$

where the tensor product $\otimes$ lists all pairwise products of coordinates and a final coordinate equal to one is appended. These long shares have length $m = n^2 + 1$. Second, in the *matrix phase*, the parties refresh the long encoding with our refreshing protocol in dimension m and erase the refresh

matrix; Sec. 1.2 explains why this refresh is needed. Third, in the *shortening phase*, the parties shrink the refreshed long shares U and V back to n bits, so that the share length does not grow along the circuit. The left party samples a uniform nonzero row A of length n and a uniform matrix $H \in \mathbb{F}_2^{n\times m}$ of full row rank subject to $A \cdot H = U$ and privately sends H. The right party then sets $R = H \cdot V$, and (A, R) is the output encoding. If $R = 0$, the parties repeat with fresh A and H within the same leakage budget. The first attempt fails with probability at most 2^{1-n}.

For correctness, the tensor product multiplies inner products, so $a{\cdot}b = (L_1{\cdot}R_1){\cdot}(L_2{\cdot}R_2){+}1 = 1{+}s_1{\cdot}s_2$. The refresh preserves this inner product, and so does shortening, since $A \cdot R = A \cdot H \cdot V = U \cdot V$. Moreover, the completed shortening outputs an exactly uniform nonzero encoding of $U \cdot V$, even conditional on the long shares, a property we call *exact freshness*. For $m = n$, the matrix H is invertible and shortening is precisely our refresh with $M = H^{-1}$. The gadget leaks in two rounds, one per phase, each with one function per party and a total budget of λ bits per party per gadget: the shortening leakage may depend on the matrix-phase answers, but it cannot read the erased matrix M.

For composition, the compiler places a refresh between consecutive gadgets: every operand is refreshed just before it is used. Without these refreshes, an encoding read by several gates would leak in each of these operations, so its total leakage would grow with the number of times its wire is used. Concretely, a wire used at $\deg(w)$ *input ports*, that is, operand positions of gates, is refreshed in a chain of $\deg(w)$ copies, each created just before its use. Each copy then belongs to at most three operations, namely its producing refresh, its gate consumer, and the next refresh, so every encoding leaks in a bounded number of operations. This handles arbitrary fanout with linear overhead, and a gate that reads the same wire twice receives two independent copies.

Public inputs and constants are encoded directly under leakage, following the public-input step of Dziembowski and Faust [21, Figure 3]. In the stateless case, a trusted initialization supplies only the private input encodings, and at the end a trusted decoder releases the output $\mathbf{y}$ in the clear. The output shares are never revealed in full; like every encoding, they leak only through the operations that produce and use them. In the stateful case, the trusted initialization encodes only the initial state; the state encodings are kept between invocations and refreshed before each use like any other wire, and the decoder releases the output $\mathbf{y}_t$ of every invocation in the clear.

Efficiency and security of the compiler. For t_0 NAND gates of the source circuit (after rewriting it into a circuit with only NAND gates in case it had other gates) and I_{pub} public-input or constant wires, the compiler has $S = 3t_0 + I_{\text{pub}}$ operations: t_0 computation gadgets, $2t_0$ input-port refreshes, and I_{pub} public-input samplers, which encode the public inputs and constants. The compiled circuit has three operations per gate of the source circuit, and each gate costs an expected $O(n^4)$ bits of communication and $O(n^6)$ bit operations, both dominated by the $(n^2+1) \times (n^2+1)$ refresh matrix of its gadget. Since tensoring squares the share length, the matrix phase refreshes shares of length $m = n^2 + 1$, so its matrix takes $m^2 = O(n^4)$ bits to send and $O(m^3) = O(n^6)$ bit operations to sample and invert, as discussed above.

Security is shown by an efficient simulator, described below, that receives the public input $\mathbf{x}$ and the output $\mathbf{y} = \Gamma(\mathbf{x}, \mathbf{s})$ but not the private input $\mathbf{s}$, and reproduces the adversary's view of the leakage. With at most λ leaked bits per party per operation, Thm. 5.2 bounds the simulation error, the statistical distance between the real and the simulated view, by $C \cdot S \cdot 2^{18\lambda - n/27}$ for some constant C. For polynomial-size circuits this is exponentially small whenever $\lambda \le \alpha n$ with a fixed $\alpha < 1/486$, so the share length can depend only on the security parameter. The same bound extends to stateful circuits under continual leakage (see Thm. 6.1): over T invocations, with the input of each invocation chosen by the adversary after seeing the earlier leakage and outputs, the error is $C \cdot S \cdot 2^{18\lambda - n/27}$ with S counting the operations of all invocations, so that S grows linearly in T; the share length therefore grows only logarithmically in T, while the total leakage grows linearly. Our simulator is simple and

efficient: it runs the compiled circuit on the all-zero private input and the public input $\mathbf{x}$, answers the adversary's leakage queries from this execution, and outputs the given $\mathbf{y}$, the *authorized output*, instead of decoding its own output encodings. In particular, it never needs a private input that produces $\mathbf{y}$. Sec. 1.2 explains why this dummy execution cannot be distinguished from the real one.

Tab. 1 compares the results using the same parameters: $n \geq 2$ is the length of the stored shares, T counts refreshes, S counts all operations of the compiled circuit (gate gadgets, input-port refreshes, and public-input samplers), λ is the total leakage per party per operation, and δ is the statistical distance between the compared views. The errors accumulate linearly in the number of operations. For a prescribed finite error $2^{-\kappa}$, this gives the usual additive $\log(S+2)$ term in a sufficient share length. For polynomial-size circuit families, the share length need not depend on the circuit size.

Application	Statistical-distance bound	Sufficient leakage fraction
Continual refreshing (see Thm. 4.1)	$\delta \leq C \cdot T \cdot 2^{6\lambda - n/9}$	$\alpha < 1/54$
Circuit compilation, hidden output shares (see Thm. 5.2)	$\delta \leq C \cdot S \cdot 2^{18\lambda - n/27}$	$\alpha < 1/486$

Table 1: Bounds for continual refreshing and circuit compilation, with a common absolute constant C. For T or S polynomial in n and $\lambda \leq \alpha n$ with fixed α, the last column guarantees distance $2^{-\Omega(n)}$. The compiler's output encodings remain hidden.

Proof techniques. In the extractor argument for a single encoding, the leakage splits into a function of the left shares times a function of the right shares; for such split leakage, the probability of each answer is almost the same whether the shares encode 0 or 1. Our leakage does not split in this way, because the parties communicate: a message, such as the refresh matrix M or the shortening matrix H, is held by both parties and can be read by both leakage functions, so the probability of an answer depends on the two parties' shares jointly. Our main tool to handle such dependencies is a *local approximation*: we approximate the probability of each leakage answer of a single operation, up to a small error, by a sum of split functions with bounded total weight, so that the extractor argument applies to each summand. This sum is obtained from the singular value decomposition (SVD) of the answer probability.

For refreshing, we combine these approximations with the hybrid argument described above: each step changes the bit encoded by one state, which affects only the leakage of the neighboring refreshes, so the local errors add up linearly in the number of refreshes. For the compiler, the hybrid argument does not extend directly, because the private inputs determine the bits of all wires at once. Instead, we expand the probability of the whole leakage transcript into a common term that does not depend on the private inputs and correction terms that each contain a local approximation error or the difference between an encoding and independent shares; each correction term can be bounded locally, and the total error is linear in the number of operations. Sec. 1.2 explains both proofs in more detail.

Scope and limitations. The storage construction treats one-bit secrets and binary shares; the compiler handles vectors of bits. The tolerated leakage rate is smaller than for a fixed encoding without refresh, and we keep the absolute constants conservative to simplify the local estimates. If the initial storage encoding may contain a zero share, the refreshing bound increases by at most $4/2^n$ (see Sec. 4.9). In the compiler, every internal operation, including the input-port refreshes and the public-input samplers, may leak, so the total leakage grows with the size of the source

circuit. We assume private communication, the specified erasures, and that only the active operation leaks; moreover, the encoding of the private input or initial state and the decoding of the output in every invocation are trusted. Allowing the decoding to leak would require the simulator to program the output encodings, for instance with selector gates, and we leave this open. Sec. 6 proves the stateful extension and discusses leakage that is adaptive within a refresh and larger finite fields; our quantitative theorems concern the binary field and the stated phase structure.

Lean formalization. We provide a Lean 4 formalization of the refreshing and composition arguments for deterministic leakage strategies, the NAND compiler with input-port refreshes, and its counted expected-work simulator; randomized strategies reduce to deterministic ones by averaging, an argument that is not formalized. Sec. 6 describes its scope and simplifying assumptions. The source files are available in the `version-1.0` folder of the project repository.

AI usage. The authors obtained the results of this paper with assistance from ChatGPT and Claude AI. The idea for the matrix-based refresh came from the authors, but the security proofs were developed with help from the AI. The initial proof was first formulated as a result of the authors' interaction with ChatGPT 6.0 on September 5, 2026 (earlier attempts to solve this problem with other versions of ChatGPT were unsuccessful). The authors spent considerable time iterating with the AI to simplify the proofs. The AI also helped substantially in writing the paper. The Lean formalization was done by ChatGPT.

Paper's website and supplementary materials. The paper and its supplementary materials (including the Lean formalization) are available `github.com/sdziembowski/MatrixRefresh` in the `version-1.0` folder.

1.2 Technical outline

We now explain the main ideas behind our two security proofs. For refreshing, we follow the hybrid argument outlined with our refreshing protocol above: we change the bit encoded by one of the successive encodings at a time and bound the effect of each change. The main new ingredient is an explicit local approximation of the leakage of one refresh, which we motivate and construct below. For the compiler, we use the same kind of local approximation for every operation, but instead of a hybrid argument we expand the whole computation into terms that can each be bounded locally (see the last paragraph of this subsection). We first fix a few terms informally. Precise definitions follow in Secs. 2 to 5. A *round* is one refresh, a *state* $X_i = (L_i, R_i)$ is the pair of shares held after round i (see Sec. 2.2), the sequence $X_0, \ldots, X_T$ is the *chain*, a *table* is a real function of a left argument and a right argument, and a *leakage answer* z is the pair of outputs of the two leakage functions in one round (see Sec. 3). We write $K := 2^{2\lambda}$ for the number of possible leakage answers in one round. Each factor K in the bounds below comes from enumerating one answer.

The challenge and our approach. To see where the difficulty lies, consider first a single encoding (ℓ, r) that is never refreshed, with leakage $f(\ell)$ from the left party and $g(r)$ from the right party. The encodings that produce a given answer then form a *rectangle*, a set of left shares times a set of right shares, so the probability of the answer is a product of a function of ℓ and a function of r. Lindsey's lemma [30, Lemma 14.5] bounds the inner-product discrepancy of each such rectangle (see Lem. 2.20 for the formulation used here). Summing these bounds over the possible leakage answers shows that, for sufficiently small leakage budgets, the leakage distributions for the two secrets are statistically close.

A refresh destroys this product structure, because both leakage functions also read the matrix M. For a fixed matrix M, the old states that produce a given answer still form a rectangle, since M and the old state determine the new state. The hybrid argument, however, changes the bit encoded by one state at a time, and each state is the new state of one round and the old state of the next. We therefore view the probability of an answer as a table whose left argument is the pair of left shares (ℓ, ℓ') before and after the round and whose right argument is the pair (r, r'). Given both states, the matrix M is still uniformly random among the matrices that map the old state to the new one, and the answer probability is the average of the rectangles over these matrices. Such an average is in general not a product: through the common matrix, the leakage of the two parties is correlated. This is the main obstacle, and it is why the extractor argument for a single encoding does not apply directly.

Our approach is to *approximate* this table by a sum of left/right products $F = \sum_j a_j \cdot b_j^{\mathsf{T}}$. By linearity, the extractor argument applies to each product separately, so it suffices to control the total size of the sum. The approximation need not be a probability and may take negative values, so we call it a *signed* table. We measure the size of the sum by its *cost* $\sum_j \|a_j\|_2 \cdot \|b_j\|_2$, where $\|\cdot\|_2$ is the root-mean-square value under the uniform measure. The minimum cost over all such representations of a table F is its *nuclear norm* $\|F\|_*$ (see Eq. (A.3) in the App. A), so every representation satisfies (see Eq. (2.5))

$$\|F\|_* \le \sum_j \|a_j\|_2 \cdot \|b_j\|_2 .$$

Equality is attained by the balanced factors $\sqrt{\sigma_j} \cdot u_j$ and $\sqrt{\sigma_j} \cdot v_j$ of an orthonormal expansion $F = \sum_j \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}$ (see Lem. A.3 in the App. A). This expansion is the singular value decomposition of F for the uniform inner products. Its coefficients σ_j are the *singular coefficients* of F, and their sum is $\|F\|_*$ (see Sec. 2.3). The inner-product discrepancy estimate [15] (see Eq. (2.14)) turns the extractor property into a bound for arbitrary tables:

$$\left|\mathbb{E}_{(\ell,r)\leftarrow\$\mu_0}[F(\ell,r)] - \mathbb{E}_{(\ell,r)\leftarrow\$\mu_1}[F(\ell,r)]\right| \le \mathrm{disc}_n \cdot \|F\|_* , \qquad \mathrm{disc}_n = 3/\sqrt{2^n - 2},$$

where μ_s is the uniform distribution on nonzero encodings of s. Thus changing the bit encoded by one state changes the average of a table by at most disc_n times its nuclear norm. The rest of the refreshing proof constructs, for every answer of a refresh, such an approximation with small nuclear norm and small error.

Independent encodings and conditional leakage. To apply the discrepancy bound to one state, we must average over that state with its encoding distribution μ_s, independently of the other states. Observed leakage, however, correlates the states: given the answers, the successive states are in general not independent. We therefore never condition the states on the answers. A uniform invertible matrix sends every fixed encoding to a uniform encoding of the same bit (see Lem. 2.19). Before leakage, the successive states $X_i = (L_i, R_i)$ are therefore independent samples from μ_s. We may thus sample all states first and then draw each answer from its conditional leakage probability function $\mathsf{prob}_{i,h,z}(X_{i-1}, X_i)$, the probability that round i yields the answer z given the incoming history h (the answers of the earlier rounds) and the old and new states. The incoming history only determines which leakage functions are evaluated. The states are not conditioned on the listed answers having occurred. This description is the exact real execution: the probability of a transcript is the expectation, over independent states, of the product of the answer probabilities of its rounds. All dependencies created by the leakage are carried by these answer probabilities, each of which involves only the two states of its round, and this is what makes local replacements possible.

Explicit local approximation. We now construct the approximation of a single answer. Fix one answer z and any formal incoming history, that is, any sequence of earlier answers, whether or not

it can occur (see Sec. 3). The conditional probability table prob_z of this answer is a function of the four shares $((L, L'), (R, R'))$ of the old and new state. We write simply prob when the answer and history are fixed and clear from context. Our approximation is the simplest one suggested by the nuclear norm: we take the singular value decomposition of the table and keep only the terms with large singular coefficients. One point needs care. In an actual refresh, the old and new state always encode the same bit, and we call a four-share tuple *consistent* if it has this property. The table prob_z is thus defined only on consistent tuples, and these do not form a product set, since the condition $\ell \cdot r = \ell' \cdot r'$ couples the two sides. To obtain a table on all pairs of left and right arguments, we extend prob_z by zero on inconsistent tuples and take its orthonormal expansion $\mathsf{prob}_z = \sum_j \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}$ with probability-normalized singular coefficients, that is, with inner products and norms taken as expectations under uniform measures (see Lem. 2.10). Keep precisely the terms whose singular coefficients exceed β:

$$\mathsf{prob}_z^{>\beta} := \sum_{j:\, \sigma_j > \beta} \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}.$$

The superscript refers to singular coefficients, not to table entries (see Eq. (4.7)), and our approximation is $2 \cdot \mathsf{prob}_z^{>\beta}$. The factor two compensates for the consistency condition, satisfied by roughly half of independent four-share tuples. For any table F, the consistency projection $\mathsf{project}(F)$ keeps its entries at tuples whose two encoded bits agree and sets the others to zero. The *residual* $\Delta_z = \mathsf{prob}_z - 2 \cdot \mathsf{project}\big(\mathsf{prob}_z^{>\beta}\big)$ (see Eq. (4.8)) measures what the approximation misses. It equals $\mathsf{prob}_z - 2 \cdot \mathsf{prob}_z^{>\beta}$ on actual refresh tuples.

What do we need from this approximation? The discrepancy bound needs a small nuclear norm, and the hybrid argument below needs a small error under the actual encoding distributions. The hybrid argument also multiplies the approximations of two adjacent rounds, which requires control of the individual factors and not only of their total cost. This control is given by the *factor energies* Φ_β and Ψ_β of $\mathsf{prob}_z^{>\beta}$, the sums of squares of its balanced left and right factors (see Def. 2.16). The threshold β trades these requirements against each other: a smaller threshold keeps more terms, which decreases the error but increases the energies. For $q := 2^n - 1$ and $q^{-1/3} \le \beta \le 1$, the construction has

$$\begin{aligned}
\left\|\mathsf{prob}_z^{>\beta}\right\|_* &\le 2, && \\
\Phi_\beta(\ell, \ell') &\le 1/\beta && \text{for every } (\ell, \ell') \in U_{\mathsf{L}}^2, \\
\Psi_\beta(r, r') &\le 1/\beta && \text{for every } (r, r') \in U_{\mathsf{R}}^2, \\
\mathbb{E}_{(\ell,\ell') \leftarrow \$ U_{\mathsf{L}}^2}\big[\Phi_\beta(\ell, \ell')\big] &\le 2, && \\
\mathbb{E}_{(r,r') \leftarrow \$ U_{\mathsf{R}}^2}\big[\Psi_\beta(r, r')\big] &\le 2, && \\
\left\|\mathsf{prob}_z - 2 \cdot \mathsf{prob}_z^{>\beta}\right\|_{2,\mu_s \otimes \mu_s} &\le C \cdot \beta^{1/3} && \text{for every } s \in \{0, 1\}.
\end{aligned}$$

Here U_{L} and U_{R} are the nonzero left and right share spaces. Multiplying each factor by $\sqrt{2}$ gives the approximation $2 \cdot \mathsf{prob}_z^{>\beta}$, whose energies are $2 \cdot \Phi_\beta$ and $2 \cdot \Psi_\beta$. The error norm $\|\cdot\|_{2,\mu_s \otimes \mu_s}$ is the root-mean-square value when the old and new states are sampled independently from μ_s (see Sec. 4.3). The same $2 \cdot \mathsf{prob}_z^{>\beta}$ works for either secret and is defined even on inconsistent tuples, which will let us use it in the hybrids, where a round may connect encodings of different bits. Under any assignment of the bits encoded by the old and new state, its L_2 norm is at most 6. All bounds in this display are per answer and uniform over formal histories.

The nuclear bound uses the rectangle structure for a fixed matrix: it combines the masks of the answer, the indicator functions of the left and right sets whose product is its rectangle (see Sec. 3),

with an exact variance identity for the Gram matrices (matrices of pairwise inner products) under uniform invertible refreshing. The error bound is the heart of the argument, since it must show that the approximation misses little under the actual encoding distributions. Two facts combine. First, cancellation among the binary signs $(-1)^{\ell \cdot r + \ell' \cdot r'}$ that encode the consistency condition controls the largest singular coefficient of Δ_z. Second, a four-cycle count (an average of products of four table entries over two rows and two columns) and an exact identity for the squared norm of Δ_z (see Eq. (4.39)) bound this norm in terms of itself: for $t := \|\Delta_z\|_2$, they give the inequality $t^2 \le C \cdot \beta^{1/2} \cdot t^{1/2} + C/(q \cdot \beta)$ (see Eq. (4.15)), and hence $t \le C \cdot \beta^{1/3}$. Sec. 4.3.3 proves these bounds for refreshing. Sec. 5.3.1 states the extension needed for the long encodings (a, b) that the gadget forms from tensor products (see Sec. 1.1), whose additional proof details are in App. B.1.

The refreshing hybrid. We now combine the approximation with the hybrid argument. The difficulty is that the intermediate experiments of the hybrid argument are not real executions. We change the secret from zero to one in steps, labeling the states along the chain by the bits they encode, a prefix of ones followed by zeros (see Fig. 4). At the boundary between the ones and the zeros, one round connects an old state and a new state that carry different labels. We call it the *cheating round*, and the other rounds *valid*. No actual refresh behaves like this, and the true probability function prob describes only consistent tuples, so it cannot be used at the cheating round. This is exactly where the approximation helps, since it is defined on inconsistent tuples and is the same for both secrets. Smaller thresholds make the error smaller but the energies, and hence the discrepancy cost below, larger, and we take the smallest threshold allowed above, $\beta = q^{-1/3}$, giving the per-answer L_2 error $\varepsilon = C \cdot q^{-1/9}$. Write G for the resulting approximation $2 \cdot \mathsf{prob}_z^{>\beta}$, omitting the round, history, and answer indices for brevity. Each hybrid uses G at the cheating round and the true probability functions prob elsewhere. A round that uses G instead of prob is called *signed*, and a round that uses prob is *unsigned*. The endpoints, in which all labels are equal, are the actual transcript distributions. To move the boundary by one state, replace the next prob by G, change the bit of the state between the two signed rounds, and restore the preceding G to prob. Only two rounds are signed at any time, and true functions are inserted or removed only at valid rounds.

Each such move changes the experiment only slightly. We must bound the approximation error, the cost of changing a label, and the effect of adaptive leakage. We start with the approximation error. For fixed answers, write $e := \mathsf{prob} - G$ for the local approximation error. When we replace prob by G at a valid round, the error e appears multiplied by a signed companion G', the table of the other signed round that shares a state with the replaced round. The replacement error is therefore $e \cdot G'$ on three independent encoded states A, X, and B. These states are sampled from the encoding distributions specified by their current hybrid labels, without conditioning on the observed history. The L_2 bounds on e and G' (see Eqs. (4.63) and (4.64)) and the Cauchy–Schwarz inequality (see Lem. 2.7) give $\mathbb{E}_{A,X,B}[|e \cdot G'|] \le 6\varepsilon$, even though the two functions share the middle state.

Next, changing the label of a state, that is, the bit it encodes, is where the discrepancy bound enters. The state is shared by the two signed rounds adjacent to it. We multiply their product representations and average the nuclear norm of the result, as a table in that state, over the neighboring states. This is where we need the factor energies and not only the nuclear norms: we bound one energy pointwise by $2/\beta$ and average the other, whose mean is at most 4, which gives the bound $8/\beta$. The discrepancy cost is therefore $8\mathrm{disc}_n/\beta$.

Finally, the leakage is adaptive: the leakage functions of later rounds depend on earlier answers, so we cannot fix all leakage functions in advance. We handle this without conditioning on a history. First, sum the answers of the unsigned rounds after the signed ones in reverse order, which is possible because their true probability functions sum to one. Then enumerate the answers of the two signed rounds (the *block*) and of the one possible earlier neighboring round, and finally sum

the remaining prefix of answers. The enumeration costs at most K^3 and uses no independence conditioned on a history. Adding $O(T)$ steps gives

$$\delta\Big(P_0^T, P_1^T\Big) \le C \cdot T \cdot K^3 \cdot \Big(q^{-1/9} + \mathrm{disc}_n \cdot q^{1/3}\Big) \le C' \cdot T \cdot K^3 \cdot q^{-1/9}.$$

The initial and final stored encodings remain private.

This is also where our proof departs from earlier ones. The security proofs of Dziembowski and Faust [20, 21] also switch the encoded bit one encoding at a time. They simulate the leakage of the refresh at which the bit changes, the counterpart of our cheating round, from leakage of a fixed encoding. Our comparisons instead use signed approximations common to both secrets.

Why the gadget refreshes before shortening. We now turn to the compiler, starting with a design choice in the gadget of Sec. 1.1. Since shortening already outputs an exactly fresh encoding, one might try to shorten the long encoding (a, b) directly, without the matrix phase. This is not safe, because the right party's shortening leakage may read the message H. Since $A \cdot H = U$, the row space of H contains U. Without the refresh, U would be the long row $a = (L_1 \otimes L_2, 1)$, which is typically the only vector of this tensor form in that row space, so the right party's leakage could recover L_1 and L_2 from H and leak $s_1 = L_1 \cdot R_1$. After the refresh, U is a uniformly random long share, and once M is erased, the row space of H no longer identifies the operand shares.

Transport to a computation gadget. For circuits, we need a local approximation of the same kind for every operation. For the input-port refreshes, this is the local approximation behind Thm. 4.1, which controls one refresh with its input and output encodings retained. A computation gadget, however, has two new features.

First, each party's leakage may also read its two operand shares, so the approximation must keep these as additional arguments. The possible operand pairs, (L_1, L_2) on the left and (R_1, R_2) on the right, form sets of size $d = q^2$. The augmented tensor maps $(L_1, L_2) \mapsto (L_1 \otimes L_2, 1)$ and $(R_1, R_2) \mapsto (R_1 \otimes R_2, 1)$ take this set into nonzero vectors of length $m = n^2 + 1$ and are injective. Thanks to this injectivity, the approximation lemma with retained local data (see Lem. 5.4) applies to the matrix phase for $q^{-2/3} \le \beta \le 1$.

Second, the gadget ends by shortening, so the approximation, which is a table in the long shares, must be transported to the short output encoding. A random rectangular matrix shortens the fully refreshed long encoding after the refresh matrix is erased. A successful shortening attempt can also be sampled backward, starting from its output (A, R): the shortening matrix H is uniform of full row rank, V is uniform on $H \cdot V = R$, and $U = A \cdot H$. This backward description transports each product representation by averaging over H and its fibers, the sets of long shares consistent with a given short output. For each H, averaging a product over a fiber again gives a product, so the transported table is again a sum of left/right products. Jensen's inequality preserves the pointwise energy bound $2/\beta$, and since U and V are uniform in the backward description, the mean energy bounds 4 of the left and right factors are preserved. The Cauchy–Schwarz inequality (see Lem. 2.7) contracts the L_2 error under valid labelings, those in which the three wire bits satisfy the gate equation. The rare retry changes each answer probability by at most $4/2^n$. Thus the complete gadget (see Lem. 5.6) retains the per-answer nuclear and energy bounds and error $C \cdot \beta^{1/3}$, so the composition argument below can treat gadgets and refreshes alike. Under any assignment of bits to its three wires (the two operands and the output), the approximation has L_2 norm at most $C_0 := 4\sqrt{27}$.

Composition with hidden output shares. It remains to compose the local approximations of all operations of a compiled circuit. As for refreshing, we first make the wire encodings independent. For fixed public and private circuit inputs, exact freshness of each completed operation permits a

canonical experiment in which we sample one independent encoding for each wire before drawing leakage answers from their conditional probability functions (see Lem. B.5 in the App. B). These are unconditional samples, and observing the answers can create dependencies between them.

Two features make the composition harder than for a chain of refreshes. First, a wire can occur in three operations, so the errors of three local approximations may meet on one wire, and products of local errors call for an L_3 bound instead of an L_2 bound. Second, the wires form a circuit rather than a chain. Instead of moving a boundary through the circuit, we compare the executions for all private inputs with one common main term.

To obtain the L_3 bound, we use the same threshold $\beta = q^{-1/3}$ for each computation gadget and each input-port refresh, and we *trim* each approximation G: we remove its rows and columns whose factor energy exceeds $B := \beta^{-1/3}$. The separate mean-energy bounds and the valid-label L_2 error imply that the trimmed approximation has L_3 error $O(\beta^{1/9})$, while both pointwise energies are at most B. Write $e := \mathsf{prob} - G$ for this trimmed error, and write $D_s := \mu_s - \nu$ for the signed difference between an encoding distribution and the distribution ν of two independent uniform nonzero shares. The density of D_s relative to ν has largest singular coefficient $O(q^{-1/2})$.

To obtain the main term, we expand the product of the local functions using $\mathsf{prob} = G + e$. For each resulting set of error operations, keep the true encoding distributions on their incident wires and expand $\mu_s = \nu + D_s$ only on the other non-public wires. The term with no errors and no differences is the main term, called the *empty term* in Sec. 5.4, and every other term contains at least one error e or one difference D_s. Bounding these terms one at a time does not work: each term still contains approximations G at the remaining operations, and since these are not probabilities, their answers do not sum to one, so we would have to enumerate the answers of all operations, at a cost of up to a factor K per operation. We therefore group the terms. Fix a set $A_\star$ of *marked* errors and a set $W_\star$ of marked wires, and sum all terms containing these marked differences before taking absolute values. This sum restores the true probability functions everywhere except at the marked errors and at the operations touching marked wires, whose joint factor energies are at most $B^{3|W_\star|}$. Tensoring the discrepancy estimate, applying Hölder's inequality to the marked errors, and enumerating the answers of at most $9|A_\star| + 3|W_\star|$ neighboring operations gives the bound

$$\left(C \cdot K^9 \cdot q^{-1/27}\right)^{|A_\star|} \cdot \left(C \cdot K^3 \cdot q^{-1/6}\right)^{|W_\star|}.$$

The remaining true answer probabilities sum to one in reverse time, even under adaptive leakage. Inclusion–exclusion combines these marked bounds and controls the sum of all terms other than the main term by $O(S \cdot K^9 \cdot q^{-1/27})$ in the nontrivial range. Finally, the main term is a *signed measure* on leakage transcripts: it assigns each transcript a real weight that may be negative. It is independent of every private wire label and is therefore common to all private inputs. It cancels when the real and dummy-input executions are compared, and it never needs to be sampled. Sec. 5.4 gives the full argument.

1.3 Related work

Leakage models. Micali and Reyzin [34] proposed the axiom that only computation leaks information. Dziembowski and Pietrzak [23] combined it with bounded leakage per round, and constructed a stream cipher whose memory is divided into parts that leak independently. Akavia, Goldwasser, and Vaikuntanathan [2] studied bounded leakage from memory, and [11, 18] introduced continual memory leakage, in which secret keys are updated periodically. Tolerating leakage during the update itself is a recurring difficulty in that line of work. See [11, 33] and the references therein. Our

refresh plays the role of such an update, and our leakage functions see all of its randomness. Our model combines the two-party split state of [23, 20] with the axiom of [34] at the granularity of one operation, as in the compilers of [25, 21]. Kalai and Reyzin [32] survey this area.

Other models are closer to the practice of side-channel attacks. An early formal treatment of power analysis is due to Chari, Jutla, Rao, and Rohatgi [13], who proposed an abstract model of power consumption and a generic technique for provably resistant implementations. In the probing model of Ishai, Sahai, and Wagner [28], the adversary learns the values on a bounded number of wires of a circuit. In the noisy leakage model, leakage is inherently noisy, and Prouff and Rivain [39] proved the security in it of *masking*, a common countermeasure that splits each sensitive value into random shares and computes on these shares. Duc, Dziembowski, and Faust [19] gave a proof that avoids the leak-free gates of [39] by reducing noisy leakage to probing, and Brian, Dziembowski, and Faust [12] removed the dependence of the noise parameter on the field size. Our leakage is bounded and local, and it is neither probing nor noisy.

Leakage-resilient storage and its refreshing. Davì, Dziembowski, and Venturi [17] introduced leakage-resilient storage based on two-source extractors, including the inner product. Dziembowski and Faust [20] showed how to refresh inner-product encodings under continual leakage, and used the refreshed encodings to protect standard schemes such as the Okamoto identification scheme and an ElGamal-based encryption scheme. As explained above, their refresh delivers its randomness through a leak-free component that outputs pairs of orthogonal vectors. For fields of size $\Omega(n)$, their refreshing protocol tolerates $0.15 \cdot n \cdot \log_2(|\mathbb{F}|) - 1$ bits of leakage per refresh [20, full version, Corollary 2]. Their circuit compiler [21] uses a variant of this refresh whose output encoding is independent of the input encoding apart from the secret, again with a leak-free component, and Andrychowicz [4] gave a more efficient refreshing protocol under the same type of assumption. Refreshing of encoded secrets has also been studied for masking schemes. For circuits with only affine gates, the compilers of Andrychowicz, Dziembowski, and Faust [5] give constant-rate refreshing of an encoded secret against probing. Dziembowski, Faust, and Żebrowski [22] prove that a simple and efficient refreshing of masked values is secure in the noisy leakage model. In our protocol, the randomness that correlates the old and new encodings is exposed in full to the leakage functions, and no component is leak-free. The price is a smaller leakage rate: over $\mathbb{F}_2$, our refreshing bound is exponentially small for rates below $1/54$. We do not know whether this loss is inherent to the protocol or only to our analysis.

Local leakage from shares. Benhamouda, Degwekar, Ishai, and Rabin [8] study linear secret sharing under local leakage, where a function with a bounded output is applied to each share separately. They show that additive sharing and high-threshold Shamir sharing withstand such leakage over large prime fields when the number of parties is sufficiently large, whereas every linear scheme over a field of small characteristic is insecure, regardless of the number of parties. The inner-product encoding is bilinear rather than linear in its shares, and its security against local leakage over $\mathbb{F}_2$ rests on the extractor property [15, 17].

Fixed encodings and two-source extractors. For an encoding that is never refreshed, much more leakage can be tolerated. Dziembowski and Faust [21, Lemma 1 and Corollary 1] show that a variant of the k-coordinate inner-product encoding, with the first left coordinate required to be nonzero, over a field of size $q = \Omega(k)$ tolerates $0.49 \cdot k \cdot \log_2(q) - 1$ bits of leakage from each share, with error negligible in k. For a discrete random variable X, its *min-entropy* is $H_\infty(X) := -\log_2 \max_x \mathbb{P}[X = x]$. For an n-bit source its min-entropy rate is $H_\infty(X)/n$. The inner product is an extractor for two independent sources whose min-entropy rate exceeds $1/2$ [15]. Other explicit two-source extractors need less: Bourgain [10] reaches min-entropy rate 0.499, and

Chattopadhyay and Zuckerman [14] reach polylogarithmic min-entropy. We use the inner product because the refresh multiplies the shares by M and M^{-1}, which preserves their inner product.

Circuit compilers. Ishai, Sahai, and Wagner [28] introduced circuit compilers that protect computations on randomly masked shares against probing attacks, and Andrychowicz, Dziembowski, and Faust [5] construct compilers with perfect security against probing and leakage rate $O(1/\log n)$, where n is the security parameter. Other compilers restrict the class of leakage functions. Faust et al. [24] treat computationally bounded and noisy leakage. Rothblum [40] compiles circuits that resist AC^0 leakage without secure hardware, under a computational assumption about inner products. Miles and Viola [35] use products over the alternating group to resist leakage by NC^1 functions under the assumption $\mathrm{L} \neq \mathrm{NC}^1$, and by further classes for which average-case lower bounds are known. Ishai and Song [29] construct leakage-tolerant circuits, whose leakage can be simulated by the same class of leakage functions applied only to the input and output, for global leakage by parities, disjunctions, or conjunctions of wires. Ajtai [1] protects a RAM computation against an adversary who observes the memory accesses and, at chosen times, the memory contents.

Closest to our setting are compilers that allow arbitrary bounded leakage functions on parts of the computation that leak independently. Juma and Vahlis [31] rely on fully homomorphic encryption and a single leak-free token that samples from a fixed distribution independent of the key. Goldwasser and Rothblum [25] use public-key encryption and leak-free hardware that outputs encryptions of zero; their later compiler [26, 27] avoids leak-free hardware, but its proof has gaps (see App. C). The compiler of Dziembowski and Faust [21] is information-theoretic and works in a model of independently leaking parts similar to ours, but it requires leak-free components. Later works seek to remove this hardware requirement by adding computing components or using computational assumptions. Bitansky, Dachman-Soled, and Lin [9] propose using two main computing components and $O(1)$ auxiliary components to emulate Goldwasser and Rothblum's ciphertext banks and replace the leak-free hardware of the Dziembowski–Faust compiler. The security argument for this construction [9, full version, Sec. C.2.4] relies on Goldwasser and Rothblum's bank analysis and is therefore also subject to the proof gap involving Claim 6.24 described in App. C. Our counterexample to that claim does not establish an attack on the Bitansky–Dachman-Soled–Lin construction. Dachman-Soled, Liu, and Zhou [16] obtain computationally secure compilers with only two computing components and no leak-free hardware, using universal deniable encryption and receiver non-committing encryption under assumptions including indistinguishability obfuscation.

In comparison, our compiler is statistically secure without leak-free hardware inside the computation, but it trusts the encoding of the secret and the decoding of the outputs, and each leakage function reads only its party's data for the active operation, whereas the two-component guarantee of [16] allows leakage on each component's full evaluation state. Our leakage model also differs from the probing models above, so their numerical leakage rates are not directly comparable.

1.4 Organization

Sec. 2 introduces the encodings, the linear algebra of real tables, and the two-secret discrepancy bound. Sec. 3 defines leakage from a refresh, its answer masks, and histories of successive refreshes. Sec. 4 gives the complete refreshing argument: the security statement, the independent-encoding experiment, the explicit local approximation, and the hybrid proof that moves a boundary along the chain. It also derives the leakage rate and share length and treats initialization with zero shares. Sec. 5 gives the compiler, with its input-port refreshes and trusted output decoding, the computation gadget and its local approximation, the composition proof, and the simulator. Sec. 6 proves the extension to stateful circuits, records the formalization status, and discusses further extensions of

the leakage model and field. The appendices contain the remaining proofs and supporting material. App. A proves the preliminary lemmas. App. B proves the extension of the approximation lemma to retained local data (the operand shares that the leakage may also read), its tensor-encoding instance, and the shortening results. App. C gives a counterexample to an auxiliary claim in the proof of Goldwasser and Rothblum. Fig. 1 shows the progression from refreshing to computation.

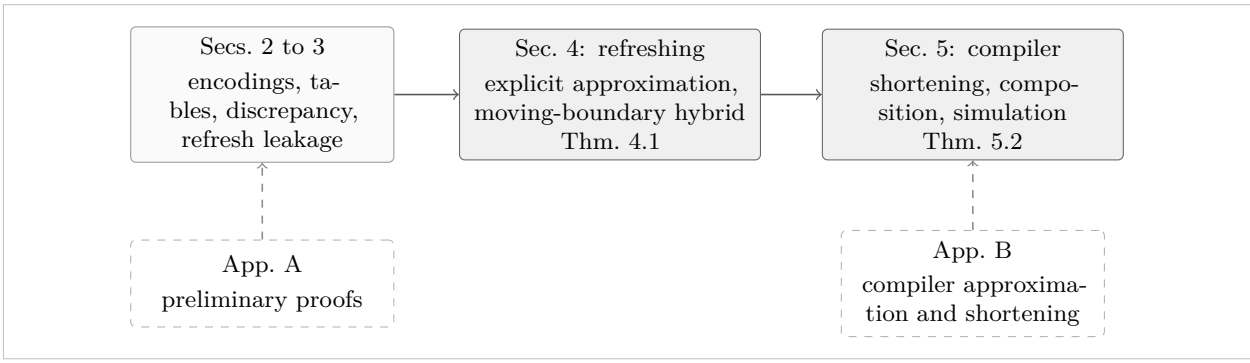


Figure 1: The proof is first presented for continual refreshing and then extended to circuits. Dashed boxes are appendices. The arrow to the compiler represents an extension of the method, not a deduction from refreshing security alone.

2 Preliminaries

The refreshing and circuit compilation proofs use the inner-product encoding and describe local leakage by real-valued tables. We first fix notation in Sec. 2.1 and define the encoding and matrix refresh in Sec. 2.2. We then develop the linear algebra of tables, including the definition of the nuclear norm, in Sec. 2.3, and establish the counting, freshness, and discrepancy estimates in Sec. 2.4 that connect these tables to encoded secrets. Proofs and auxiliary definitions are collected in the App. A.

2.1 Conventions

The share and matrix spaces are finite, and all real vector spaces used below are finite-dimensional. Share arithmetic takes place in $\mathbb{F}_2$, whereas the tables (real functions of a left share and a right share, defined in Def. 2.5 below) and their factor functions are real. The factor functions, introduced below in Def. 2.12, depend only on one party's data. We write log for the base-two logarithm and ln for the natural logarithm. The letter C denotes an absolute positive constant whose value may increase between inequalities. On a finite or countable set, a probability distribution also denotes its point-probability function. We write $x := f(y)$ for deterministic assignment. For a nonempty finite set S, the notation $X \leftarrow\!\$\, S$ means that X is sampled uniformly from S. If ρ is a probability distribution, $X \leftarrow\!\$\, \rho$ means sampling according to ρ. The notation $\rho \otimes \rho'$ denotes independent sampling from the two distributions. The marginal distribution of one component of a random tuple is its distribution when the other components are ignored. For real tables, $F \cdot H$ denotes entrywise multiplication, and inequalities between tables are pointwise. Each expectation indicates the variables over which it averages, and its argument is enclosed in square brackets.

Definition 2.1 (Finite signed weights)**.** *Let V be a finite set. A* signed weight function *on V, also called a* signed measure*, is a function $P : V \to \mathbb{R}$ whose values may be negative. We*

use the unnormalized L_1 norm

$$\|P\|_1 := \sum_{v \in V} |P(v)|,$$

and evaluate a real function $F : V \to \mathbb{R}$ against P by the finite weighted sum

$$\sum_{v \in V} F(v) \cdot P(v).$$

Every probability distribution, such as the distribution of the transcripts of a real execution, is a signed measure with nonnegative values. The security proofs also need signed measures with negative values. These arise when the probability of a leakage answer is replaced by a signed approximation, and when an encoding distribution is written as independent uniform shares plus a signed correction. Such intermediate measures need not have total mass one and are never sampled: they only serve to compare the genuine transcript distributions at the endpoints of an argument. Keeping the signs in a weighted sum allows positive and negative contributions to cancel, and the later discrepancy estimates bound such a sum as a whole before taking its absolute value. Note that the L_1 norm above is an unnormalized sum of absolute values, unlike the uniform averages that define the table norms in Def. 2.5.

Definition 2.2 (Statistical distance)**.** *Let V be a finite set and let P and Q be probability distributions on V. Their* statistical distance *is*

$$\delta(P, Q) := \frac{1}{2} \cdot \|P - Q\|_1 = \max_{\mathcal{M}: V \to \{0,1\}} \left| \mathbb{P}_{v \leftarrow\$ P}[\mathcal{M}(v) = 1] - \mathbb{P}_{v \leftarrow\$ Q}[\mathcal{M}(v) = 1] \right|.$$

We reserve δ for probability distributions and use the L_1 norm for arbitrary signed weights. App. A justifies the two expressions for δ. Tab. 2 in the App. A lists the main symbols for reference. Each symbol is defined where it is introduced.

2.2 Encodings and the matrix refresh

We first fix the encoding and the refresh protocol whose security we analyze. Throughout the main text, arithmetic on shares and refresh matrices is over $\mathbb{F}_2 = \{0, 1\}$, and $n \geq 2$ is an integer.

Definition 2.3 (Shares, states, and encoding distributions)**.** *Put $N := 2^n$ and $q := N - 1$. Let U_{L} be the set of nonzero row vectors and U_{R} the set of nonzero column vectors in $\mathbb{F}_2^n$. Each set has q elements. A* left share *is a row $\ell \in U_{\mathsf{L}}$, a* right share *is a column $r \in U_{\mathsf{R}}$, and a* joint state*, or simply* state*, is a pair $(\ell, r) \in U_{\mathsf{L}} \times U_{\mathsf{R}}$ of shares held at the same time. The state* encodes *the secret bit*

$$\ell \cdot r = \sum_{k=1}^{n} \ell_k \cdot r_k \in \mathbb{F}_2.$$

For $s \in \mathbb{F}_2$ define

$$\Omega_s := \{(\ell, r) \in U_{\mathsf{L}} \times U_{\mathsf{R}} : \ell \cdot r = s\},$$

and let μ_s be the uniform distribution on Ω_s. For a map $f : X \to Y$ and $y \in Y$, the fiber *of f at y is the set $\{x \in X : f(x) = y\}$. We call Ω_s the class of the secret s, or the fiber of the inner-product map at s: it is the set of all encodings with that secret.*

Uppercase letters denote random shares and states. The corresponding lowercase letters denote their values. Requiring both shares to be nonzero is a convenience. Sec. 4.9 shows that allowing zero shares at initialization increases the refreshing bounds by at most $4/N$.

Definition 2.4 (Matrix refresh)**.** *Let $s \in \mathbb{F}_2$ and let $T \geq 0$ be an integer. Initialize $(L_0, R_0) \leftarrow\!\$\ \mu_s$ and write $X_0 := (L_0, R_0)$. In each round $i = 1, \ldots, T$:*

1. *The left party samples $M_i \leftarrow\!\$\ \mathsf{InvMatr}_2^n$, where $\mathsf{InvMatr}_2^n$ is the set of invertible $n \times n$ matrices over $\mathbb{F}_2$, independently of the entire preceding execution, and sends it to the right party over a private channel.*

2. *The parties update their shares:*

$$L_i := L_{i-1} \cdot M_i, \qquad R_i := M_i^{-1} \cdot R_{i-1}.$$

3. *Each party erases its old share and the matrix, and keeps only its new share.*

The state after round i is $X_i = (L_i, R_i)$.

Every refresh preserves the secret and keeps both shares nonzero (see App. A.1). Because of the erasure in step 3, the data held by the parties at the start of round $i + 1$ are exactly the state X_i. Sending the square matrix costs n^2 bits per refresh.

2.3 Linear algebra for tables

Having fixed the protocol, we turn to the tool we use to analyze its leakage: the linear algebra of real functions on a product of two finite sets, which we call tables. In the security proofs, a table records the probability of a leakage answer as a function of the left party's data and the right party's data. The simplest tables are products $a(\ell) \cdot b(r)$. For these tables, the extractor property of the inner product bounds the difference between their means under μ_0 and μ_1 by $\frac{3}{\sqrt{2^n - 2}} \cdot \|a\|_2 \cdot \|b\|_2$ (see Eq. (2.15)). Here the L_2 norms, defined below in Def. 2.5, use the uniform measures on nonzero shares, so the means are close when these norms are bounded independently of n. For a fixed refresh matrix, the Boolean function indicating whether a given answer occurs is such a product, namely of the two masks defined below in Def. 3.4, because each party's output depends only on its own share and the matrix. The security proof, however, must also retain the new shares, since they feed the next round. It therefore uses a table on both the old and the new shares, obtained by averaging over the matrices compatible with those endpoints (see Sec. 1.2). This table differs from the ordinary average of the fixed-matrix masks on the old shares alone, and in general it is not a product. By linearity, this bound extends to sums of products, with a cost proportional to the total size of their terms. The *nuclear norm* is the minimum such cost, so a table with small nuclear norm has a mean that is nearly independent of the secret. The averaged table above need not have a small nuclear norm, and this is why the proofs approximate it.

We define the nuclear norm below in Def. 2.11 as the sum of the singular coefficients in the orthonormal expansion of Lem. 2.10. The uniform inner products and orthonormality used here are defined in Defs. 2.5 and 2.8. This expansion is the singular value decomposition for those inner products. For sets of equal size q, the normalized singular coefficients are $1/q$ times the ordinary singular values of the matrix $(F(\ell, r))$. We call this convention, in which inner products and norms are expectations under the uniform measures, the *probability normalization*. The analysis also uses product factorizations whose factors need not be orthogonal. App. A.2 proves

the properties we need, including the characterization of the nuclear norm as a minimum over product representations and the extension to tables on sets of different sizes. Both security proofs use product representations to control local comparisons, and the local approximation in Sec. 4.3 also separates an orthonormal expansion into its large and small coefficients. Every expectation in this section is uniform on the indicated set or product of sets. Although U_{L} and U_{R} both have size q, all the arguments also apply to two finite sets of different sizes, each equipped with its own uniform measure. In Sec. 4.3, we use the sets U_{L}^2 and U_{R}^2.

2.3.1 Tables and the Cauchy–Schwarz inequality

Definition 2.5 (Tables, inner products, and norms)**.** *A table is a real function $F : U_{\mathsf{L}} \times U_{\mathsf{R}} \to \mathbb{R}$. We call $\ell \in U_{\mathsf{L}}$ the* row *and $r \in U_{\mathsf{R}}$ the* column *of the entry $F(\ell, r)$. Let U be one of the two sets U_{L} or U_{R}. For a real function $g : U \to \mathbb{R}$,*

$$\mathbb{E}_{x \leftarrow \$ U}[g(x)] := \frac{1}{|U|} \sum_{x \in U} g(x).$$

For real functions a and b from U to $\mathbb{R}$, put

$$\langle a, b \rangle := \mathbb{E}_{x \leftarrow \$ U}[a(x) \cdot b(x)], \qquad \|a\|_2 := \left(\mathbb{E}_{x \leftarrow \$ U}\left[a(x)^2\right]\right)^{1/2}.$$

For tables F and W, put

$$\langle F, W \rangle := \mathbb{E}_{\ell, r}[F(\ell, r) \cdot W(\ell, r)], \qquad \|F\|_2 := \left(\mathbb{E}_{\ell, r}\left[F(\ell, r)^2\right]\right)^{1/2},$$

where ℓ and r are independent and uniform, so $\mathbb{E}_{\ell,r}$ is $1/q^2$ times the sum over all entries. We call $\langle F, W \rangle$ the pairing *and $\|F\|_2$ the L_2* norm*, or root-mean-square value. We write $\mathbb{E}_\ell$ for $\mathbb{E}_{\ell \leftarrow \$ U_{\mathsf{L}}}$ and $\mathbb{E}_r$ for $\mathbb{E}_{r \leftarrow \$ U_{\mathsf{R}}}$ when the set is clear.*

Definition 2.6 (Product tables)**.** *A* product table *is a table of the form $(\ell, r) \mapsto a(\ell) \cdot b(r)$, where $a : U_{\mathsf{L}} \to \mathbb{R}$ and $b : U_{\mathsf{R}} \to \mathbb{R}$. We also write it as $a \cdot b^{\mathsf{T}}$. A function with L_2 norm one is a* unit function*, and $a \cdot b^{\mathsf{T}}$ is a* unit product table *when both factors are unit functions.*

Here "unit" refers to the norms of the factors, not to their individual entries.

Lemma 2.7 (Cauchy–Schwarz inequality)**.** *For every finite index set I, every pair of real families $(x_i)_{i \in I}$ and $(y_i)_{i \in I}$, and all nonnegative real weights $(p_i)_{i \in I}$,*

$$\left| \sum_{i \in I} p_i \cdot x_i \cdot y_i \right|^2 \le \left(\sum_{i \in I} p_i \cdot x_i^2 \right) \cdot \left(\sum_{i \in I} p_i \cdot y_i^2 \right).$$

The inequality includes the sum and expectation versions. See Axler [6, Sec. 6A, Thm. 6.14, p. 189] for the Cauchy–Schwarz inequality. The inequality is used throughout the proofs, starting in Secs. 4.3 to 4.5.

2.3.2 Orthonormal expansions and the nuclear norm

Definition 2.8 (Orthonormal families)**.** *Functions on the same finite set are* orthogonal *when their inner product is zero. A family $(u_j)_j$ is* orthonormal *if each u_j has L_2 norm one and $\langle u_i, u_j \rangle = 0$ whenever $i \neq j$. The inner products and norms use the uniform measure of Def. 2.5.*

Lemma 2.9 (Pythagoras' theorem and Bessel's inequality)**.** *Let $m \geq 0$ be an integer, let U be a finite nonempty set, and let $(w_j)_{j=1}^m$ be an orthonormal family of functions $U \to \mathbb{R}$ for the uniform inner product. For every coefficient vector $(c_1, \ldots, c_m) \in \mathbb{R}^m$, Pythagoras' theorem gives*

$$\left\| \sum_{j=1}^{m} c_j \cdot w_j \right\|_2^2 = \sum_{j=1}^{m} c_j^2. \tag{2.1}$$

For every function $f : U \to \mathbb{R}$, Bessel's inequality gives

$$\sum_{j=1}^{m} \langle f, w_j \rangle^2 \leq \|f\|_2^2 . \tag{2.2}$$

Both statements also hold for the empty family.

See Axler [6, Sec. 6B, Thms. 6.24 and 6.26, p. 198]. We use Eqs. (2.1) and (2.2) in Sec. 4.3 and in the App. A.2.

Lemma 2.10 (Orthonormal expansion)**.** *For every real table $F : U_{\mathsf{L}} \times U_{\mathsf{R}} \to \mathbb{R}$, there exists an expansion*

$$F(\ell, r) = \sum_{j=1}^{m} \sigma_j \cdot u_j(\ell) \cdot v_j(r). \tag{2.3}$$

Here m is the rank of the matrix $(F(\ell, r))_{\ell,r}$, that is, the dimension of its column span. The coefficients satisfy $\sigma_j > 0$, and $(u_j)_j$ and $(v_j)_j$ are orthonormal families on U_{L} and U_{R}, respectively. The coefficients σ_j, called the singular coefficients *of F, are uniquely determined up to ordering. For $F = 0$, the expansion is the empty sum.*

The expansion Eq. (2.3) is the singular value decomposition for the uniform inner products. See Axler [6, Sec. 7E, Thm. 7.70, p. 273]. Truncation means discarding the expansion terms whose singular coefficients are at or below a chosen threshold. We use it in Secs. 4.3 and 5.3.

Definition 2.11 (Nuclear norm)**.** *For an orthonormal expansion as in Eq.* (2.3)*, define the* nuclear norm *by*

$$\|F\|_* := \sum_{j=1}^{m} \sigma_j .$$

In particular, $\|0\|_* = 0$. Lem. 2.10 ensures that this value does not depend on the chosen expansion.

The orthonormal expansion gives a convenient way to choose factors, but multiplying tables or averaging them during the shortening step introduced in Sec. 5.2 below need not preserve orthogonality. We therefore also keep track of product representations whose factors are not orthogonal.

Definition 2.12 (Product factors)**.** *In a representation*

$$F(\ell, r) = \sum_{j=1}^{m} a_j(\ell) \cdot b_j(r),$$

the functions $a_j : U_{\mathsf{L}} \to \mathbb{R}$ are the left factors*, and $b_j : U_{\mathsf{R}} \to \mathbb{R}$ are the* right factors*. The number m is finite and may depend on the representation. The empty representation is allowed for the zero table.*

The next lemma collects basic properties of the nuclear norm, including a bound for arbitrary product representations.

Lemma 2.13 (Basic nuclear-norm bounds)**.** *The function $\|\cdot\|_*$ is a norm on the real tables on $U_{\mathsf{L}} \times U_{\mathsf{R}}$. For every such table F,*

$$\|F\|_2 \le \|F\|_* . \tag{2.4}$$

For every integer $m \ge 0$ and all functions $a_j : U_{\mathsf{L}} \to \mathbb{R}$ and $b_j : U_{\mathsf{R}} \to \mathbb{R}$, indexed by $j = 1, \dots, m$,

$$\left\| \sum_j a_j \cdot b_j^{\mathsf{T}} \right\|_* \le \sum_j \|a_j\|_2 \cdot \|b_j\|_2 . \tag{2.5}$$

For every pair of functions $a : U_{\mathsf{L}} \to \mathbb{R}$ and $b : U_{\mathsf{R}} \to \mathbb{R}$,

$$\left\| a \cdot b^{\mathsf{T}} \right\|_* = \|a\|_2 \cdot \|b\|_2 . \tag{2.6}$$

Proof: see App. A.2.2.

We use Eq. (2.5) whenever approximations are multiplied, in Sec. 4.5 and Sec. 5.4.4 and in the App. B.3. We also use Eq. (2.4) in the App. B.3.

2.3.3 Largest coefficients and factor energies

The nuclear norm summarizes a table by one number. We next define two quantities that give finer information: the largest singular coefficient (see Def. 2.14 below) and factor energies (see Def. 2.16 below). The largest singular coefficient bounds the pairing of a table with every product of functions of unit L_2 norm, so it shows how little the terms discarded by a truncation can correlate with a product test. Factor energies record where, among the rows and columns, a chosen product representation is large. We need this local information when two approximations that share an encoding are multiplied, because their sizes at the same row or column multiply.

Definition 2.14 (Largest singular coefficient)**.** *Let $\sigma_{\max}(F)$ denote the largest coefficient of an orthonormal expansion of F, with $\sigma_{\max}(0) := 0$.*

Lemma 2.15 (Largest singular coefficient)**.** *For every real table $F : U_{\mathsf{L}} \times U_{\mathsf{R}} \to \mathbb{R}$ and all functions $u : U_{\mathsf{L}} \to \mathbb{R}$ and $v : U_{\mathsf{R}} \to \mathbb{R}$,*

$$\left| \left\langle F, u \cdot v^{\mathsf{T}} \right\rangle \right| \le \sigma_{\max}(F) \cdot \|u\|_2 \cdot \|v\|_2 . \tag{2.7}$$

For every nonzero F, there exist unit functions u and v attaining equality in Eq. (2.7). For every real table F,

$$\sigma_{\max}(F) \le \|F\|_2 . \tag{2.8}$$

Proof: see App. A.2.3.

The local approximation proof in Sec. 4.3 uses Eq. (2.7) to control the part of an expansion with small coefficients, and Sec. 5.4.3 uses it for encoding differences.

Definition 2.16 (Factor energies)**.** *Fix a finite product representation $F = \sum_j a_j \cdot b_j^{\mathsf{T}}$. Its left and right* factor energies *are*

$$\Phi_F(\ell) := \sum_j a_j(\ell)^2, \qquad \Psi_F(r) := \sum_j b_j(r)^2.$$

They refer to the chosen representation, not only to the table F, and have means

$$\mathbb{E}_\ell[\Phi_F(\ell)] = \sum_j \|a_j\|_2^2\,, \qquad \mathbb{E}_r[\Psi_F(r)] = \sum_j \|b_j\|_2^2\,.$$

Definition 2.17 (Balanced factors)**.** *Fix an orthonormal expansion $F = \sum_j \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}$. Its* balanced factors *are*

$$a_j := \sqrt{\sigma_j} \cdot u_j, \qquad b_j := \sqrt{\sigma_j} \cdot v_j,$$

so that $F = \sum_j a_j \cdot b_j^{\mathsf{T}}$. Their energies are those of Def. 2.16, for this chosen representation.

For these balanced factors, $\|a_j\|_2^2 = \|b_j\|_2^2 = \sigma_j$, so the mean energies are

$$\mathbb{E}_\ell[\Phi_F(\ell)] = \mathbb{E}_r[\Psi_F(r)] = \sum_j \sigma_j = \|F\|_*\,. \tag{2.9}$$

These factors also attain the minimum in the characterization of the nuclear norm (see Eq. (A.3) in the App. A.2.2). At a row ℓ, the energy $\Phi_F(\ell)$ is the squared Euclidean length of the vector $(a_j(\ell))_j$: it measures how large the left factors are at that row. Similarly, $\Psi_F(r)$ measures the right factors at column r. The energies bound each entry (see App. A.2.4): for every $\ell \in U_{\mathsf{L}}$ and $r \in U_{\mathsf{R}}$,

$$|F(\ell, r)| \le \sqrt{\Phi_F(\ell) \cdot \Psi_F(r)}. \tag{2.10}$$

Thus energy bounds also control the entries of the table. For balanced factors, each energy has mean $\|F\|_*$, while its values record where this size is concentrated. For example, on three rows the energy profiles $(1, 1, 1)$ and $(3, 0, 0)$ both have mean 1, but the second concentrates all the energy in one row. The local approximation supplies both pointwise and mean energy bounds: when multiplying the approximations of neighboring rounds or gates, the proofs bound some energies pointwise and average another. The representations used for rectangular shortening in Sec. 5 need not be balanced, so there the two mean energies are bounded separately. Fig. 2 works out these notions for a 3×3 table.

2.3.4 Example: a small nuclear norm need not bound every energy

The following example shows why the mean energies, which equal the nuclear norm for balanced factors, do not suffice on their own. Consider the table

$$F = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \end{pmatrix}.$$

Multiplying the left factor by $3^{-1/4}$ and the right factor by $3^{1/4}$ balances their norms. The resulting energies are

$$\Phi_F = \left(1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3}\right), \qquad \Psi_F = \left(\sqrt{3}, 0, 0\right),$$

and both means equal $\|F\|_* = 1/\sqrt{3}$. The energy at the first column is nevertheless three times its mean, and for the analogous $d \times d$ table the ratio is d. Thus a nuclear-norm bound alone does not control every energy, or the nuclear norm of a product of several tables. Our explicit approximation therefore supplies additional pointwise energy bounds, which are used together with the mean bounds.

$$F = \begin{array}{c} \\ \ell{=}\texttt{01} \\ \ell{=}\texttt{10} \\ \ell{=}\texttt{11} \end{array} \begin{array}{c} \begin{array}{ccc} r{=}\texttt{01} & r{=}\texttt{10} & r{=}\texttt{11} \end{array} \\ \begin{pmatrix} -1 & -4 & 13 \\ 13 & 4 & -1 \\ 4 & 10 & -4 \end{pmatrix} \end{array} = a_1 \cdot b_1^{\mathsf{T}} + a_2 \cdot b_2^{\mathsf{T}} + a_3 \cdot b_3^{\mathsf{T}},$$

$$F = \underbrace{\begin{pmatrix} -6 & -6 & 6 \\ 6 & 6 & -6 \\ 6 & 6 & -6 \end{pmatrix}}_{a_1 \cdot b_1^{\mathsf{T}}} + \underbrace{\begin{pmatrix} 6 & 0 & 6 \\ 6 & 0 & 6 \\ 0 & 0 & 0 \end{pmatrix}}_{a_2 \cdot b_2^{\mathsf{T}}} + \underbrace{\begin{pmatrix} -1 & 2 & 1 \\ 1 & -2 & -1 \\ -2 & 4 & 2 \end{pmatrix}}_{a_3 \cdot b_3^{\mathsf{T}}}.$$

$$a_1 = \begin{pmatrix} \sqrt{6} \\ -\sqrt{6} \\ -\sqrt{6} \end{pmatrix}, \qquad a_2 = \begin{pmatrix} \sqrt{6} \\ \sqrt{6} \\ 0 \end{pmatrix}, \qquad a_3 = \begin{pmatrix} 1 \\ -1 \\ 2 \end{pmatrix},$$

$$b_1^{\mathsf{T}} = \begin{pmatrix} -\sqrt{6} & -\sqrt{6} & \sqrt{6} \end{pmatrix}, \qquad b_2^{\mathsf{T}} = \begin{pmatrix} \sqrt{6} & 0 & \sqrt{6} \end{pmatrix}, \qquad b_3^{\mathsf{T}} = \begin{pmatrix} -1 & 2 & 1 \end{pmatrix}.$$

Here $(\sigma_1, \sigma_2, \sigma_3) = (6, 4, 2)$. Dividing a_j and b_j by $\sqrt{\sigma_j}$ gives the unit functions

$$u_1 = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}, \qquad u_2 = \begin{pmatrix} \sqrt{3/2} \\ \sqrt{3/2} \\ 0 \end{pmatrix}, \qquad u_3 = \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \\ \sqrt{2} \end{pmatrix},$$

$$v_1^{\mathsf{T}} = \begin{pmatrix} -1 & -1 & 1 \end{pmatrix}, \qquad v_2^{\mathsf{T}} = \begin{pmatrix} \sqrt{3/2} & 0 & \sqrt{3/2} \end{pmatrix}, \qquad v_3^{\mathsf{T}} = \begin{pmatrix} -1/\sqrt{2} & \sqrt{2} & 1/\sqrt{2} \end{pmatrix}.$$

These form two orthonormal families under uniform averaging over three coordinates, with $\|u_j\|_2 = \|v_j\|_2 = 1$. Thus $F = \sum_{j=1}^{3} \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}$.

First contribution to F

$$a_1 \cdot b_1^{\mathsf{T}} = \begin{pmatrix} \sqrt{6} \\ -\sqrt{6} \\ -\sqrt{6} \end{pmatrix} \cdot \begin{pmatrix} -\sqrt{6} & -\sqrt{6} & \sqrt{6} \end{pmatrix} = \begin{pmatrix} -6 & -6 & 6 \\ 6 & 6 & -6 \\ 6 & 6 & -6 \end{pmatrix}.$$

First unit product table

$$u_1 \cdot v_1^{\mathsf{T}} = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix} \cdot \begin{pmatrix} -1 & -1 & 1 \end{pmatrix} = \begin{pmatrix} -1 & -1 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & -1 \end{pmatrix}.$$

$$\underbrace{a_1 \cdot b_1^{\mathsf{T}}}_{\text{balanced factors}} = \underbrace{6}_{\sigma_1} \cdot \underbrace{u_1 \cdot v_1^{\mathsf{T}}}_{\text{unit functions}}.$$

ℓ	$a_1(\ell)^2$	$a_2(\ell)^2$	$a_3(\ell)^2$	$\Phi_F(\ell)$
01	6	6	1	13
10	6	6	1	13
11	6	0	4	10
mean	6	4	2	12

r	01	10	11	mean
$b_1(r)^2$	6	6	6	6
$b_2(r)^2$	6	0	6	4
$b_3(r)^2$	1	4	1	2
$\Psi_F(r)$	13	10	13	12

Averaging either energy gives the nuclear norm:

$$\mathbb{E}_\ell[\Phi_F(\ell)] = \mathbb{E}_r[\Psi_F(r)] = 6 + 4 + 2 = 12 = \|F\|_*.$$

Figure 2: An orthonormal expansion with different left and right factors. The functions a_j and u_j depend on ℓ, and b_j and v_j depend on r, with coordinate order 01, 10, and 11. The energy tables sum squared factors across rows on the left and down columns on the right.

2.4 Counting and discrepancy

We now collect the binary counting estimates used to analyze a refresh. The first two facts describe the distributions μ_s and how a fresh matrix acts on them. The discrepancy bound then compares the means of real tables under two secrets. The refreshing proof uses this bound to change the bit encoded by one state.

Definition 2.18 (Independent reference distribution)**.** *Let ν be the uniform distribution on $U_{\mathsf{L}} \times U_{\mathsf{R}}$. For each $s \in \mathbb{F}_2$, put $p_s := \mathbb{P}_{(L,R)\leftarrow\$\nu}[L \cdot R = s]$.*

Under ν the two shares are independent, whereas under μ_s they are not.

Counting encodings. The fiber sizes and class probabilities are

$$\begin{aligned} |\Omega_0| &= q \cdot (N-2)/2, \\ |\Omega_1| &= q \cdot N/2, \\ p_0 &= (N-2)/(2q), \qquad (2.11) \\ p_1 &= N/(2q). \qquad (2.12) \end{aligned}$$

Both μ_0 and μ_1 have uniform left and right marginals, and conditioning ν on Ω_s gives μ_s. For $n \geq 2$,

$$p_1 \geq p_0 = \frac{N-2}{2(N-1)} \geq \frac{1}{3}. \qquad (2.13)$$

The counting argument is in the App. A.1.

Lemma 2.19 (Freshness)**.** *For every $s \in \mathbb{F}_2$ and every $(\ell, r) \in \Omega_s$, if $M \leftarrow\$ \mathsf{InvMatr}_2^n$, then the refreshed pair $(\ell \cdot M, M^{-1} \cdot r)$ has distribution μ_s.*

Proof: see App. A.1.

Freshness also holds conditionally on the entire preceding execution, but it need not hold after conditioning on the leakage answers of the current round. For example, an answer that reveals one coordinate of a new share restricts the possible values of that share. Thus exact freshness describes the sampling before the answer is observed. Hence, the security proof must still account for what the answer reveals. Freshness gives the exact experiment with independent encodings in Sec. 4.2, and its argument is reused in Sec. 5.3.1 and in the App. B.3.

The next lemma says that a small nuclear norm makes the mean of a table nearly independent of the secret.

Lemma 2.20 (Two-secret discrepancy)**.** *For every real table F,*

$$\left|\mathbb{E}_{(\ell,r)\leftarrow\$\mu_0}[F(\ell,r)] - \mathbb{E}_{(\ell,r)\leftarrow\$\mu_1}[F(\ell,r)]\right| \leq \frac{3}{\sqrt{N-2}} \cdot \|F\|_* . \qquad (2.14)$$

In particular, for real functions $u : U_{\mathsf{L}} \to \mathbb{R}$ and $v : U_{\mathsf{R}} \to \mathbb{R}$,

$$\left|\mathbb{E}_{(\ell,r)\leftarrow\$\mu_0}[u(\ell) \cdot v(r)] - \mathbb{E}_{(\ell,r)\leftarrow\$\mu_1}[u(\ell) \cdot v(r)]\right| \leq \frac{3}{\sqrt{N-2}} \cdot \|u\|_2 \cdot \|v\|_2 . \qquad (2.15)$$

Proof: see App. A.3.1.

We use Eq. (2.14) in Sec. 4.5 to change the bit encoded by one state. In Sec. 5.4.3, we prove the analogous estimate for the differences $\mu_s - \nu$.

3 Leakage in continual refreshing

Before analyzing the storage protocol of Def. 2.4, we must specify what an adversary can learn from it. Recall that each refresh replaces (L, R) by $(L', R') = (L \cdot M, M^{-1} \cdot R)$. We first specify, for one refresh, which data a leakage function may read, when the adversary must choose it, and what the adversary then observes. We then introduce histories and answer rectangles in Defs. 3.3 and 3.4 below, before reducing randomized strategies to deterministic ones for the proof in Sec. 4.

3.1 Leakage in a matrix refresh

During a refresh, each party's leakage function, an arbitrary function whose output the adversary observes, may read everything the party holds: its old share, its new share, and the entire refresh matrix. It cannot read the other party's shares. The only protocol data common to the two functions are the entries of the matrix, which both parties hold.

The timing of the queries is a separate restriction. The adversary chooses both parties' leakage functions using the answers of earlier refreshes and its random tape. Both functions are fixed before either output of the current refresh is revealed. Thus leakage is adaptive across refreshes and nonadaptive within a refresh: an answer may influence the functions of every later refresh, but neither output can influence the other party's function in the same refresh. Sec. 6.2 sketches how the proof extends to adaptivity within a refresh.

Definition 3.1 (Leakage queries)**.** *Fix an integer $\lambda \geq 0$, the leakage budget of each party per refresh. For every round $i \in \{1, \ldots, T\}$, let (L, R) be its old state and let M be its refresh matrix. For a preceding history h, that is, the sequence of answers already observed in refreshes $1, \ldots, i-1$ (defined formally in Def. 3.3), the two leakage functions are arbitrary maps*

$$\mathsf{Leak}^{\mathsf{L}}_{i,h} : U_{\mathsf{L}} \times \mathsf{InvMatr}_2^n \longrightarrow \{0,1\}^{\lambda},$$
$$\mathsf{Leak}^{\mathsf{R}}_{i,h} : U_{\mathsf{R}} \times \mathsf{InvMatr}_2^n \longrightarrow \{0,1\}^{\lambda}.$$

The functions may also depend on the adversary's tape, which we omit from the notation. The left function is evaluated on (L, M) and the right function on (R, M). These inputs determine the party's new share, so the function may read that share as well. Returning exactly λ bits is a convention: a function with a smaller fixed output length can be padded with zeros. The adversary observes no other information about the shares, and in particular does not observe the private transmission of the matrix.

Definition 3.2 (Complete answers)**.** *The* complete answer*, or* answer*, of the refresh is the pair of leakage outputs*

$$Z_i := \left(\mathsf{Leak}^{\mathsf{L}}_{i,h}(L, M), \mathsf{Leak}^{\mathsf{R}}_{i,h}(R, M)\right).$$

It belongs to the fixed answer alphabet

$$\mathcal{Z} := \{0,1\}^{\lambda} \times \{0,1\}^{\lambda}, \qquad K := |\mathcal{Z}| = 2^{2\lambda}.$$

Some answer pairs may have probability zero. Fig. 3 shows the local data and the answers observed in one refresh.

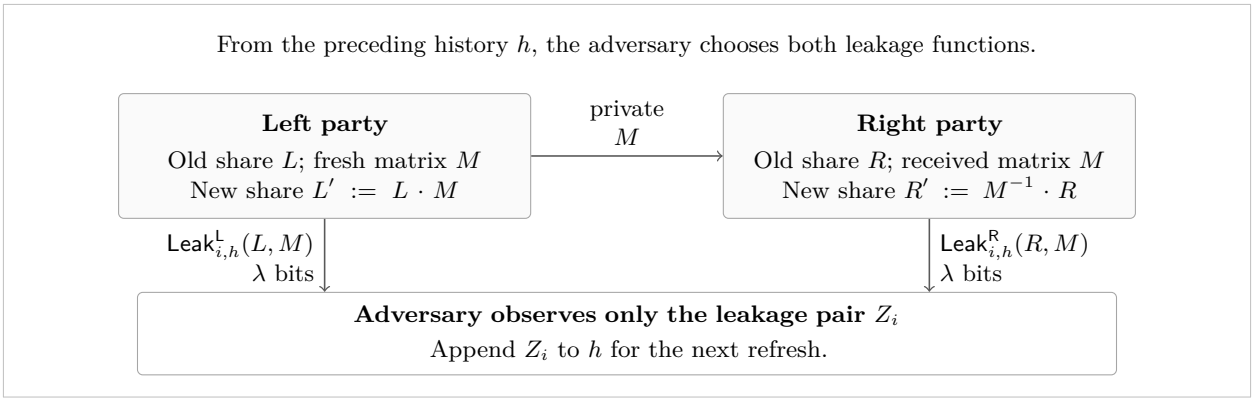


Figure 3: Leakage in one matrix refresh. Both functions are fixed before either answer is revealed. After leakage the parties erase their old shares and M, keeping L' and R'.

3.2 Histories

To specify an adaptive query, we must record which answers the adversary has already seen. We also retain sequences that have probability zero in the real execution, because the intermediate expressions in the proof can assign them nonzero weight.

Definition 3.3 (Histories)**.** *For an execution of T refreshes and each $i \in \{0, \ldots, T\}$, the* history *after refresh i is $H_i := (Z_1, \ldots, Z_i) \in \mathcal{Z}^i$, and $H_0 := \varnothing$ is the empty history. The history H_{i-1} is the* incoming history *of refresh i. For $h = (z_1, \ldots, z_{i-1})$, write $h \| z := (z_1, \ldots, z_{i-1}, z)$ for the history obtained by appending z. A* formal history *is any element of $\mathcal{Z}^i$, that is, any sequence of i answers, including those with probability zero.*

Together with the adversary's tape, the incoming history determines the two leakage functions of the refresh. The *leakage strategy* is the rule by which the adversary chooses each pair of leakage functions from the preceding history. The experiments for the two secrets use the same rule. The strategy is defined on every formal history, including those that no execution can reach. Its choices there may be arbitrary functions allowed by Def. 3.1, with the same output lengths. Every sum over z ranges over $\mathcal{Z}$, unless stated otherwise, and history sums include all formal histories.

Fixing a formal history selects the leakage functions. It does not condition the shares or matrices on the listed answers having occurred. Conditioning on an observed history can instead change the state distributions and make them dependent. The proofs keep these operations apart by sampling states without conditioning on the transcript and accounting for the answers through their probabilities (see Secs. 4.2 and 4.6).

3.3 Answer rectangles and masks

Histories tell us which functions are evaluated in a refresh. We now ask what one answer of these functions says about the old state. The new share is determined by the old share and M, so, once the incoming history and the adversary's tape are fixed, each leakage function depends only on its party's old share and M. Consequently, for a fixed matrix, the old states that produce a given answer form a rectangle: the product of a set of left shares and a set of right shares. In Def. 3.4 below, we describe each answer rectangle by two masks, Boolean functions equal to one on its left and right sets, respectively, and zero elsewhere. For each fixed matrix M, their product is a

product table (see Def. 2.6), so Lem. 2.20 controls how its mean depends on the encoded bit. We also record that the rectangles of all answers partition the set of old states.

Definition 3.4 (Answer rectangles and masks)**.** *Fix $i \in \{1, \dots, T\}$, a formal incoming history $h \in \mathcal{Z}^{i-1}$, the adversary's tape, a matrix $M \in \mathsf{InvMatr}_2^n$, and a complete answer $z = (z_{\mathsf{L}}, z_{\mathsf{R}}) \in \mathcal{Z}$. Define*

$$U_{\mathsf{L},z}(M) := \{\ell \in U_{\mathsf{L}} : \mathsf{Leak}_{i,h}^{\mathsf{L}}(\ell, M) = z_{\mathsf{L}}\},$$
$$U_{\mathsf{R},z}(M) := \{r \in U_{\mathsf{R}} : \mathsf{Leak}_{i,h}^{\mathsf{R}}(r, M) = z_{\mathsf{R}}\}.$$

The old states producing z are exactly the rectangle $U_{\mathsf{L},z}(M) \times U_{\mathsf{R},z}(M)$. These rectangles partition $U_{\mathsf{L}} \times U_{\mathsf{R}}$, because each old state produces exactly one answer pair. The left and right masks *of the answer z are functions $f_{\mathsf{L},z} : U_{\mathsf{L}} \times \mathsf{InvMatr}_2^n \to \{0,1\}$ and $f_{\mathsf{R},z} : U_{\mathsf{R}} \times \mathsf{InvMatr}_2^n \to \{0,1\}$, defined by*

$$f_{\mathsf{L},z}(\ell, M) := \begin{cases} 1, & \ell \in U_{\mathsf{L},z}(M), \\ 0, & \ell \notin U_{\mathsf{L},z}(M), \end{cases} \tag{3.1}$$

and

$$f_{\mathsf{R},z}(r, M) := \begin{cases} 1, & r \in U_{\mathsf{R},z}(M), \\ 0, & r \notin U_{\mathsf{R},z}(M). \end{cases} \tag{3.2}$$

We omit the fixed indices i and h for brevity. The rectangle partition is the identity

$$\sum_{z \in \mathcal{Z}} f_{\mathsf{L},z}(\ell, M) \cdot f_{\mathsf{R},z}(r, M) = 1 \quad \text{for every } \ell \in U_{\mathsf{L}},\ r \in U_{\mathsf{R}},\ M \in \mathsf{InvMatr}_2^n. \tag{3.3}$$

The identity in Eq. (3.3) says that every fixed old state and matrix produce exactly one answer. Averaging over the matrix then shows that the answer probabilities of a refresh sum to one. The bound K on the number of answers enters when estimates for single answers are summed.

The rectangle describes a set of inputs for each fixed matrix. It does not assert that the two parties' outputs are independent. The encoding correlates the two shares, and averaging over the common matrix can introduce further dependence. The approximation constructed in Sec. 4.3 below uses the masks separately for each matrix before controlling the average. The leakage strategy enters Sec. 4.3 only through the masks in Eqs. (3.1) and (3.2).

3.4 Removing randomness

For a fixed tape, the leakage functions above are deterministic. An adversary may, however, use randomness both when choosing the functions and inside them. We now explain why it suffices to prove security for deterministic leakage functions. The encodings and refresh matrices remain random throughout the proof. We fix only the adversary's tape and the presampled auxiliary choices described below while proving the bound, and then average over those choices. This lets us work with deterministic leakage functions without changing the randomized security claim.

Randomized adversaries. The adversary's random tape is independent of the secret, the initial encodings, and every refresh matrix. We fix an arbitrary tape throughout the proofs. Since the bound holds uniformly for every tape, taking the ordinary expectation over the possibly infinite tape preserves it, also when the tape is included in the adversary's view (see Thm. 4.1).

Leakage functions with private randomness. The two leakage functions of a refresh may also use fresh auxiliary randomness. The parties' auxiliary values may be correlated, with a joint

distribution depending on (i, h, M) but, conditional on this triple, independent of the shares. For the proof, we may sample all auxiliary randomness before the execution as follows. For every possible triple (i, h, M), sample a separate pair of auxiliary values, one for each party, from the joint distribution prescribed for that triple. Sample these pairs independently across triples and independently of the secret, initial encodings, and refresh matrices. In round i, the incoming history h and the sampled matrix M determine which presampled pair the two leakage functions use. Because the indexing triple includes the round number, none of the pairs indexed by round i is used in an earlier round. Consequently, conditional on the current triple (i, h, M), the selected pair still has the prescribed joint distribution and is independent of the current shares. Thus presampling leaves the execution distribution unchanged. For fixed presampled values, the leakage functions are deterministic functions of the local data and matrix, selected according to the preceding history. Averaging the deterministic security bounds therefore also covers these randomized functions.

4 Continual refreshing

With the leakage model in place, we can now prove that the matrix refresh of Def. 2.4 is secure. We first state the theorem and then prove it, working from a single refresh up to an entire execution. In round i, the matrix M_i takes the state $X_{i-1} = (L_{i-1}, R_{i-1})$ to $X_i = (L_i, R_i)$, where

$$L_i := L_{i-1} \cdot M_i, \qquad R_i := M_i^{-1} \cdot R_{i-1}.$$

When we analyze a single round, we drop the index and write (L, R) and (L', R') for its old and new shares and M for its matrix. The adversary observes only the leakage answers. The initial and final states remain private. The bit encoded by a state will be called its label. Def. 4.3 below makes this terminology precise. The proof has three steps. First, it approximates the leakage of one round by a *signed* table that retains both the old and the new state (see Lem. 4.13). Such a table is real-valued and may take negative values. Second, it bounds local replacement errors by the Cauchy–Schwarz inequality (see Lem. 2.7) and label changes by averaged factor energies (see Lems. 4.15 and 4.16). Third, it compares the executions for the two secrets by a hybrid argument: we introduce intermediate transcript weight functions, assign a bit to every state, and move the boundary between the states labeled 1 and those labeled 0 along the sequence of states (see Sec. 4.7 below).

4.1 Security statement

Security means that the complete leakage transcript is almost independent of the stored secret.

When the stored secret is s, let P_s^T denote the distribution of the complete transcript $H_T = (Z_1, \ldots, Z_T)$ of T refreshes. This distribution depends on the adversary's strategy for choosing leakage functions, which we omit from the notation for brevity. Quantitatively, we bound the statistical distance $\delta\left(P_0^T, P_1^T\right)$ (see Def. 2.2), which bounds the advantage of every distinguisher.

Theorem 4.1 (Security of continual refreshing)**.** *There exist absolute constants $C \geq 1$ and $C' \geq 1$ such that, for all integers $n \geq 2$, $T \geq 0$, and $\lambda \geq 0$, with $q := 2^n - 1$ and $K := 2^{2\lambda}$, every adversary in the leakage model of Sec. 3 satisfies*

$$\delta\left(P_0^T, P_1^T\right) \leq C \cdot T \cdot K^3 \cdot q^{-1/9}. \tag{4.1}$$

The bound also satisfies

$$C \cdot T \cdot K^3 \cdot q^{-1/9} \leq C' \cdot T \cdot 2^{6\lambda - n/9}. \tag{4.2}$$

The same bound holds when the adversary's independent random tape is included in its view.

The theorem applies to all T and λ, and it gives a nontrivial guarantee exactly when the right-hand side is below one. For $T = 0$, the transcript is empty, so the distance is zero. The proof is given at the end of Sec. 4.7.

4.2 An exact experiment with independent encodings

Our analysis rests on an exact description of an execution in which the states are sampled first, independently of one another, and the answers are drawn afterward. To set it up, we fix the adversary's tape as in Sec. 3 and define the conditional probabilities of the answers.

Definition 4.2 (Leakage probability functions)**.** *For every round $i \in \{1, \dots, T\}$, formal incoming history $h \in \mathcal{Z}^{i-1}$, answer $z \in \mathcal{Z}$, secret $s \in \mathbb{F}_2$, and states $x = (\ell, r)$ and $x' = (\ell', r')$ in Ω_s, define*

$$\mathsf{prob}_{i,h,z}(x, x') := \mathbb{P}_{M \leftarrow\$ \mathsf{InvMatr}_2^n}\Big[Z_i = z \mid \ell \cdot M = \ell',\ M^{-1} \cdot r = r'\Big]. \tag{4.3}$$

Here Z_i is the answer that the two leakage functions selected by i and h produce from the old state x and the matrix M. We call $\mathsf{prob}_{i,h,z} : \Omega_s \times \Omega_s \to [0, 1]$ a leakage probability function*. When the round, history, and answer are fixed, we may abbreviate it to* prob*.*

The conditioning event in Eq. (4.3) is nonempty, since x' is a possible refresh of x by Lem. 2.19. For every $(x, x') \in \Omega_s \times \Omega_s$, we have $\sum_{z \in \mathcal{Z}} \mathsf{prob}_{i,h,z}(x, x') = 1$. The value of $\mathsf{prob}_{i,h,z}$ for states that encode different bits is not defined, because a refresh never produces such a pair.

For each secret $s \in \mathbb{F}_2$, sample the states $X_0, \dots, X_T$ independently from μ_s, and then draw the answers $z_1, \dots, z_T$ in order, drawing z_i with probability $\mathsf{prob}_{i,(z_1,\dots,z_{i-1}),z_i}(X_{i-1}, X_i)$. This procedure gives exactly the real transcript distribution, for the following reason. A fresh matrix sends every fixed state to a uniform state that encodes the same bit (see Lem. 2.19), so the real states are independent samples from μ_s. For a fixed sequence of states, call the set of matrices that take X_{i-1} to X_i the *transition set* of round i. The matrix tuples compatible with the sequence form the Cartesian product of the transition sets, one set per round. Conditioning the initially independent uniform matrices on this product makes them independent and uniform on their respective transition sets. Choosing each round's leakage functions from the preceding answers then reproduces the real execution. In particular, for every complete history $h = (z_1, \dots, z_T) \in \mathcal{Z}^T$,

$$P_s^T(h) = \mathbb{E}_{(X_0,\dots,X_T) \leftarrow\$ \mu_s^{\otimes(T+1)}}\left[\prod_{i=1}^{T} \mathsf{prob}_{i,h_{i-1},z_i}(X_{i-1}, X_i)\right], \qquad h_{i-1} := (z_1, \dots, z_{i-1}). \tag{4.4}$$

The independence in Eq. (4.4) concerns the states before any answer is observed. The states need not remain independent after conditioning on the observed history. This distinction will matter when we apply the approximation error bound in Eq. (4.63) below, whose average uses independent endpoint encodings.

The hybrids of the proof also use states that encode different bits, which we describe by labels.

Definition 4.3 (Labels, valid rounds, and cheating rounds)**.** *A* label vector*, or* labeling*, $(s_0, \dots, s_T) \in \mathbb{F}_2^{T+1}$ specifies independent samples $X_i \leftarrow\$ \mu_{s_i}$ for $i = 0, \dots, T$. The* label *s_i is the bit encoded by state X_i. Round i is* valid *for these labels when $s_{i-1} = s_i$. Otherwise, we call it a* cheating round*, because the encoded bit changes across that round in the hybrid.*

This terminology describes the proof, not malicious behavior in the real protocol. The leakage probability functions $\mathsf{prob}_{i,h,z}$ are used only at valid rounds, and the signed approximations introduced below are defined even at cheating rounds. The real execution for secret s uses the constant label vector $(s, \dots, s)$, all of whose rounds are valid.

4.3 Approximating one refresh

We start with the first step of the proof, which concerns a single round: we replace the leakage probability function $\mathsf{prob}_{i,h,z}$ of an answer (see Def. 4.2) by a real table G that may take negative values. This table is a sum of products of a left-party function and a right-party function, with a bound on the total size of these products. We need this replacement because the hybrid argument changes the bit encoded by a state using the discrepancy bound of Lem. 2.20, whose cost grows with the nuclear norm of the table being averaged. For a fixed matrix, the answer mask is a product (see Def. 3.4), but the true function averages over matrices compatible with the four shares, and the result need not have a small nuclear norm. The table keeps both the old and the new state: its left argument is the pair (L, L') and its right argument is the pair (R, R'). Keeping the new shares is essential, because they become the old shares of the next round.

4.3.1 The four shares and their reference distribution

Orthonormal expansions and the nuclear norm treat the row and the column of a table as independent uniform coordinates, whereas the four shares of a refresh are related: the old and new pairs encode the same bit. We therefore define our tables on the full product of all left pairs and all right pairs, where these tools apply, and use a density to convert averages over this product into averages in a refresh. A refresh preserves the inner product, so tuples whose old and new pairs encode different bits cannot arise from a refresh. We therefore also introduce an operation that restricts a table to tuples whose two pairs encode the same bit.

Definition 4.4 (Four-share tuples and the one-round experiment)**.** *A* four-share tuple *is an element $\omega = ((\ell, \ell'), (r, r'))$ of $U_{\mathsf{L}}^2 \times U_{\mathsf{R}}^2$, where $U_{\mathsf{L}}^2 = U_{\mathsf{L}} \times U_{\mathsf{L}}$ and $U_{\mathsf{R}}^2 = U_{\mathsf{R}} \times U_{\mathsf{R}}$. The tuple is* consistent *when $\ell' \cdot r' = \ell \cdot r$. Let ν_{joint} be the uniform distribution on $U_{\mathsf{L}}^2 \times U_{\mathsf{R}}^2$, under which all four shares are independent and uniform on the nonzero shares. The* one-round experiment *samples L and R as independent uniform nonzero shares and M as an independent uniform invertible matrix, and sets $L' := L \cdot M$ and $R' := M^{-1} \cdot R$.*

We treat (L, L') and (R, R') as two *grouped* arguments: a table F on $U_{\mathsf{L}}^2 \times U_{\mathsf{R}}^2$ is written $F((\ell, \ell'), (r, r'))$, and its norms are those of Sec. 2.3 for the two sets U_{L}^2 and U_{R}^2, each with its uniform measure. The one-round experiment produces only consistent tuples, and its tuple ω has density φ relative to ν_{joint}: for each tuple, $\varphi(\omega)$ is its probability in the one-round experiment divided by its probability under ν_{joint}. Explicitly,

$$\varphi\left((\ell, \ell'), (r, r')\right) = \begin{cases} 1/p_{\ell \cdot r}, & \ell' \cdot r' = \ell \cdot r, \\ 0, & \ell' \cdot r' \neq \ell \cdot r. \end{cases} \tag{4.5}$$

Recall from Sec. 2.4 that p_s is the probability that two independent uniform nonzero shares encode the bit s. Eq. (4.5) holds because the old shares are independent and uniform, and for fixed old shares freshness (see Lem. 2.19) makes the new pair a uniform encoding of their bit.

Definition 4.5 (Consistency projection)**.** *For every real table* $F : U_{\mathsf{L}}^2 \times U_{\mathsf{R}}^2 \to \mathbb{R}$*, its* consistency projection *is the table on the same domain that discards the entries a refresh cannot produce:*

$$\big(\mathsf{project}(F)\big)\,((\ell,\ell'),(r,r')) := \begin{cases} F\,((\ell,\ell'),(r,r'))\,, & \ell' \cdot r' = \ell \cdot r, \\ 0, & \ell' \cdot r' \neq \ell \cdot r. \end{cases}$$

For example, the projection discards an entry whose old shares encode 0 and whose new shares encode 1. Applying the projection twice changes nothing.

4.3.2 An explicit approximation to one answer

We now express the conditional answer probability as a table on the full reference space, so that we can use its singular coefficients to construct an approximation. Fix a round $i \in \{1,\dots,T\}$, a formal incoming history $h \in \mathcal{Z}^{i-1}$, the adversary's tape, and an answer $z \in \mathcal{Z}$. We omit the fixed round and history indices in prob. Let $f_{\mathsf{L},z}(\ell, M)$ and $f_{\mathsf{R},z}(r, M)$ be the Boolean masks of z from Eqs. (3.1) and (3.2). For a consistent tuple $\omega = ((\ell,\ell'),(r,r'))$, put

$$\mathsf{prob}_z\,(\omega) := \mathbb{P}_{M \leftarrow\$ \mathsf{InvMatr}_2^n}\Big[f_{\mathsf{L},z}(\ell,M) = f_{\mathsf{R},z}(r,M) = 1 \mid \ell \cdot M = \ell',\ M^{-1} \cdot r = r'\Big], \tag{4.6}$$

and set $\mathsf{prob}_z\,(\omega) := 0$ on inconsistent tuples. By Lem. 2.19, every consistent tuple can arise from a refresh, so the conditional probability in Eq. (4.6) is well defined. The result is a table on the full product space with $0 \le \mathsf{prob}_z \le 1$ and $\mathsf{project}(\mathsf{prob}_z) = \mathsf{prob}_z$. At a cheating round, whose endpoint states encode different bits, the true probability function is not used. Instead, we use the approximation constructed below.

The discrepancy estimate is useful only if the table has a controlled nuclear norm. We obtain such a table by discarding small singular coefficients and then correcting for the fact that the true answer table is zero off consistent tuples. The example following the construction explains the correction and why we project only when measuring the error. Recall that $q = 2^n - 1$ is the number of nonzero shares. Take an orthonormal expansion $\mathsf{prob}_z = \sum_j \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}$ (see Lem. 2.10) with the probability normalization of Sec. 2.3: orthonormality refers to the uniform measures on U_{L}^2 and U_{R}^2, so each σ_j is q^{-2} times an ordinary singular value of the $q^2 \times q^2$ matrix of entries of prob_z. For $q^{-1/3} \le \beta \le 1$, keep only the coefficients strictly greater than β:

$$\mathsf{prob}_z^{>\beta} := \sum_{j:\,\sigma_j > \beta} \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}. \tag{4.7}$$

The superscript $> \beta$ refers to the singular coefficients, not to the entries of the table. Our approximation is $2 \cdot \mathsf{prob}_z^{>\beta}$. If no coefficient exceeds β, the retained family is empty and $\mathsf{prob}_z^{>\beta} = 0$. The construction depends only on the masks of the answer, not on the encoded bit. Define the *residual table* on the full four-share space by

$$\Delta_z := \mathsf{prob}_z - 2 \cdot \mathsf{project}\Big(\mathsf{prob}_z^{>\beta}\Big). \tag{4.8}$$

Thus Δ_z is the approximation error on consistent tuples, extended by zero elsewhere.

Why the factor two. Consider the case with no leakage: the answer alphabet has a single element z, so its probability is 1 for every actual refresh. The table prob_z also includes tuples that cannot arise from a refresh, and those entries are 0. Roughly half the entries describe possible refreshes, so the table contains roughly half 1s and half 0s. Replacing all entries by their average gives about

1/2, although the desired value on possible refreshes is 1. Doubling brings that value back to approximately 1. The projection in Eq. (4.8) ensures that we measure the error only on possible refreshes. For a general answer, the kept singular terms need not form a constant table, and Lem. 4.6 below proves the error bound for $2 \cdot \mathsf{prob}_z^{>\beta}$. The approximation itself remains defined on inconsistent tuples, where it is used later in the hybrid argument.

Lemma 4.6 (Approximation of an answer under refreshing)**.** *There exists an absolute constant $C > 0$ such that, for every dimension $n \geq 2$, every fixed formal incoming history and adversary tape, every leakage answer $z \in \mathcal{Z}$, and every threshold $\beta \in [q^{-1/3}, 1]$, the truncation in Eq.* (4.7) *and its residual satisfy*

$$\left\|\mathsf{prob}_z^{>\beta}\right\|_* \leq 2, \tag{4.9}$$

$$\left\|\mathsf{prob}_z^{>\beta}\right\|_2 \leq 1, \tag{4.10}$$

$$\|\Delta_z\|_2 \leq C \cdot \beta^{1/3}. \tag{4.11}$$

The balanced factors $\sqrt{\sigma_j} \cdot u_j$ and $\sqrt{\sigma_j} \cdot v_j$ of $\mathsf{prob}_z^{>\beta}$ have energies Φ_β and Ψ_β bounded at every input:

$$\Phi_\beta(\ell, \ell') \leq 1/\beta \qquad \text{for every } (\ell, \ell') \in U_{\mathsf{L}}^2, \tag{4.12}$$

$$\Psi_\beta(r, r') \leq 1/\beta \qquad \text{for every } (r, r') \in U_{\mathsf{R}}^2. \tag{4.13}$$

Since the factors are balanced, both mean energies equal $\left\|\mathsf{prob}_z^{>\beta}\right\|_*$ by Eq. (2.9)*, and hence are at most* 2 *by Eq.* (4.9)*. The same C applies to every dimension, threshold, answer, formal incoming history, and admissible pair of leakage functions.*

We now explain informally why the bounds of Lem. 4.6 hold. Sec. 4.3.3 gives the full proof.

The L_2 bound in Eq. (4.10) and the energy bounds in Eqs. (4.12) and (4.13) follow row by row. By Pythagoras' theorem (see Eq. (2.1)), the row of prob_z indexed by (ℓ, ℓ') has squared L_2 norm $\sum_j \sigma_j^2 \cdot u_j(\ell, \ell')^2$, which is at most one because $0 \leq \mathsf{prob}_z \leq 1$. Truncation only drops terms of this sum, so the rows of $\mathsf{prob}_z^{>\beta}$ also have squared norm at most one, which gives Eq. (4.10). Each kept coefficient exceeds β, so $\Phi_\beta(\ell, \ell') = \sum_{j:\, \sigma_j > \beta} \sigma_j \cdot u_j(\ell, \ell')^2 \leq 1/\beta$. The argument for columns is analogous.

The main step is the nuclear-norm bound in Eq. (4.9). Let W be the sum of the kept unit product tables $u_j \cdot v_j^{\mathsf{T}}$ (see Def. 2.6). By orthonormality, $\langle \mathsf{prob}_z, W \rangle$ is the sum of the kept coefficients, that is, $\left\|\mathsf{prob}_z^{>\beta}\right\|_*$. Since prob_z is the conditional probability of the answer given the four shares, this pairing is, up to a weight $p_{\ell \cdot r}$ within $1/(2q)$ of $1/2$, the average over ℓ, r, and M of $f_{\mathsf{L},z}(\ell, M) \cdot f_{\mathsf{R},z}(r, M)$ times W at the refreshed tuple $((\ell, \ell \cdot M), (r, M^{-1} \cdot r))$. For fixed M, the average over ℓ and r splits as $\sum_j c_j \cdot d_j$, where c_j is the inner product of the left mask $\ell \mapsto f_{\mathsf{L},z}(\ell, M)$ with $\ell \mapsto u_j(\ell, \ell \cdot M)$, and d_j is its right-side counterpart. If these functions of ℓ were orthonormal, Bessel's inequality (see Eq. (2.2)) would give $\sum_j c_j^2 \leq 1$, as the mask takes values in $\{0, 1\}$, and with the same bound for the d_j the sum would be at most 1. The functions are only approximately orthonormal: their Gram matrix (of pairwise inner products) equals the identity on average over M, and its mean squared deviation is at most the number of kept terms divided by $(q-1) \cdot \beta^2$, because M maps any two distinct nonzero shares to a uniform pair of distinct ones (see Lem. 4.7). The number of kept terms is at most $\left\|\mathsf{prob}_z^{>\beta}\right\|_* / \beta$, so with $q \cdot \beta^3 \geq 1$ we get an inequality with the nuclear norm on both sides,

$$2 \cdot \left\|\mathsf{prob}_z^{>\beta}\right\|_* \leq 2 + \sqrt{2 \cdot \left\|\mathsf{prob}_z^{>\beta}\right\|_*}, \tag{4.14}$$

which forces Eq. (4.9) (see Lem. 4.8). This argument relies on the mask structure of prob_z, so it does not apply to an arbitrary table F with $0 \le F \le 1$ and $\mathsf{project}(F) = F$.

Finally, consider the error bound in Eq. (4.11), with $t := \|\Delta_z\|_2$. The argument combines two properties of the residual. Its largest singular coefficient is controlled, limiting its correlation with product tests, and its squared norm is nearly its pairing with prob_z. The answer-mask identity, derived below in Eq. (4.16), expresses this pairing as a weighted average over M of the two masks multiplied by the residual evaluated at refreshed tuples. Together, these properties and the answer-mask identity prevent Δ_z from having a large norm. The mask products appear only after the refresh equations are substituted, however, so the argument must also carry the information about small coefficients from independent grouped coordinates to refreshed tuples. The four-cycle average, defined below in Eq. (4.41), provides this control. The consistency condition is encoded by the signs $(-1)^{\ell \cdot r + \ell' \cdot r'}$, which form a restriction of a binary sign matrix whose rows are orthogonal on the full bit-vector domain. The corresponding near-orthogonality to product tables is proved in Lem. 4.9 below. Hence $2 \cdot \mathsf{project}\big(\mathsf{prob}_z^{>\beta}\big)$ differs from $\mathsf{prob}_z^{>\beta}$ by a table nearly orthogonal to product tables. Since the discarded coefficients are at most β, the largest singular coefficient of Δ_z is at most 3β (see Lem. 4.9). A small largest coefficient alone does not make t small, as many small coefficients can have a large sum of squares. But truncation is an orthogonal projection, so the approximation is nearly orthogonal to its error, and t^2 equals $\langle \mathsf{prob}_z, \Delta_z \rangle$ up to a term of order $1/(q \cdot \beta)$. As for the nuclear norm, this pairing averages, over M, correlations of the two masks with Δ_z at refreshed tuples. Each correlation is at most the fourth root of the *four-cycle average* of the table: the mean product of its entries at the corners of a random rectangle (two random rows and two random columns). For Δ_z this average is at most $\sigma_{\max}(\Delta_z)^2 \cdot t^2 \le 9\beta^2 \cdot t^2$, and refreshing raises it, on average over M, by at most a constant factor plus small corrections (see Lem. 4.12). Together these estimates give

$$t^2 \le C \cdot \beta^{1/2} \cdot t^{1/2} + C/(q \cdot \beta), \tag{4.15}$$

and since $q \cdot \beta^3 \ge 1$, Eq. (4.15) implies the error bound in Eq. (4.11).

4.3.3 Proof of the refreshing approximation

We prove Lem. 4.6 in two steps:

- Bound the nuclear norm of $\mathsf{prob}_z^{>\beta}$ using the answer masks and the approximate orthonormality of the singular functions after refreshing.
- Bound the L_2 norm of the residual Δ_z by relating its squared norm to a correlation with the answer masks, then controlling that correlation through a four-cycle average.

The row and column projection argument also gives the pointwise energy bounds and the L_2 bound on $\mathsf{prob}_z^{>\beta}$. Throughout, fix a formal incoming history, the adversary's tape, an answer z, and a threshold $q^{-1/3} \le \beta \le 1$. Recall that $q = N - 1 \ge 3$ is the number of nonzero shares, where $N = 2^n$. We use the grouped coordinates $x = (\ell, \ell') \in U_{\mathsf{L}}^2$ and $y = (r, r') \in U_{\mathsf{R}}^2$. As in Sec. 2.3, all averages and norms use the uniform measures on their respective row and column sets unless another distribution is specified.

Relating a table to the leakage masks. For any table F on $U_{\mathsf{L}}^2 \times U_{\mathsf{R}}^2$ and fixed $M \in \mathsf{InvMatr}_2^n$, substituting the refresh equations gives a table on the old shares:

$$F^M(\ell, r) := F\big((\ell, \ell \cdot M), (r, M^{-1} \cdot r)\big).$$

Since a refresh produces only consistent tuples, $\big(\mathsf{project}(F)\big)^M = F^M$. Write $\|F\|_\infty := \max_{\ell,\ell',r,r'} |F((\ell,\ell'),(r,r'))|$ for its largest absolute entry. For any real table W on the four-share space, conditioning on the four shares yields the *answer-mask identity*

$$\langle \mathsf{prob}_z, W\rangle = \mathbb{E}_{\ell,r,M}\Big[p_{\ell\cdot r}\cdot f_{\mathsf{L},z}(\ell,M)\cdot f_{\mathsf{R},z}(r,M)\cdot W^M(\ell,r)\Big]. \tag{4.16}$$

Here ℓ and r are independent uniform nonzero shares, and M is independent of them and uniform in $\mathsf{InvMatr}_2^n$. The factor $p_{\ell\cdot r}$ cancels the density $\varphi = 1/p_{\ell\cdot r}$ of the one-round experiment on consistent tuples (see Eq. (4.5)). Recall from Eqs. (2.11) and (2.12) that $p_0 = (q-1)/(2q)$ and $p_1 = (q+1)/(2q)$, and hence $|2p_s - 1| = 1/q$. We will use Eq. (4.16) twice: first to bound the nuclear norm, and later with $W = \Delta_z$ to bound the approximation error.

Projection bounds. Write $I := \{j : \sigma_j(\mathsf{prob}_z) > \beta\}$ for the indices of the kept coefficients. Each row of $\mathsf{prob}_z^{>\beta}$ is the orthogonal projection of the corresponding row of prob_z onto the span of the kept right singular functions. Each column is similarly projected onto the span of the kept left singular functions. Pythagoras' theorem (see Eq. (2.1)) and $0 \le \mathsf{prob}_z \le 1$ give, for every row x,

$$\mathbb{E}_y\Big[\big(\mathsf{prob}_z^{>\beta}(x,y)\big)^2\Big] = \sum_{j\in I}\sigma_j^2\cdot u_j(x)^2 \le \mathbb{E}_y\Big[\big(\mathsf{prob}_z(x,y)\big)^2\Big] \le 1. \tag{4.17}$$

The same argument applies to every column, and averaging Eq. (4.17) over x gives Eq. (4.10). Since each kept σ_j exceeds β, we obtain the following bounds at every x and y:

$$e_u(x) := \sum_{j\in I} u_j(x)^2 \le \beta^{-2}, \tag{4.18}$$

$$e_v(y) := \sum_{j\in I} v_j(y)^2 \le \beta^{-2}, \tag{4.19}$$

$$\Phi_\beta(x) := \sum_{j\in I}\sigma_j\cdot u_j(x)^2 \le \beta^{-1},$$

$$\Psi_\beta(y) := \sum_{j\in I}\sigma_j\cdot v_j(y)^2 \le \beta^{-1}. \tag{4.20}$$

Orthonormality also gives $\mathbb{E}_x[e_u(x)] = \mathbb{E}_y[e_v(y)] = |I|$. Together with the pointwise bounds in Eqs. (4.18) and (4.19), this implies that both $\mathbb{E}_x[e_u(x)^2]$ and $\mathbb{E}_y[e_v(y)^2]$ are at most $|I|/\beta^2$. The unweighted sums e_u and e_v will control the Gram matrices below, that is, the matrices of pairwise inner products of the functions involved. The weighted sums Φ_β and Ψ_β are the factor energies required by Lem. 4.6.

Bounding the nuclear norm. The kept singular functions are orthonormal on the full grouped space, but their restrictions along a fixed refresh need not remain orthonormal. We must control this change before applying the mask bounds. The next lemma controls how much an average along a random refresh can differ from its mean. We will apply it to matrices of products $u_j(x)\cdot u_k(x)$, whose averages form the Gram matrix of the kept functions after refreshing.

Lemma 4.7 (Gram variance for an invertible refresh)**.** *Let $\mathbb{V}$ be any finite-dimensional real inner-product space with induced norm $\|\cdot\|$. For each $\ell \in U_\mathsf{L}$, let $g_\ell : U_\mathsf{L} \to \mathbb{V}$, and put*

$$\overline{g}_\ell := \mathbb{E}_{\ell'\leftarrow\$U_\mathsf{L}}[g_\ell(\ell')], \quad \widehat{g}_\ell(\ell') := g_\ell(\ell') - \overline{g}_\ell, \quad Y_M := \frac{1}{q}\sum_{\ell\in U_\mathsf{L}} g_\ell(\ell\cdot M).$$

For independent uniform $M \in \mathsf{InvMatr}_2^n$ and $M' \in \mathsf{InvMatr}_2^n$,

$$\mathbb{E}_M\Big[\|Y_M - \mathbb{E}_{M'}[Y_{M'}]\|^2\Big] = \frac{1}{q(q-1)}\sum_{\ell}\mathbb{E}_{\ell'}\Big[\|\widehat{g}_\ell(\ell')\|^2\Big] - \frac{1}{q^2(q-1)}\mathbb{E}_{\ell'}\left[\left\|\sum_{\ell\in U_{\mathsf{L}}}\widehat{g}_\ell(\ell')\right\|^2\right], \tag{4.21}$$

$$\mathbb{E}_M\Big[\|Y_M - \mathbb{E}_{M'}[Y_{M'}]\|^2\Big] \le \frac{1}{q-1}\mathbb{E}_{\ell,\ell'}\Big[\|g_\ell(\ell')\|^2\Big]. \tag{4.22}$$

All ℓ' averages are uniform on the nonzero rows and ℓ is uniform on U_{L}. The same statement holds for column images under M^{-1}.

Proof. Any two distinct nonzero binary rows are linearly independent, so for $\ell \neq \tilde{\ell}$ their images under M are a uniform ordered pair of distinct nonzero rows. Since every $\widehat{g}_\ell$ has mean zero, summing first over all ordered pairs and removing the equal pairs gives

$$\mathbb{E}_{(\ell',\tilde{\ell}')\leftarrow\$\{(u,u')\in U_{\mathsf{L}}^2 : u\neq u'\}}\Big[\Big\langle\widehat{g}_\ell(\ell'),\widehat{g}_{\tilde{\ell}}(\tilde{\ell}')\Big\rangle\Big] = -\frac{1}{q-1}\mathbb{E}_{\ell'}\left[\langle\widehat{g}_\ell(\ell'),\widehat{g}_{\tilde{\ell}}(\ell')\rangle\right]. \tag{4.23}$$

Expand the squared norm of $q^{-1}\sum_{\ell\in U_{\mathsf{L}}}\widehat{g}_\ell(\ell\cdot M)$ and use Eq. (4.23) for its off-diagonal terms. Adding and subtracting the diagonal terms inside the final square gives Eq. (4.21). Discard its nonpositive last term and use $\mathbb{E}_{\ell'}\Big[\|g_\ell(\ell')-\overline{g}_\ell\|^2\Big] \le \mathbb{E}_{\ell'}\Big[\|g_\ell(\ell')\|^2\Big]$ to obtain Eq. (4.22). Uniformity of M^{-1} and the same pair-transitivity argument prove the column version. □

Lemma 4.8 (Nuclear norm of the truncated answer table)**.** *Fix any formal incoming history and adversary tape. For every answer z and every $q^{-1/3} \le \beta \le 1$, the truncation defined in Eq.* (4.7) *satisfies $\left\|\mathsf{prob}_z^{>\beta}\right\|_* \le 2$.*

Proof. If I is empty, the claim is immediate. Otherwise, put $\tau := 2\left\|\mathsf{prob}_z^{>\beta}\right\|_*$ and $W := \sum_{j\in I} u_j\cdot v_j^{\mathsf{T}}$. Then $\langle\mathsf{prob}_z, W\rangle = \tau/2$ and, by Eqs. (4.18) and (4.19), $\|W\|_\infty \le \beta^{-2}$. Write $u(x) := (u_j(x))_{j\in I}$ and $v(y) := (v_j(y))_{j\in I}$ as column vectors of coefficients, and define

$$\mathrm{Gram}_{\mathsf{L},M} := \mathbb{E}_{\ell\leftarrow\$U_{\mathsf{L}}}\Big[u(\ell,\ell\cdot M)\cdot u(\ell,\ell\cdot M)^{\mathsf{T}}\Big]. \tag{4.24}$$

Define $\mathrm{Gram}_{\mathsf{R},M}$ analogously with $v(r, M^{-1}\cdot r)$. The entries of $\mathrm{Gram}_{\mathsf{L},M}$ in Eq. (4.24) are the pairwise inner products of the functions $\ell \mapsto u_j(\ell,\ell\cdot M)$, so it is their *Gram matrix*. The same interpretation applies on the right. For coefficient vectors and matrices only, $\|\cdot\|_{\mathrm{E}}$ denotes the square root of the sum of squared entries, without averaging. Apply Lem. 4.7 in this inner-product space of coefficient matrices, with $g_\ell(\ell') := u(\ell,\ell')\cdot u(\ell,\ell')^{\mathsf{T}}$. The mean Gram matrix is the $|I|\times|I|$ identity matrix Id by orthonormality, and $\left\|u\cdot u^{\mathsf{T}}\right\|_{\mathrm{E}}^2 = e_u^2$. Consequently, on each side,

$$\mathbb{E}_M\Big[\|\mathrm{Gram}_{\mathsf{L},M} - \mathrm{Id}\|_{\mathrm{E}}^2\Big] \le \frac{|I|}{(q-1)\cdot\beta^2}, \tag{4.25}$$

$$\mathbb{E}_M\Big[\|\mathrm{Gram}_{\mathsf{R},M} - \mathrm{Id}\|_{\mathrm{E}}^2\Big] \le \frac{|I|}{(q-1)\cdot\beta^2}. \tag{4.26}$$

For each fixed M, define coefficient vectors $c = (c_j)_{j\in I}$ and $d = (d_j)_{j\in I}$ by

$$c_j := \mathbb{E}_\ell[f_{\mathsf{L},z}(\ell, M) \cdot u_j(\ell, \ell \cdot M)],$$
$$d_j := \mathbb{E}_r\Big[f_{\mathsf{R},z}(r, M) \cdot v_j(r, M^{-1} \cdot r)\Big].$$

For any coefficient vector ξ of ordinary Euclidean norm one, the Cauchy–Schwarz inequality (see Lem. 2.7) and $0 \le f_{\mathsf{L},z} \le 1$ give

$$|\xi^{\mathsf{T}} \cdot c|^2 \le \xi^{\mathsf{T}} \cdot \mathrm{Gram}_{\mathsf{L},M} \cdot \xi \le 1 + \|\mathrm{Gram}_{\mathsf{L},M} - \mathrm{Id}\|_{\mathrm{E}}. \tag{4.27}$$

Maximizing Eq. (4.27) over ξ gives the same bound on $\|c\|_{\mathrm{E}}^2$, and likewise for d. Since the average of $f_{\mathsf{L},z} \cdot f_{\mathsf{R},z} \cdot W^M$ at fixed M is $c^{\mathsf{T}} \cdot d$, the Cauchy–Schwarz inequality (see Lem. 2.7) over M and Eqs. (4.25) and (4.26) yield

$$\mathbb{E}_{\ell,r,M}\Big[f_{\mathsf{L},z}(\ell, M) \cdot f_{\mathsf{R},z}(r, M) \cdot W^M(\ell, r)\Big] \le 1 + \sqrt{\frac{|I|}{(q-1)\cdot\beta^2}}. \tag{4.28}$$

This step uses no independence between the two Gram matrices. Using Eqs. (4.16) and (4.28), $|2p_s - 1| = 1/q$, and $\|W\|_\infty \le \beta^{-2}$, we get

$$\tau \le 1 + \sqrt{\frac{|I|}{(q-1)\cdot\beta^2}} + \frac{1}{q\cdot\beta^2} \le 1 + \sqrt{\frac{1}{2(q-1)\cdot\beta^3}} \cdot \sqrt{\tau} + \frac{1}{q\cdot\beta^2}, \tag{4.29}$$

where $|I| \cdot \beta \le \left\|\mathsf{prob}_z^{>\beta}\right\|_* = \tau/2$. Since $q \cdot \beta^3 \ge 1$ and $q \ge 3$, the coefficient of $\sqrt{\tau}$ and the last term in Eq. (4.29) are at most one. Thus Eq. (4.29) gives $\tau \le 2 + \sqrt{\tau}$, the inequality in Eq. (4.14), implying $\tau \le 4$. The balanced factors of $\mathsf{prob}_z^{>\beta}$ therefore have mean energies $\tau/2 \le 2$, while their pointwise bounds $1/\beta$ and the bound $\left\|\mathsf{prob}_z^{>\beta}\right\|_2 \le 1$ follow from the projection bounds. □

Cancellation from consistency. The remaining task is to bound the residual $\Delta_z = \mathsf{prob}_z - 2 \cdot \mathsf{project}\Big(\mathsf{prob}_z^{>\beta}\Big)$. To separate the effect of the consistency projection from that of truncation, define the real table

$$J\big((\ell,\ell'),(r,r')\big) := \begin{cases} 1/2, & \ell \cdot r = \ell' \cdot r', \\ -1/2, & \ell \cdot r \ne \ell' \cdot r'. \end{cases} \tag{4.30}$$

For any real table F on the same domain,

$$\mathsf{project}(F) = F/2 + J \cdot F, \tag{4.31}$$

where $J \cdot F$ denotes entrywise multiplication. The following lemma shows that the positive and negative entries of J in Eq. (4.30) almost cancel when paired with a product of a left function and a right function.

Lemma 4.9 (Cancellation of the consistency sign)**.** *For arbitrary real functions u on U_{L}^2 and v on U_{R}^2,*

$$\Big|\Big\langle J, u \cdot v^{\mathsf{T}}\Big\rangle\Big| \le \frac{N}{2q^2}\,\|u\|_2\,\|v\|_2 \le \frac{1}{q}\,\|u\|_2\,\|v\|_2. \tag{4.32}$$

Fix any formal incoming history and adversary tape. For every answer z, every $\beta \in [q^{-1/3}, 1]$, and all functions $u : U_{\mathsf{L}}^2 \to \mathbb{R}$ and $v : U_{\mathsf{R}}^2 \to \mathbb{R}$ with $\|u\|_2 = \|v\|_2 = 1$,

$$\Big|\Big\langle J \cdot \mathsf{prob}_z^{>\beta}, u \cdot v^{\mathsf{T}}\Big\rangle\Big| \le \frac{1}{q\cdot\beta}. \tag{4.33}$$

Proof. The sign table $2J$ is the submatrix of $(-1)^{\ell \cdot r + \ell' \cdot r'}$ indexed by (ℓ, ℓ') and (r, r'). Extend functions by zero to all pairs of n-bit vectors. The grouped coordinates form subsets of this full domain. On the full $2n$-bit domain, distinct binary sign rows are orthogonal: the sum of a nontrivial binary sign is zero by pairing vectors that differ in a coordinate where its coefficient is one. For the full sign matrix Υ^{full}, this gives $\Upsilon^{\text{full}} \cdot (\Upsilon^{\text{full}})^{\mathsf{T}} = N^2 \cdot \mathrm{Id}$. The Cauchy–Schwarz inequality (see Lem. 2.7) therefore bounds the unnormalized pairing by N times the two Euclidean norms. Each grouped domain here has q^2 points. Dividing the pairing by q^4 and converting each Euclidean norm to its probability-normalized L_2 norm gives Eq. (4.32). Finally $N/(2q) \le 1$.
For the last assertion, write $\mathsf{prob}_z^{>\beta} = \sum_j a_j \cdot b_j^{\mathsf{T}}$ with its balanced factors. Apply Eq. (4.32) to $u \cdot a_j$ and $v \cdot b_j$ and then the Cauchy–Schwarz inequality (see Lem. 2.7) over j:

$$\left|\left\langle J \cdot \mathsf{prob}_z^{>\beta}, u \cdot v^{\mathsf{T}} \right\rangle\right| \le \frac{1}{q} \sqrt{\mathbb{E}_x[u(x)^2 \cdot \Phi_\beta(x)]} \cdot \sqrt{\mathbb{E}_y[v(y)^2 \cdot \Psi_\beta(y)]} \le \frac{1}{q \cdot \beta}.$$

□

The residual and its exact identity. Applying Eq. (4.31) to the table $\mathsf{prob}_z^{>\beta}$ rewrites the residual in Eq. (4.8) as

$$\Delta_z = \mathsf{prob}_z - 2 \cdot \mathsf{project}\left(\mathsf{prob}_z^{>\beta}\right) = (\mathsf{prob}_z - \mathsf{prob}_z^{>\beta}) - 2 \cdot J \cdot \mathsf{prob}_z^{>\beta}. \tag{4.34}$$

Put $t := \|\Delta_z\|_2$. The residual is supported on consistent tuples and equals $\mathsf{prob}_z - 2 \cdot \mathsf{prob}_z^{>\beta}$ there. Recall that $\sigma_{\max}$ denotes the largest singular coefficient (see Def. 2.14). The discarded singular coefficients are at most β, so applying Eqs. (2.7) and (4.33) to Eq. (4.34) gives

$$\sigma_{\max}(\Delta_z) \le \beta + \frac{2}{q \cdot \beta} \le 3\beta, \tag{4.35}$$

$$\|\Delta_z\|_\infty \le 3/\beta. \tag{4.36}$$

To obtain Eq. (4.35), check the pairing bound on every unit product table and then on a pair of singular functions of Δ_z. The entry bound Eq. (4.36) follows from $|\mathsf{prob}_z^{>\beta}(x,y)| \le \sqrt{\Phi_\beta(x) \cdot \Psi_\beta(y)} \le 1/\beta$ and $|\mathsf{prob}_z| \le 1$. Every row and column of Δ_z has L_2 norm at most three: prob_z has norm at most one, $\mathsf{prob}_z^{>\beta}$ has norm at most one on each row and column, and the consistency projection contracts each such norm. These bounds alone do not yet show that t is small: a table can have many small singular coefficients whose squares have a large sum. We therefore use an identity specific to this residual. The support of prob_z and orthogonality of the kept terms give

$$\left\langle \mathsf{prob}_z, \mathsf{project}\left(\mathsf{prob}_z^{>\beta}\right) \right\rangle = \left\langle \mathsf{prob}_z, \mathsf{prob}_z^{>\beta} \right\rangle = \left\| \mathsf{prob}_z^{>\beta} \right\|_2^2. \tag{4.37}$$

Also, because the consistency projection leaves each entry unchanged or sets it to zero,

$$\left\| \mathsf{project}\left(\mathsf{prob}_z^{>\beta}\right) \right\|_2^2 = \frac{1}{2} \cdot \left\| \mathsf{prob}_z^{>\beta} \right\|_2^2 + \left\langle \mathsf{prob}_z^{>\beta}, J \cdot \mathsf{prob}_z^{>\beta} \right\rangle. \tag{4.38}$$

Expanding $\|\Delta_z\|_2^2$ and using Eqs. (4.37) and (4.38) yields

$$t^2 = \langle \mathsf{prob}_z, \Delta_z \rangle + 4 \left\langle \mathsf{prob}_z^{>\beta}, J \cdot \mathsf{prob}_z^{>\beta} \right\rangle. \tag{4.39}$$

Indeed, both sides expand to $\|\mathsf{prob}_z\|_2^2 - 2\left\|\mathsf{prob}_z^{>\beta}\right\|_2^2 + 4\left\langle \mathsf{prob}_z^{>\beta}, J \cdot \mathsf{prob}_z^{>\beta}\right\rangle$. Applying Eq. (4.33) to each normalized product in the expansion of $\mathsf{prob}_z^{>\beta}$ yields

$$\left|\left\langle \mathsf{prob}_z^{>\beta}, J \cdot \mathsf{prob}_z^{>\beta}\right\rangle\right| \le \frac{\left\|\mathsf{prob}_z^{>\beta}\right\|_*}{q \cdot \beta} \le \frac{2}{q \cdot \beta}. \tag{4.40}$$

It remains to bound $\langle \mathsf{prob}_z, \Delta_z\rangle$. The answer-mask identity Eq. (4.16) expresses this pairing through Δ_z^M, the residual evaluated along an actual refresh. The largest-coefficient bound concerns independent grouped coordinates. It does not directly control the correlation with masks after those coordinates are restricted along a fixed refresh. We instead bound this correlation on average over the matrix. The four-cycle average introduced next bounds correlation with two masks at each fixed matrix, admits a counting argument when averaged over matrices, and is controlled by the largest singular coefficient under independent sampling.

Four-cycle averages. For a real table F on a product of two finite nonempty sets, let ν_{L} and ν_{R} denote the uniform measures on its row and column sets. Its *four-cycle average* is

$$\mathrm{Cyc}(F) := \mathbb{E}_{(x_1,x_2,y_1,y_2)\leftarrow\$\nu_{\mathsf{L}}^{\otimes 2}\otimes\nu_{\mathsf{R}}^{\otimes 2}}[F(x_1,y_1)\cdot F(x_1,y_2)\cdot F(x_2,y_1)\cdot F(x_2,y_2)]. \tag{4.41}$$

The four entries in Eq. (4.41) are the corners of a rectangle formed by two independently sampled rows and two independently sampled columns. Expanding an orthonormal decomposition from Lem. 2.10 gives $\mathrm{Cyc}(F) = \sum_j \sigma_j(F)^4$. In particular, the average is nonnegative even when individual products are negative, and

$$0 \le \mathrm{Cyc}(F) \le \sigma_{\max}(F)^2 \cdot \|F\|_2^2 . \tag{4.42}$$

For arbitrary real functions f and g on the row and column sets, two applications of the Cauchy–Schwarz inequality (see Lem. 2.7) give

$$|\mathbb{E}_{x,y}[f(x)\cdot F(x,y)\cdot g(y)]| \le \|f\|_2 \cdot \|g\|_2 \cdot \mathrm{Cyc}(F)^{1/4}. \tag{4.43}$$

Indeed, first remove f by the Cauchy–Schwarz inequality (see Lem. 2.7) over x. The remaining squared expression is the pairing of $g(y)\cdot g(y')$ with $\mathbb{E}_x[F(x,y)\cdot F(x,y')]$, and the Cauchy–Schwarz inequality over (y,y') bounds it by $\|g\|_2^2 \cdot \sqrt{\mathrm{Cyc}(F)}$. For Boolean masks, both function norms are at most one, so the fourth root of $\mathrm{Cyc}(F)$ alone bounds their correlation with F.

Uniform binary matrices. The four-cycle calculation is simpler for an unrestricted uniform matrix, whose entries are independent. A singular matrix has no inverse, so we will first express the refresh constraints by equations on an enlarged table. The complete four-cycle of that table is nonnegative, allowing us to compare its average over invertible matrices with its average over all matrices. The next two lemmas supply this comparison and count the linear constraints in the unrestricted average.

Lemma 4.10 (Comparison with unrestricted matrices)**.** *Let A_0 be uniform on $\mathbb{F}_2^{n\times n}$ and M uniform on $\mathsf{InvMatr}_2^n$. Then*

$$\mathbb{P}[A_0 \text{ is invertible}] = \prod_{j=1}^{n}\left(1-2^{-j}\right) > \frac{1}{4}, \tag{4.44}$$

and for every nonnegative function g on all binary $n\times n$ matrices,

$$\mathbb{E}_M[g(M)] \le 4\mathbb{E}_{A_0}[g(A_0)]. \tag{4.45}$$

Proof. The jth row of an invertible matrix must avoid the span of the first $j-1$ rows, which has 2^{j-1} elements, so the probability that a uniform matrix is invertible is $\prod_{j=1}^{n}\left(1-2^{-n+j-1}\right)=\prod_{j=1}^{n}\left(1-2^{-j}\right)$. Since $\prod_{j=2}^{n}\left(1-2^{-j}\right)\geq 1-\sum_{j=2}^{n}2^{-j}>1/2$, this probability exceeds $1/4$. Conditioned on invertibility, A_0 is uniform on $\mathsf{InvMatr}_2^n$, and $g\geq 0$ gives $\mathbb{E}_{A_0}[g(A_0)]\geq\mathbb{P}[A_0 \text{ is invertible}]\cdot\mathbb{E}_M[g(M)]$. □

Lemma 4.11 (Prescribed rows and columns)**.** *Let x_1 and x_2 be rows in $\mathbb{F}_2^{1\times n}$, and let v_1 and v_2 be columns in $\mathbb{F}_2^{n\times 1}$. Let r_x and r_v be the ranks of the lists (x_1,x_2) and (v_1,v_2), respectively. Let u_1 and u_2 be prescribed rows in $\mathbb{F}_2^{1\times n}$, and let c_1 and c_2 be prescribed columns in $\mathbb{F}_2^{n\times 1}$. For uniform $A_0\in\mathbb{F}_2^{n\times n}$,*

$$\mathbb{P}_{A_0}[x_i\cdot A_0=u_i \text{ and } A_0\cdot v_j=c_j \text{ for all } i,j\in\{1,2\}]=\begin{cases}2^{r_xr_v}N^{-(r_x+r_v)}, & \text{if consistent,}\\ 0, & \text{otherwise.}\end{cases}\tag{4.46}$$

The prescriptions are consistent *when every linear relation among the x_i holds among the u_i, every linear relation among the v_j holds among the c_j, and $u_i\cdot v_j=x_i\cdot c_j$ for all $i,j\in\{1,2\}$.*

Proof. Choose a maximal independent sublist of the rows and one of the columns, with r_x and r_v elements. The conditions for the remaining indices follow from the relation conditions. An invertible change of basis on each side maps a uniform matrix to a uniform matrix and turns the chosen rows and columns into standard basis vectors. The row conditions then fix r_x full rows of A_0 and the column conditions fix r_v full columns. The r_xr_v entries where a fixed row meets a fixed column are prescribed twice, and the two prescriptions agree exactly when $u_i\cdot v_j=x_i\cdot c_j$ for the chosen indices. All other pairings follow from the relation conditions. Hence consistent prescriptions fix exactly $r_xn+r_vn-r_xr_v$ independent binary entries. The probability of satisfying them is $2^{-(r_xn+r_vn-r_xr_v)}=2^{r_xr_v}N^{-(r_x+r_v)}$, as stated in Eq. (4.46). Inconsistent prescriptions have probability zero. □

The bound on $\sigma_{\max}(\Delta_z)$ controls $\mathrm{Cyc}(\Delta_z)$ under independent sampling. The next lemma transfers this control to $\mathrm{Cyc}\left(\Delta_z^M\right)$, as required by the answer-mask identity. The bound in Eq. (4.47) below includes correction terms for repeated rows or columns in the four-cycle expansion.

Lemma 4.12 (Four cycles after a refresh)**.** *Let $\Delta:U_{\mathsf{L}}^2\times U_{\mathsf{R}}^2\to\mathbb{R}$ be any real table supported on consistent tuples. Put $t:=\|\Delta\|_2$ and $B_\infty:=\|\Delta\|_\infty$, and let $B_2\geq 0$ be a real number bounding the L_2 norm of every row and every column. Then*

$$\mathbb{E}_{M\leftarrow\$\mathsf{InvMatr}_2^n}\left[\mathrm{Cyc}\left(\Delta^M\right)\right]\leq 64\mathrm{Cyc}(\Delta)+\frac{32B_2^2}{q}\cdot t^2+\frac{72B_\infty^2}{q^2}\cdot t^2.\tag{4.47}$$

Proof. Allowing singular matrices. Extend Δ by zero whenever ℓ' or r' is zero, obtaining $\widehat{\Delta}$. In the extended averages, ℓ and r remain uniform on U_{L} and U_{R}, while ℓ' and r' range independently over all N binary rows and columns. Then

$$\widehat{t}^2:=\mathbb{E}_{\ell,\ell',r,r'}\left[\widehat{\Delta}((\ell,\ell'),(r,r'))^2\right]=(q/N)^2\cdot t^2,\tag{4.48}$$

$$\mathrm{Cyc}\left(\widehat{\Delta}\right)=(q/N)^4\cdot\mathrm{Cyc}(\Delta).\tag{4.49}$$

The rows and columns of $\widehat{\Delta}$, with these measures, still have L_2 norm at most B_2. For a possibly singular matrix we cannot use $A_0^{-1}\cdot r$. Instead, we keep r' as part of the column coordinate and impose the equation $A_0\cdot r'=r$. For an unrestricted uniform binary matrix A_0, define

$$\widetilde{\Delta}_{A_0}(\ell,(r,r')):=\begin{cases}\sqrt{N}\cdot\widehat{\Delta}((\ell,\ell\cdot A_0),(r,r')), & A_0\cdot r'=r,\\ 0, & \text{otherwise.}\end{cases}\tag{4.50}$$

The table in Eq. (4.50) has row measure uniform on U_{L} and column measure uniform on $U_{\mathsf{R}} \times \mathbb{F}_2^{n\times 1}$. For invertible M, each r permits exactly one $r' = M^{-1}\cdot r$. Expanding the complete four-cycle shows that the two r' averages contribute N^{-2}, exactly canceled by the four factors $\sqrt{N}$. Hence $\mathrm{Cyc}\big(\widetilde{\Delta}_M\big) = \mathrm{Cyc}\big(\Delta^M\big)$. By nonnegativity of $\mathrm{Cyc}\big(\widetilde{\Delta}_{A_0}\big)$ and Eq. (4.45),

$$\mathbb{E}_M\Big[\mathrm{Cyc}\big(\Delta^M\big)\Big] \le 4\mathbb{E}_{A_0}\Big[\mathrm{Cyc}\big(\widetilde{\Delta}_{A_0}\big)\Big]. \tag{4.51}$$

We apply Eq. (4.45) to the complete four-cycle, which is nonnegative. An individual product of four entries may be negative.

Separating the four rank cases. Expanding the four-cycle and inserting independent uniform rows ℓ_1' and ℓ_2' in $\mathbb{F}_2^{1\times n}$ gives

$$\begin{aligned}&\mathbb{E}_{A_0}\Big[\mathrm{Cyc}\big(\widetilde{\Delta}_{A_0}\big)\Big]\\ &\quad= N^4\mathbb{E}_{\ell_1,\ell_2,\ell_1',\ell_2',r_1,r_2,r_1',r_2'}\left[\begin{array}{l}\displaystyle\prod_{i,j=1}^{2}\widehat{\Delta}((\ell_i,\ell_i'),(r_j,r_j'))\\ \cdot\,\mathbb{P}_{A_0}\Big[\ell_i\cdot A_0=\ell_i',\ A_0\cdot r_j'=r_j\ (i,j=1,2)\Big]\end{array}\right].\end{aligned} \tag{4.52}$$

The factor N^4 in Eq. (4.52) consists of N^2 from the four factors $\sqrt{N}$ in $\widetilde{\Delta}_{A_0}$ and N^2 compensating for the two newly sampled rows ℓ_1' and ℓ_2'. The pairing constraints in Lem. 4.11 are enforced by the support of $\widehat{\Delta}$. Terms with $\ell_i' = 0$ or $r_j' = 0$ vanish by the zero extension. Over $\mathbb{F}_2$, rank one for the two old rows means $\ell_1 = \ell_2$, which forces $\ell_1' = \ell_2'$. Rank one for the two nonzero columns means $r_1' = r_2'$, which forces $r_1 = r_2$ by the prescribed column equations. For the next display, $\mathbb{E}[g;\mathcal{E}]$ means the unnormalized restricted average: the quantity being averaged is g on $\mathcal{E}$ and zero otherwise. All listed variables are still independent and uniform on the full spaces specified above. Distinct old rows and distinct new columns give the main four-cycle term. Equal rows or columns give products of squares. Applying Eq. (4.46) to the four rank cases in Eq. (4.52) gives the exact decomposition

$$\begin{aligned}&\mathbb{E}_{A_0}\Big[\mathrm{Cyc}\big(\widetilde{\Delta}_{A_0}\big)\Big]\\ &= 16\mathbb{E}_{\ell_1,\ell_2,\ell_1',\ell_2',r_1,r_2,r_1',r_2'}\left[\prod_{i,j=1}^{2}\widehat{\Delta}((\ell_i,\ell_i'),(r_j,r_j'));\ \ell_1\neq\ell_2,\ r_1'\neq r_2'\right]\\ &+\frac{4}{q}\mathbb{E}_{\ell,\ell',r_1,r_2,r_1',r_2'}\Big[\widehat{\Delta}((\ell,\ell'),(r_1,r_1'))^2\widehat{\Delta}((\ell,\ell'),(r_2,r_2'))^2;\ r_1'\neq r_2'\Big]\\ &+\frac{4}{q}\mathbb{E}_{\ell_1,\ell_2,\ell_1',\ell_2',r,r'}\Big[\widehat{\Delta}((\ell_1,\ell_1'),(r,r'))^2\widehat{\Delta}((\ell_2,\ell_2'),(r,r'))^2;\ \ell_1\neq\ell_2\Big]\\ &+\frac{2}{q^2}\mathbb{E}_{\ell,\ell',r,r'}\Big[\widehat{\Delta}((\ell,\ell'),(r,r'))^4\Big].\end{aligned} \tag{4.53}$$

For example, rank $(1,2)$ has weight $4N$ after multiplication by N^4. The equalities $\ell_1 = \ell_2$ and $\ell_1' = \ell_2'$ cost $1/q$ and $1/N$, giving $4/q$. Rank $(2,1)$ costs $1/N$ and $1/q$ in the same way, and rank $(1,1)$ has weight $2N^2$ and four equalities of joint probability $1/(q^2N^2)$. The main term excludes only ℓ- and r'-collisions. Collisions in ℓ' and r remain allowed.

Bounding the four contributions. Apply inclusion–exclusion to the two restrictions in the main term. The contribution with $\ell_1 = \ell_2$ is nonnegative: for each fixed ℓ, it is a complete four-cycle of the sliced table with row coordinate ℓ' and column coordinate (r,r'), multiplied by $1/q$. Likewise the contribution with $r_1' = r_2'$ is a nonnegative complete four-cycle with row coordinate (ℓ,ℓ') and column

coordinate r, averaged over r' and multiplied by $1/N$. Their subtraction therefore only lowers the upper bound. The intersection is bounded by $B_\infty^2 \widehat{t}^2/(q \cdot N)$: for four entries e_i bounded by B_∞,

$$|e_1 \cdot e_2 \cdot e_3 \cdot e_4| \le \frac{B_\infty^2}{4} \sum_{i=1}^{4} e_i^2,$$

and conditioned on both collisions every individual entry retains its full product marginal. Thus the first term on the right-hand side of Eq. (4.53) is at most $16\mathrm{Cyc}\Big(\widehat{\Delta}\Big) + 16B_\infty^2 \widehat{t}^2/(q \cdot N)$.

Drop the remaining restrictions in the two mixed-rank terms, whose summands are nonnegative. For a fixed row, its mean squared entry is at most B_2^2, so the square of this mean is at most B_2^2 times the mean itself. Averaging over rows bounds the first mixed-rank term by $(4B_2^2/q)\widehat{t}^2$. The other has the same bound using columns. The final term is at most $(2B_\infty^2/q^2)\widehat{t}^2$. Combining the four estimates, multiplying by four in Eq. (4.51), and substituting Eqs. (4.48) and (4.49) with $q/N \le 1$ and $N > q$ gives Eq. (4.47): the coefficients are $4 \cdot 16 = 64$, $4 \cdot 8 = 32$, and $4 \cdot (16 + 2) = 72$ after bounding $1/(q \cdot N)$ by $1/q^2$. □

Completion of the proof of Lem. 4.6. Apply Lem. 4.12 to the residual Δ_z, with $t = \|\Delta_z\|_2$, $B_2 = 3$, and $B_\infty \le 3/\beta$. By Eqs. (4.35) and (4.42), $\mathrm{Cyc}(\Delta_z) \le 9\beta^2 \cdot t^2$. Since $q \cdot \beta^3 \ge 1$ and $\beta \le 1$, both q^{-1} and $q^{-2} \cdot \beta^{-2}$ are at most β^2. It follows that

$$\mathbb{E}_M\Big[\mathrm{Cyc}\Big(\Delta_z^M\Big)\Big] \le C \cdot \beta^2 \cdot t^2. \tag{4.54}$$

For each M, the test bound Eq. (4.43) bounds the pairing with the two masks by $\mathrm{Cyc}\Big(\Delta_z^M\Big)^{1/4}$. Averaging over M and applying Jensen's inequality with Eq. (4.54) gives

$$\Big|\mathbb{E}_{\ell,r,M}\Big[f_{\mathsf{L},z}(\ell, M) \cdot f_{\mathsf{R},z}(r, M) \cdot \Delta_z^M(\ell, r)\Big]\Big| \le C \cdot \beta^{1/2} \cdot t^{1/2}. \tag{4.55}$$

Apply the answer-mask identity in Eq. (4.16) with $W = \Delta_z$ and replace $p_{\ell \cdot r}$ by $1/2$. The additional absolute error is at most $\|\Delta_z\|_\infty / (2q) \le 3/(2q \cdot \beta)$. Consequently, Eq. (4.55) gives

$$|\langle \mathsf{prob}_z, \Delta_z \rangle| \le C \cdot \beta^{1/2} \cdot t^{1/2} + \frac{3}{2q \cdot \beta}. \tag{4.56}$$

Combining Eqs. (4.39), (4.40) and (4.56) gives

$$t^2 \le C \cdot \beta^{1/2} \cdot t^{1/2} + \frac{C}{q \cdot \beta} \le C \cdot \beta^{1/2} \cdot t^{1/2} + C \cdot \beta^2. \tag{4.57}$$

If the second term on the final right-hand side of Eq. (4.57) is at least $t^2/2$, then $t \le C \cdot \beta \le C \cdot \beta^{1/3}$. Otherwise, the first term is at least $t^2/2$. For $t > 0$, divide by $t^{1/2}$ to obtain $t \le C \cdot \beta^{1/3}$. The case $t = 0$ is immediate. This argument covers the case where no coefficient is kept and every threshold in the stated range, including $\beta = 1$, for every $n \ge 2$. □

4.3.4 One family of approximations for both secrets

The estimates just proved use independent uniform grouped coordinates. To compare executions, we instead sample encodings of prescribed bits and need the same approximation while those bits change. We now transfer the estimates to these encoding distributions. For states $X = (\ell, r)$ and $X' = (\ell', r')$, write $G(X, X')$ for $G((\ell, \ell'), (r, r'))$. For any distribution ρ, use $\|F\|_{2,\rho} := \big(\mathbb{E}_{x \leftarrow\$ \rho}\big[|F(x)|^2\big]\big)^{1/2}$ for the root-mean-square value of F under ρ. The unqualified norms continue to use independent uniform grouped coordinates.

Lemma 4.13 (Local approximation for a refresh)**.** *There exists an absolute constant $C > 0$ such that the following holds with $\beta := q^{-1/3}$ and $\varepsilon := C \cdot q^{-1/9}$. For every round $i \in \{1, \dots, T\}$, formal incoming history $h \in \mathcal{Z}^{i-1}$, and answer $z \in \mathcal{Z}$, there exists a single table $G_{i,h,z} : U_{\mathsf{L}}^2 \times U_{\mathsf{R}}^2 \to \mathbb{R}$, common to all endpoint labels, with a chosen product representation whose energies (see Def. 2.16) satisfy*

$$\|G_{i,h,z}\|_* \le 4, \tag{4.58}$$

$$\Phi_{i,h,z}(\ell, \ell') \le 2/\beta \qquad \text{for every } (\ell, \ell') \in U_{\mathsf{L}}^2, \tag{4.59}$$

$$\Psi_{i,h,z}(r, r') \le 2/\beta \qquad \text{for every } (r, r') \in U_{\mathsf{R}}^2, \tag{4.60}$$

$$\mathbb{E}_{(\ell,\ell') \leftarrow \$ U_{\mathsf{L}}^2} \left[\Phi_{i,h,z}(\ell, \ell')\right] \le 4, \tag{4.61}$$

$$\mathbb{E}_{(r,r') \leftarrow \$ U_{\mathsf{R}}^2} \left[\Psi_{i,h,z}(r, r')\right] \le 4, \tag{4.62}$$

$$\left\| \mathsf{prob}_{i,h,z} - G_{i,h,z} \right\|_{2,\mu_s \otimes \mu_s} \le \varepsilon \qquad \text{for every } s \in \{0,1\}, \tag{4.63}$$

$$\|G_{i,h,z}\|_{2,\mu_s \otimes \mu_{s'}} \le 6 \qquad \text{for every } (s, s') \in \{0,1\}^2. \tag{4.64}$$

The same table and product representation satisfy these bounds simultaneously for every $s \in \{0,1\}$ and every $(s, s') \in \{0,1\}^2$. All bounds are per answer and uniform over formal histories, including histories that are never reached.

Before we prove the lemma, we explain its meaning and how it is used in the hybrid argument. The two leakage functions of a round share the refresh matrix M, so the true leakage probability function $\mathsf{prob}_{i,h,z}$ need not split into a function of the left pair (ℓ, ℓ') times a function of the right pair (r, r'). Such a *product* would reduce the analysis to one function of each party's grouped shares. The nuclear norm of a table is the least product cost $\sum_j \|a_j\|_2 \cdot \|b_j\|_2$ over its decompositions $\sum_j a_j \cdot b_j^{\mathsf{T}}$ into products (see Eq. (A.3) in the App. A). The approximation bound in Eq. (4.63) says that $\mathsf{prob}_{i,h,z}$ is nevertheless close to a signed sum of products, and Eq. (4.58) bounds their total cost independently of q. Closeness is required only where the later comparison of executions uses it: for an old and a new state that encode the same bit s and are sampled from $\mu_s \otimes \mu_s$, as in a real execution. The same table $G_{i,h,z}$ serves every assignment of labels to the endpoint states, so executions with different labels multiply identical tables and differ only in the distributions of the states. At a cheating round, the old and new states encode different bits, so the true leakage probability function is undefined. In that case, $G_{i,h,z}$ is only a signed table of controlled size, with L_2 norm at most 6 under $\mu_s \otimes \mu_{s'}$ by Eq. (4.64) and the same product-representation bounds as at every other round.

The bounds of Lem. 4.13 serve different later steps. The nuclear bound Eq. (4.58) and the mean-energy bounds Eqs. (4.61) and (4.62) show that, averaged over the pairs (ℓ, ℓ') and (r, r') respectively, the products have cost at most 4. The mean bounds alone would allow the energy at a single pair to be as large as $4q^2$, since each side has q^2 pairs. The pointwise bounds Eqs. (4.59) and (4.60), each $2/\beta = 2q^{1/3}$, rule this out, although they are far above the mean. A nuclear-norm bound alone would not give such a pointwise bound (see Sec. 2.3), and Lem. 4.16 needs it: when two neighboring rounds are multiplied over their common state, it bounds one energy pointwise and averages the other. The L_2 bounds Eqs. (4.63) and (4.64), with constants ε and 6, are used through the Cauchy–Schwarz inequality (see Lem. 2.7) in Lem. 4.15, which shows that replacing the true table of one round by $G_{i,h,z}$ costs at most 6ε. Finally, $\beta = q^{-1/3}$ is the smallest threshold for which Lem. 4.6 applies, and it gives the smallest error $\varepsilon = C \cdot q^{-1/9}$. The price is the large pointwise energy $2q^{1/3}$, which the discrepancy bound of order $q^{-1/2}$ (see Eq. (2.14)) more than compensates for when one encoding is changed.

Proof of Lem. 4.13. Apply Lem. 4.6 with $\beta = q^{-1/3}$ to the masks of the answer z, and set $G_{i,h,z} := 2 \cdot \mathsf{prob}_{i,h,z}^{>\beta}$. The nuclear norm doubles, so Eq. (4.9) gives Eq. (4.58). Multiplying each balanced

factor by $\sqrt{2}$ doubles both energies: they become $2 \cdot \Phi_\beta$ and $2 \cdot \Psi_\beta$. Thus Eqs. (4.12) and (4.13) give Eqs. (4.59) and (4.60), respectively. The factors remain balanced, so each mean energy equals the nuclear norm by Eq. (2.9). This proves Eqs. (4.61) and (4.62). By Eq. (2.13), each fixed-bit encoding distribution μ_s has density at most three relative to independent uniform shares, so $\mu_s \otimes \mu_{s'}$ has density at most nine relative to ν_{joint}. On the support of $\mu_s \otimes \mu_s$, the residual Δ_z equals $\mathsf{prob}_z - 2 \cdot \mathsf{prob}_z^{>\beta}$. Eq. (4.11) therefore gives Eq. (4.63) after enlarging C. For arbitrary endpoint labels, the same density bound and the L_2 bound in Eq. (4.10) give $\left\|2 \cdot \mathsf{prob}_z^{>\beta}\right\|_{2,\mu_s \otimes \mu_{s'}} \le 3 \cdot 2 = 6$, proving Eq. (4.64). For an unreachable history, fix any admissible leakage functions (see Sec. 3) and use the same construction. □

The tables may be signed, and their sum over answers need not be one. Only the true leakage probability functions have the positivity and normalization used below.

4.4 Comparing two neighboring rounds

We now carry out the second step of the proof: bounding what it costs to replace a true leakage probability function by its approximation and, in Sec. 4.5, to change the label of a state. We use the tables of Lem. 4.13 with $\beta = q^{-1/3}$ and $\varepsilon = C \cdot q^{-1/9}$. Recall from Def. 4.3 that the label of a state is the bit it encodes in the hybrid. A round is valid when its old and new states have equal labels. Otherwise, it is a cheating round. Recall also that G comes with a chosen product representation whose factor energies are Φ and Ψ (see Def. 2.16). All tables for every formal history are fixed before any labels are changed.

Definition 4.14 (Signed rounds and signed measures)**.** *A round is* signed *when its leakage probability function* prob *is replaced by its approximation G, and* unsigned *otherwise. Every unsigned round uses its true probability function, called its* true factor*, and must have equal endpoint labels. For a fixed labeling, the resulting* signed measure *assigns to a complete history the expected product of these local factors over the independently sampled states, as in Eq.* (4.4)*, using G at signed rounds and* prob *elsewhere.*

The total mass of a signed measure need not be one.

The hybrid argument of Sec. 4.7 changes the bit encoded by one state at a time. An interior state is shared by the round producing it and the round reading it. One of these rounds has unequal endpoint labels before the change, and the other has unequal endpoint labels after it. Both rounds are therefore signed when the label changes, and replacing or restoring a true function can put its approximation error next to another signed factor. Because the factors share a state, we bound their product using their L_2 norms under the joint distribution of the three states.

Lemma 4.15 (Replacing one round)**.** *Fix $j \in \{1, \dots, T-1\}$, formal histories and answers in rounds j and $j+1$, and labels $s_{j-1}, s_j, s_{j+1} \in \mathbb{F}_2$ for the states $A = X_{j-1}$, $X = X_j$, and $B = X_{j+1}$. Sample $A \leftarrow\!\$\ \mu_{s_{j-1}}$, $X \leftarrow\!\$\ \mu_{s_j}$, and $B \leftarrow\!\$\ \mu_{s_{j+1}}$ independently. If round j is valid, $e(A,X) := \mathsf{prob}(A,X) - G(A,X)$ is its error, and round $j+1$ is signed with approximation $G'(X,B)$ (its* signed companion*), then*

$$\mathbb{E}_{A,X,B}\big[|e(A,X) \cdot G'(X,B)|\big] \le 6\varepsilon. \tag{4.65}$$

The same estimate holds when the valid error depends on (X,B) and its signed companion on (A,X). For every valid round with no signed companion, the corresponding bound is ε.

Proof. By Eq. (4.63), the L_2 norm of e under its two-state marginal is at most ε. By Eq. (4.64), that of G' is at most 6, even if its endpoint labels differ. The Cauchy–Schwarz inequality (see Lem. 2.7) under the original joint distribution proves Eq. (4.65):

$$\mathbb{E}_{A,X,B}\big[|e(A,X)\cdot G'(X,B)|\big] \le \big(\mathbb{E}_{A,X,B}\Big[e(A,X)^2\Big]\big)^{1/2}\cdot\big(\mathbb{E}_{A,X,B}\Big[G'(X,B)^2\Big]\big)^{1/2} \le 6\varepsilon.$$

The two tables may share the state X, since the inequality is applied under the joint distribution of all three states. With no companion, the Cauchy–Schwarz inequality (see Lem. 2.7) applied to e and the constant function 1 gives the bound ε. □

4.5 Changing one encoding

Changing the label of a state from 0 to 1 replaces its encoding distribution μ_0 by μ_1. We call this a *label change*. Put $\mathrm{disc}_n := 3/\sqrt{q-1}$, the one-encoding discrepancy bound of Eq. (2.14). Only the two incident rounds depend on the state whose label changes. After fixing the encodings at the other endpoints of those rounds, their signed tables form the product to which we apply the one-encoding discrepancy estimate. The nuclear norm of this product can be large at exceptional outer encodings. We bound its average using the pointwise bound for one energy and the mean bound for the other.

Lemma 4.16 (Changing one encoding between signed rounds)**.** *For fixed formal histories and answers, suppose both rounds incident to X_j are signed. With $A = X_{j-1}$, $X = X_j$, and $B = X_{j+1}$ as in Lem. 4.15, and G_j and G_{j+1} the approximations at rounds j and $j+1$, let $F_{A,B}(X) := G_j(A,X)\cdot G_{j+1}(X,B)$. For every pair of labels $(s_A, s_B) \in \mathbb{F}_2^2$, sample $A \leftarrow\$\ \mu_{s_A}$ and $B \leftarrow\$\ \mu_{s_B}$ independently. Then*

$$\mathbb{E}_{A,B}\big[\|F_{A,B}\|_*\big] \le 8/\beta, \tag{4.66}$$

where the nuclear norm is in the two shares of X. Consequently,

$$\mathbb{E}_{A,B}\big[|\mathbb{E}_{X\leftarrow\$\mu_0}[F_{A,B}(X)] - \mathbb{E}_{X\leftarrow\$\mu_1}[F_{A,B}(X)]|\big] \le 8\cdot\mathrm{disc}_n/\beta. \tag{4.67}$$

At a chain endpoint, with just one incident signed round, the bounds are 4 and $4\cdot\mathrm{disc}_n$, respectively.

Proof. Write Φ_j, Ψ_j, Φ_{j+1}, and Ψ_{j+1} for the energies of their chosen product representations (see Lem. 4.13). Multiply the chosen product representations of the two signed tables. At fixed $A = (L_A, R_A)$ and $B = (L_B, R_B)$, the product-representation bound Eq. (2.5) and the Cauchy–Schwarz inequality (see Lem. 2.7) over factor indices give

$$\|F_{A,B}\|_* \le \sqrt{\Phi_\times(L_A,L_B)\cdot\Psi_\times(R_A,R_B)}, \tag{4.68}$$

where

$$\Phi_\times(L_A,L_B) := \mathbb{E}_{L_X\leftarrow\$U_{\mathsf{L}}}[\Phi_j(L_A,L_X)\cdot\Phi_{j+1}(L_X,L_B)],$$
$$\Psi_\times(R_A,R_B) := \mathbb{E}_{R_X\leftarrow\$U_{\mathsf{R}}}[\Psi_j(R_A,R_X)\cdot\Psi_{j+1}(R_X,R_B)].$$

Apply the Cauchy–Schwarz inequality (see Lem. 2.7) to Eq. (4.68) when averaging over A and B. Their left marginals are independent uniform shares, as are their right marginals, for every assignment of their labels. For $\Phi_\times$, bound Φ_j pointwise by $2/\beta$ using Eq. (4.59) and average Φ_{j+1} using Eq. (4.61). This gives $\mathbb{E}_{A,B}[\Phi_\times(L_A,L_B)] \le (2/\beta)\cdot 4 = 8/\beta$. The same argument with Eqs. (4.60) and (4.62) gives $\mathbb{E}_{A,B}[\Psi_\times(R_A,R_B)] \le 8/\beta$. This proves Eq. (4.66). Applying Eq. (2.14) at each fixed A and B and then averaging proves Eq. (4.67). With one signed round, the same calculation uses just its two mean energies, each at most 4 by Eqs. (4.61) and (4.62). □

4.6 Accounting for adaptive histories

The bounds in Eqs. (4.65) and (4.67) concern fixed formal histories and answers, with states that are not conditioned on an observed transcript. To use them in the full execution, we enumerate the answers near a *block* of two consecutive signed rounds that are changed together, at a cost of at most K^3. Conditioning the states on an observed prefix could destroy the independent sampling used in the local estimates. Instead, we keep the state samples unconditioned and explicitly sum over the answers near the changed block. The calculation proceeds in the following order.

- For a replacement, move the absolute value inside the state average. For a label change, first take the difference of the two averages over the changed state, and only then move the absolute value inside the average over the outer states. All unsigned factors are nonnegative.
- Sum unsigned answers after the block backward. Their true probability functions sum to one pointwise at every incoming history. These functions do not depend on the changing state, which is incident only to the signed block. This summation removes the entire suffix before any answers are enumerated.
- Enumerate the two block answers, costing at most K^2. If round $j-1$ exists, enumerate its answer too, costing at most K, and drop its true factor using $0 \le \mathsf{prob} \le 1$. Round $j-1$ is the only earlier round touching $\{X_{j-1}, X_j, X_{j+1}\}$.
- The remaining prefix factors depend only on states outside this three-state set. At each fixed formal prefix, average over the three states of this set, sampled independently from their respective encoding distributions, using Lem. 4.15 or Lem. 4.16, whose bounds are uniform in that prefix. Then sum the remaining prefix answers backward, again using normalization.

Thus a replacement costs at most $6K^3 \cdot \varepsilon$, and a label change at most $8K^3 \cdot \mathrm{disc}_n/\beta$, in the L_1 norm of signed history measures: the sum of absolute masses over complete histories (see Def. 2.1). A missing round simply removes one enumeration. These same bounds cover the endpoints. The order matters: the suffix is summed first because enumerating the later neighbor of a block as well would give an unnecessary fourth factor of K. The earlier neighbor cannot be removed in the same way. Its answer belongs to the incoming histories of the block, so the block's tables change with this answer, and normalization of its true probability function applies only once those tables are gone. The tables cannot be summed away first, because unlike true probability functions they need not sum to one over their answers. They are instead bounded by averaging over the three states, but the true factor of the earlier neighbor depends on one of these states, X_{j-1}. Enumerating its answer and discarding this factor resolves the difficulty, at the cost of the third factor of K.

4.7 Moving one boundary along the chain

We can now combine the local replacement and label-change bounds to compare the executions for the two secrets. Assume $T \ge 1$. For $j = 0, \ldots, T+1$, give each state X_i the label

$$s_i^j := \begin{cases} 1, & i < j, \\ 0, & i \ge j. \end{cases}$$

Sample the states independently from these encoding distributions. For $1 \le j \le T$, use G at round j and the true leakage probability function prob elsewhere. Call the resulting signed history measure Q_j. The labeling has exactly one cheating round, round j, where the global

table G is defined even though the encoded bit changes. At the endpoints no round is signed: $Q_0 = P_0^T$ and $Q_{T+1} = P_1^T$ are the actual transcript distributions. The intermediate measures may be signed and need not be normalized.

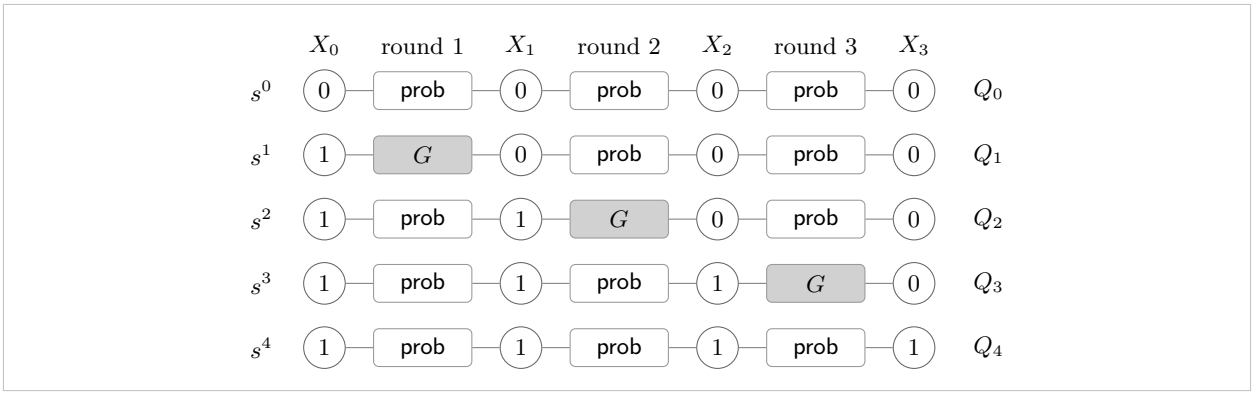


Figure 4: The labelings $s^0, \ldots, s^{T+1}$ for a chain of $T = 3$ rounds. Each intermediate labeling has exactly one cheating round, the signed round j (shaded), which uses its signed table G. Every other round uses its true leakage probability function prob.

For $1 \le j < T$, move from Q_j to Q_{j+1} as follows.

- Replace prob by G at round $j + 1$. Its endpoint labels are both zero.
- Change the label of X_j from zero to one. Both incident rounds are signed.
- Restore G to prob at round j. Its endpoint labels are now both one.

Every true factor is inserted or removed only at a valid round, and at most two rounds are signed at any time. For $Q_0 \to Q_1$, sign round 1 and change X_0. For $Q_T \to Q_{T+1}$, change X_T and restore round T. For $T = 1$, these two endpoint moves constitute the entire argument.

Consecutive labelings differ in the label of a single state, which the adversary never sees. The triangle inequality, the comparisons from Eqs. (4.65) and (4.67) with the answer enumeration in Sec. 4.6, and the factor $1/2$ between the L_1 norm and statistical distance give

$$\delta\left(P_0^T, P_1^T\right) \le C \cdot T \cdot K^3 \cdot (\varepsilon + \mathrm{disc}_n/\beta)\,. \tag{4.69}$$

The factor $1/2$ applies because the endpoints Q_0 and Q_{T+1} are actual probability distributions.

Proof of Thm. 4.1. For $q = 2^n - 1 \ge 3$, take $\beta = q^{-1/3}$ and $\varepsilon = C \cdot q^{-1/9}$. Since $\mathrm{disc}_n = 3/\sqrt{q-1} \le C \cdot q^{-1/2}$,

$$\mathrm{disc}_n/\beta \le C \cdot q^{-1/6} \le C \cdot q^{-1/9}. \tag{4.70}$$

Substituting Eq. (4.70) and $\varepsilon = C \cdot q^{-1/9}$ in Eq. (4.69) proves the first inequality in Eq. (4.1). Also $q \ge 2^{n-1}$ and $K^3 = 2^{6\lambda}$, which gives Eq. (4.2) after increasing an absolute constant. For $T = 0$, the transcript is empty and the statistical distance is zero. All estimates are uniform in the adversary's fixed tape. Averaging proves the assertion with that tape included in the view. □

4.8 Leakage rate and share length

We now translate Thm. 4.1 into a leakage rate and a sufficient share length. The per-party leakage rate is λ/n, since each stored share has n bits. The temporary refresh matrix is not counted.

Corollary 4.17 (Binary leakage rate)**.** *If T is polynomial in n and $\lambda \leq \alpha n$ for a fixed $\alpha < 1/54$, then $\delta\left(P_0^T, P_1^T\right)$ is exponentially small in n. More generally, for all integers $T \geq 1$ and $\lambda \geq 0$ and every real target parameter $\kappa \geq 0$, distance at most $2^{-\kappa}$ is obtained by every integer $n \geq 2$ satisfying*

$$n \geq 9 \cdot \left(\kappa + 6\lambda + \log_2(C' \cdot T)\right), \tag{4.71}$$

where C' is the absolute constant of Thm. 4.1.

Proof. Since $K^3 = 2^{6\lambda}$, Eqs. (4.1) and (4.2) bound the distance by $C' \cdot T \cdot 2^{6\lambda - n/9}$. Under the first assumption, the exponent is at most $-(1 - 54\alpha) \cdot n/9$. Multiplying by a polynomial factor in n preserves exponential decay. Substituting Eq. (4.71) proves the second assertion. □

In particular, the sufficient share length depends logarithmically on the number of refreshes. Since the proof leaves C' unspecified, this estimate does not give a concrete numerical share length.

4.9 Initialization with zero shares

The protocol of Def. 2.4 requires both initial shares to be nonzero. The restriction is only a convenience: allowing zero shares in the initial encoding changes every refreshing bound by at most $4/N$, where $N = 2^n$ (see Sec. 2.2). Suppose that the initial state is instead uniform on all pairs $(\ell, r) \in \mathbb{F}_2^n \times \mathbb{F}_2^n$ with $\ell \cdot r = s$, so that a share may be zero. Extend each leakage function to zero-share inputs in any fixed way. For $s = 1$, neither share can be zero, so the transcript distribution is unchanged. For $s = 0$, the number of such pairs is $N(N+1)/2$, and $2N - 1$ of them contain a zero share. The probability of a zero share is therefore

$$\gamma_0 := \frac{2(2N-1)}{N(N+1)} \leq \frac{4}{N}, \tag{4.72}$$

and conditioned on its complement the initial state has distribution μ_0. The new transcript distribution for $s = 0$ is thus $(1 - \gamma_0) \cdot P_0^T + \gamma_0 \cdot \widetilde{P}$, where $\widetilde{P}$ is the transcript distribution conditioned on the initial state containing a zero share, and its statistical distance from P_0^T is at most γ_0. By the triangle inequality and Eq. (4.72), every refreshing bound therefore increases by at most $4/N$ under this initialization.

5 Circuit compilation

In the previous section we addressed the problem of refreshing an encoded secret under leakage. Let us now turn to the more complex task of protecting an arbitrary computation against leakage. Recall from Sec. 1 that a leakage-resilient circuit compiler transforms a circuit into a protected circuit such that anything the adversary learns from the leakage together with the input/output behavior could also be learned from the input/output behavior alone. We asked there whether arbitrary circuits can be compiled so that they remain secure when every active gadget, the two-party protocol that implements one gate on encodings, leaks locally, without any leak-free internal component. Thm. 5.2 answers this question for a single evaluation: a simulator that receives the output of the

circuit but not the private input reproduces the adversary's view within a small statistical distance. Thm. 6.1 in Sec. 6.3 extends this to stateful circuits that are invoked repeatedly with inputs chosen by the adversary during the execution, which is the continual-leakage setting for computation. The guarantees for circuit compilation do not follow from Thm. 4.1 as a black box: they reuse the matrix refresh and its local approximation, but composing the gadgets requires a new argument.

The construction, previewed in Sec. 1.1, compiles NAND source circuits into two-party protocols using the matrix refresh of Def. 2.4. Chains of refreshes give every gate input a fresh encoding, which handles arbitrary fanout and repeated operands. The private-input encoder and the final output decoder are trusted. Every internal operation has its stated leakage budget. Communication is private, and queries access only the permitted data of the active operation. The trusted decoder releases only the output in the clear. The output shares stay hidden. Sec. 5.1 gives the compiler and its security statement. Sec. 5.2 specifies the NAND gadget, Sec. 5.3 extends the local approximation to it, Sec. 5.4 proves the composition bound, and Sec. 5.5 gives the concrete simulator.

5.1 Compilation with refreshing between gadgets

We first give the compiler for stateless circuits with a trusted input encoder and output decoder. The encoder supplies only the private input encodings. The decoder reveals only the output in the clear, after all leakage queries, and the output share pairs remain hidden. Every internal operation has its stated leakage budget: no refresh, sampling operation, or phase of a computation gadget is assumed to be unobserved.

5.1.1 NAND sources and preprocessing

We begin with the circuits to be compiled. Let $\Gamma : \mathbb{F}_2^{I_{\mathrm{pub}}} \times \mathbb{F}_2^{I_{\mathrm{priv}}} \to \mathbb{F}_2^{k}$ be a Boolean circuit with public input $\mathbf{x}$, private input $\mathbf{s}$, and a fixed topological order of its $t_0 \geq 0$ *source gates*. Every source gate computes $\mathrm{NAND}(s_1, s_2) := 1 + s_1 \cdot s_2$ over $\mathbb{F}_2$. An *input port* is one of the two operand positions of a gate, and a *consumer* of a wire is a gate that reads it. Repeated operand wires and arbitrary fanout are allowed. The ordered list of $k \geq 0$ *designated outputs* may repeat a wire or designate an input wire directly. Public constants are public input wires. An input is *unused* if it has neither a consumer nor an output position. Unused private inputs may be deleted, or retained without affecting the leakage transcript. Unused public inputs may also be removed during preprocessing. The compiled execution and operation counts then refer to the resulting source.

This source basis represents every Boolean circuit given in a fixed AND/OR/NOT basis with constant-factor overhead. Negation is $\mathrm{NAND}(s, s)$. Replace an AND gate with operands s_1 and s_2 by the two NAND gates

$$w' := \mathrm{NAND}(s_1, s_2), \tag{5.1}$$

$$w := \mathrm{NAND}(w', w'). \tag{5.2}$$

Here w' is one intermediate wire feeding both input ports of the second gate, and Eqs. (5.1) and (5.2) give $w = s_1 \cdot s_2$ for every pair of input bits. An OR gate is $\mathrm{NAND}(\mathrm{NAND}(s_1, s_1), \mathrm{NAND}(s_2, s_2))$, using three NAND gates. Each replacement preserves the old output value, hence preserves the circuit function and the designated output list. These replacements need no new constant wire, and any constants present in the source description are counted among the I_{pub} public input wires. All counts below refer to the source after preprocessing. We impose no fanout bound and do not require distinct source operands or a positive gate count: the refresh chains provide distinct encoding copies for repeated source operands, and a source with no gates needs no such copies.

Definition 5.1 (Operations, wire encodings, scopes, and labelings)**.** *An* operation *of the compiled protocol is a computation gadget, an input-port refresh, or a public-input sampler, all specified below. A stored wire with plaintext label $s(w) \in \mathbb{F}_2$ has an encoding $X_w = (L_w, R_w)$ with $L_w \in U_{\mathsf{L}}$, $R_w \in U_{\mathsf{R}}$, and $L_w \cdot R_w = s(w)$. The* scope *S_i of operation i is the set of encoding locations it reads or produces. For a set A of locations, write $X_A := (X_w)_{w \in A}$. A* labeling *assigns a bit to every location. It is* valid *when it satisfies the equations of all computation and refresh operations and the prescribed public labels.*

The *persistent share length*, the length of the shares that wires carry between operations, is $n \geq 2$. The sets U_{L} and U_{R} and the encoding distributions μ_s are those of Def. 2.3.

A refreshed encoding for each input port. The compiler gives each input port its own refreshed copy of the encoding of its source wire. These copies control how many operations can inspect any one encoding, even when a source wire has many consumers. The composition proof accounts for operations touching a changed encoding through their answer alphabets and factor energies (see Sec. 5.4). Bounding the number of operations touching an encoding prevents those costs from growing with the source fanout. Refreshing every copy directly from the original would still read the original once per consumer. We therefore arrange the copies in a chain so that generating each subsequent copy does not require reading the original encoding again. For each source wire w, enumerate its $\deg(w)$ input-port occurrences in a fixed public order, namely topological gate order with the left input port before the right. Introduce a chain of dimension-n refreshes, the *input-port refreshes* of w,

$$w = w_0 \longrightarrow w_1 \longrightarrow \cdots \longrightarrow w_{\deg(w)},$$

and route occurrence j to w_j. A refresh from w_{j-1} to w_j samples a fresh matrix $M \leftarrow\!\$\, \mathsf{InvMatr}_2^n$, sends it privately, and sets

$$L_{w_j} := L_{w_{j-1}} \cdot M, \qquad R_{w_j} := M^{-1} \cdot R_{w_{j-1}}.$$

Execute the next link immediately before its assigned input port is consumed. The current chain endpoint is retained until the next link is generated, and the original source-wire encoding is also retained whenever it is a designated output. An older copy may be erased only after all of its assigned uses have finished. After this refresh, erase M, its inverse, its *sampling tape* (the random bits used to sample it, including rejected proposals), and the temporary workspace. Retaining an old encoding in inactive memory does not enlarge the active view: each party sees precisely its old share and the matrix, which determine its new share. An input-port refresh is thus the local refresh analyzed in Lem. 4.13. Only the lifetime of the stored old share differs from continual storage. Fig. 5 shows the chain of a wire with two consumers.

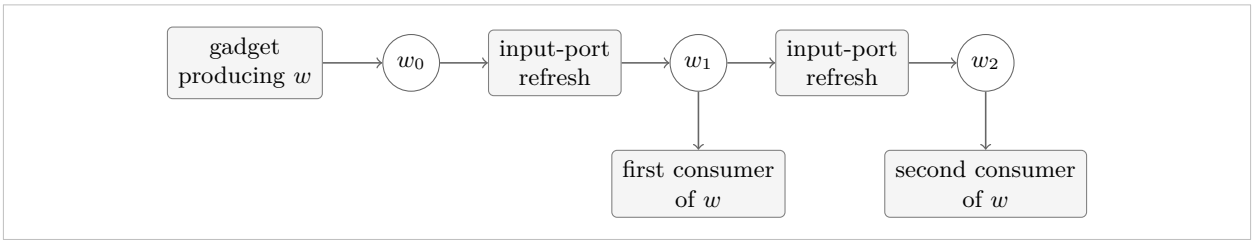


Figure 5: The input-port refreshes of a wire w with $\deg(w) = 2$. The copy w_1 lies in the scopes of three operations: its producing refresh, its consumer, and the next refresh.

Execution of the compiler.

1. Independently encode every private input bit using the trusted encoder. For each public input or constant bit c, run a *public-input sampler*: the left party samples $(L, R) \leftarrow\$ \mu_c$, sends R privately, and retains L. During this sampler, the left party's leakage may read the entire pair and sampling tape, and the right party's leakage may read its received share. The left party then erases R and the sampling tape.
2. Process source gates in topological order. Immediately before a gate, refresh the current endpoint of its left operand's chain, then refresh the current endpoint of its right operand's chain, updating the endpoint after each refresh. Evaluate the protected NAND gadget of Sec. 5.2 on those two fresh copies. If both source operands are the same wire, the right refresh starts from the newly updated endpoint, giving two consecutive, distinct copies. The gadget tensors its operands, refreshes in dimension n^2+1, erases that matrix, its inverse, and its sampling tape, and shortens back to length n. Its output is the original encoding of the new source wire and starts that wire's chain.
3. Erase each operation's temporary workspace when it finishes. Erase a wire encoding once its consumer, its next refresh, and any designated output use have all finished. Unrelated retained encodings are inaccessible to the active operation's leakage.
4. After all leakage answers, the trusted decoder reads the designated original source-wire encodings and releases their inner products, namely $\mathbf{y} = \Gamma(\mathbf{x}, \mathbf{s})$. It releases no shares or other information.

This schedule gives every original input port a preceding refresh, and a gate that uses the same source wire twice receives two distinct refreshed copies. Each copy w_j belongs to at most three operation scopes: its producing refresh, its sole source-gate consumer, and the next refresh. An original gate output belongs only to its producer and its first refresh, and an original public input belongs only to its sampler and its first refresh. Thus every scope contains at most three distinct wires, and every wire belongs to at most three scopes. Decoding is outside the leakage experiment. The number of input-port refreshes is exactly $\sum_w \deg(w) = 2t_0$, so the operation counts are

$$\text{NAND computation gadgets} = t_0, \tag{5.3}$$

$$\text{input-port refreshes} = 2t_0, \tag{5.4}$$

$$\text{public-input samplers} = I_{\text{pub}}, \tag{5.5}$$

$$S = 3t_0 + I_{\text{pub}}. \tag{5.6}$$

Eq. (5.6) counts all internal operations, including the public constants supplied after preprocessing.

Leakage model. All communication is private, and each query reads only its party's data for the active operation, with the erasures above. Each computation gadget has the matrix and shortening phases of Sec. 5.2, with budgets λ_{M} and λ_{S} per party and $\lambda = \lambda_{\mathsf{M}} + \lambda_{\mathsf{S}}$. Each input-port refresh and public-input sampler has one phase with at most λ bits per party. The functions in a phase are fixed before either answer is returned, and they may depend on all previous answers, including those of the earlier phase of the same gadget. All rejected proposals and shortening retries belong to their phase's local tape and budget, and the adversary observes no separate timing or retry count. Randomized leakage obeys the erasure-respecting convention of Sec. 5.2.1. Consequently the complete answer of each operation has at most $K := 2^{2\lambda}$ possible values.

An *auxiliary input* is a fixed string supplied to the adversary before the execution. It is the same in the compared executions and is also supplied to the simulator. The randomness used for initialization and the fresh protocol randomness are independent of this string. We fix this auxiliary input and the adversary's independent random tape. All bounds hold uniformly for these choices and can afterward be averaged over them. The *authorized output* of an evaluation is $\mathbf{y} := \Gamma(\mathbf{x}, \mathbf{s})$, the output

that the evaluation is meant to reveal. An adversary that receives $\mathbf{y}$ before its queries can include it in its auxiliary input. The comparison fixes that same auxiliary string in the real and dummy executions. Uniformity over auxiliary strings covers this choice.

Theorem 5.2 (Security of the compiler)**.** *The compiler correctly evaluates every NAND source circuit specified above. Moreover, there exist absolute constants $C \geq 1$ and $C' \geq 1$ with the following property. For every source circuit, dimension $n \geq 2$, pair of nonnegative integer phase budgets, public input $\mathbf{x}$, and admissible leakage strategy, there exists a single simulator, independent of the private input and of any target error, that is given $\mathbf{x}$, the authorized output, and the adversary's auxiliary input. For every private input $\mathbf{s}$, let $P_{\mathbf{x},\mathbf{s}}$ be the distribution of the real view and let $\widehat{P}_{\mathbf{x},\mathbf{y}}$ be the distribution of the simulator's view for $\mathbf{y} = \Gamma(\mathbf{x}, \mathbf{s})$. Each view consists of the complete leakage transcript and the output $\mathbf{y}$, and contains no output shares. Then*

$$\delta\Big(P_{\mathbf{x},\mathbf{s}}, \widehat{P}_{\mathbf{x},\mathbf{y}}\Big) \leq \min\Big\{1,\ C \cdot S \cdot K^9 \cdot (2^n - 1)^{-1/27}\Big\} \tag{5.7}$$

$$\leq C' \cdot S \cdot 2^{18\lambda - n/27}. \tag{5.8}$$

The simulator has polynomial expected primitive work in the circuit and input sizes and n. It makes $8t_0 + 2I_{\mathrm{pub}}$ local-answer evaluations, with adversary and leakage computation charged separately (see Sec. 5.5 for both notions).

For every fixed $\alpha < 1/486$, choosing $\lambda \leq \alpha \cdot n$ gives error $S \cdot 2^{-\Omega(n)}$. Thus for polynomial-size circuit families the share length can be chosen as a function of the security parameter alone, independently of the circuit size.

Proof. Correctness. Each refresh preserves its bit, and the computation gadget outputs an encoding of its gate value by Lem. B.2 in the App. B.

The canonical experiment. Exact freshness of a refresh (see Lem. 2.19) and of a computation gadget implies that, conditional on the preceding state before its fresh randomness is drawn, the output of the operation has distribution μ_s, where s is its prescribed bit. Public-input samplers also have this property. The *canonical experiment* therefore samples all wire encodings independently and then samples the leakage answers sequentially from their conditional probability functions. The argument of Lem. B.5 in the App. B applies to the mixture of these three operation types, including retained old encodings in refresh chains. The independence concerns the encodings before any answer is observed, not their distribution after conditioning on leakage.

Comparing private inputs. For every formal history and answer, the computation gadget has the common approximation of Lem. 5.6, and the dimension-n refresh has the corresponding approximation of Lem. 4.13 at $\beta = q^{-1/3}$, with $q = 2^n - 1$. Their valid-label L_2 errors are at most $C_{\mathrm{err}} \cdot \beta^{1/3}$ for an absolute constant C_{err} (see Eqs. (4.63) and (5.25)), and their separate mean factor energies are at most four (see Eqs. (4.61), (4.62), (5.23) and (5.24)). A public-input sampler retains its true normalized conditional answer functions throughout the comparison, and its encoded bit never changes. The scope and incidence bounds established above allow Thm. 5.7 to compare the leakage of any two private-input executions with error $C \cdot S \cdot K^9 \cdot q^{-1/27}$ (see Eq. (5.28)).

The simulator. The simulator encodes the all-zero private input and executes all internal operations of the actual compiled protocol against the supplied adversary. It stops before the output decoder, discards the dummy output shares, and returns the supplied $\mathbf{y}$ as the final authorized output. Apply the comparison with that dummy input, fixing the real $\mathbf{y}$ as auxiliary data if the strategy uses it. Appending the same $\mathbf{y}$ to both leakage distributions preserves their distance. The argument also covers private inputs designated directly as outputs and repeated output positions,

and unused private encodings have no effect on leakage. If $S = 0$, the transcript is empty and simulation is exact. This proves Eq. (5.7).

Efficiency and parameters. The concrete samplers and their polynomial expected-work argument are given in Sec. 5.5. Each of the t_0 computation gadgets requires four local-answer evaluations, each of the $2t_0$ refreshes requires two, and each public-input sampler requires two. If adversary selection and leakage evaluation cost at most R_{ev} per call, expected total work is at most a polynomial in the stated parameters plus $(8t_0 + 2I_{\mathrm{pub}}) \cdot R_{\mathrm{ev}}$. The geometric-tail truncation described there gives a simulator with polynomially bounded primitive work and any prescribed additional negligible statistical error. Efficiency of total simulation requires the corresponding bound on the adversary's and leakage functions' computation, whereas the statistical comparison imposes no computational restriction. Finally, $q \geq 2^{n-1}$ gives Eq. (5.8), and $18\alpha < 1/27$ gives the stated leakage fraction. □

The guarantee is for one stateless evaluation with a trusted output boundary. Thm. 6.1 in Sec. 6.3 extends it to repeated invocations of a stateful circuit with persistent private registers and inputs chosen adaptively by the adversary. The view also excludes the final encodings, because their labels can differ between a real and a dummy execution: a dummy output pair may encode a bit different from the corresponding entry of $\mathbf{y}$, and revealing its shares would expose this difference at once. Because these pairs stay hidden, the composition proof can treat output wires like other internal wires, whose labels may change between the compared executions (see Sec. 5.4.1). The signed intermediate measures used in Sec. 5.4 are solely proof devices.

5.2 The protected NAND gadget

Having described how the compiler connects its operations, we now turn to the gadget that implements a source gate: a two-party protocol that computes NAND on encodings.

5.2.1 Local views and phase order

We start by fixing the leakage model for one NAND computation gadget. Recall from Def. 5.1 that the scope of an operation is the set of wire encodings it reads or produces. The two refreshed operand encodings of a computation gadget and its new output are distinct. All protocol communication is private. Leakage is *gate-local*: a query can access only the active gadget's local data, comprising its party's encodings in X_{S_i}, fresh local randomness, and private messages. This is the only-computation-leaks granularity of the leakage model in the introduction, with each phase as one leakage round. The encodings of unrelated *live wires*, those still needed by later gates, are inaccessible. The restriction applies separately to each computation gadget, input-port refresh, and public-input sampler. After a gadget finishes, its workspace is erased, and only the encodings still needed by the circuit remain. After the last leakage query, only the trusted decoder reads the designated output encodings and releases their bits in the clear, so the leakage transcript contains no output shares.

Each gadget has two phases. Its *matrix phase* tensors the operand shares, appends a final coordinate equal to one, and applies the refresh in dimension $n^2 + 1$. The matrix is then erased, and the *shortening phase* reduces the refreshed shares to length n using a random full-row-rank matrix. Fix nonnegative integer budgets λ_{M} and λ_{S}, and set

$$\lambda := \lambda_{\mathsf{M}} + \lambda_{\mathsf{S}}, \qquad K := 2^{2\lambda}.$$

Each party leaks at most λ_{M} bits in the matrix phase and at most λ_{S} bits in shortening. The functions may depend on all earlier answers, including the matrix-phase answer when shortening

begins, so the adversary may adapt across phases and gates (see Sec. 6.2 for an extension beyond this model). Within each phase, both functions are fixed before either answer is revealed. All shortening attempts belong to that single phase: each function may read its party's entire local sampling tape and private messages from the attempts, within the original phase budget. The right party privately reports whether another attempt is needed. No leakage answer is released between attempts, and the adversary observes no separate retry count, timing, or protocol messages. Fig. 6 illustrates the two local views and their leakage to the adversary.

The *answer* Z_i of a gate concatenates the two parties' outputs from both phases. Padding gives a fixed alphabet $\mathcal{Z}$ of size at most K. As for refreshing, $H_i = (Z_1, \ldots, Z_i)$ is the history after gate i, and $H_0 = \varnothing$. The incoming history and the adversary's tape determine the matrix-phase functions. The shortening functions may additionally depend on that gate's matrix-phase answer. We use the concatenation notation $h\|z$ and the convention of fixing admissible functions at all formal histories (see Def. 3.3), so that every quantity below is defined for every formal history.

For a fixed matrix and matrix-phase answer, the operand pairs producing that answer form a *rectangle*: each party's half of the answer depends only on that party's own data, so the indicator of the answer is the product of a Boolean *mask* for the left party and one for the right party. These masks are defined exactly as in Sec. 3, with ℓ and r replaced by (ℓ_1, ℓ_2) and (r_1, r_2), respectively, and with a matrix of dimension $n^2 + 1$. The mask partition is taken separately for the matrix phase. The shortening answer is handled after the matrix has been erased.

Randomness of the leakage functions. Leakage functions are allowed to be randomized, and as in Sec. 3.4 their randomness is removed by presampling it and averaging the resulting deterministic bounds. Splitting a gadget into two phases adds one condition, because the refresh matrix is erased between them. The matrix-phase functions may use correlated auxiliary randomness whose distribution depends on the gate index, incoming history, and matrix, but is conditionally independent of the operand shares. Presampling these values gives the deterministic reduction of Sec. 3.4. In particular, the matrix sampler of Sec. 5.5 draws uniform matrices until one is invertible, and its rejected proposals are independent of both the accepted matrix and the operand shares. These rejected proposals can therefore be presampled as part of the correlated matrix-phase auxiliary randomness, fixed when defining the Boolean masks, and averaged afterward. Thus the analysis also covers functions of the complete matrix-sampling tape, without additional protocol messages. The entire tape is erased before shortening. In shortening, auxiliary randomness must respect erasure: conditional on the gate index and all preceding answers, its joint distribution is independent of the shares and the erased matrix. This randomness may therefore depend on the matrix only through those answers. Presample one auxiliary pair for each gate index, incoming history, and matrix-phase answer, independently across these choices and of the initial encodings and matrices. Fixing these pairs gives deterministic phase functions. Averaging restores their original distribution.

5.2.2 Tensor, refresh, and shorten

Every source NAND gate is implemented by the following three-step gadget, which uses the refresh of Sec. 4. First, tensoring the operands makes the gate output available as a single inner product. The refresh then randomizes the resulting long encoding, and a random projection brings it back to the persistent length n. Fig. 6 summarizes the gadget and its two leakage phases. Let (L_1, R_1) and (L_2, R_2) encode the two operands. Each L_j is a row in $\mathbb{F}_2^{1\times n}$, and each R_j is a column in $\mathbb{F}_2^{n\times 1}$. Even when the source gate repeats an operand, its two input ports receive distinct refreshed encoding pairs. The tensor product $\otimes$ of two shares lists all pairwise products of their coordinates.

Set $m := n^2 + 1$ and form the nonzero long shares

$$a := (L_1 \otimes L_2, 1), \qquad b := \begin{pmatrix} R_1 \otimes R_2 \\ 1 \end{pmatrix}.$$

The tensor $L_1 \otimes L_2$ is a row, so a is a row obtained by appending one coordinate. The displayed b is a column formed by vertical concatenation. These long shares already encode the desired NAND output:

$$a \cdot b = 1 + (L_1 \cdot R_1) \cdot (L_2 \cdot R_2).$$

The equality follows because the inner product of the two tensor products is $(L_1 \cdot R_1) \cdot (L_2 \cdot R_2)$. The two appended coordinates contribute one. Every later operation preserves this bit. The matrix phase applies Def. 2.4 in dimension m:

$$U := a \cdot M, \qquad V := M^{-1} \cdot b, \qquad M \leftarrow\!\$\ \mathsf{InvMatr}_2^m.$$

The left leakage function may read L_1, L_2, and M, and the right function may read R_1, R_2, and M. These data determine the respective long shares. Before shortening, each party retains only its old operand shares and its refreshed long share from the matrix-phase workspace. Each party erases the rest of that workspace, including M, its inverse, and its sampling tape. The adversary retains the answers already obtained.

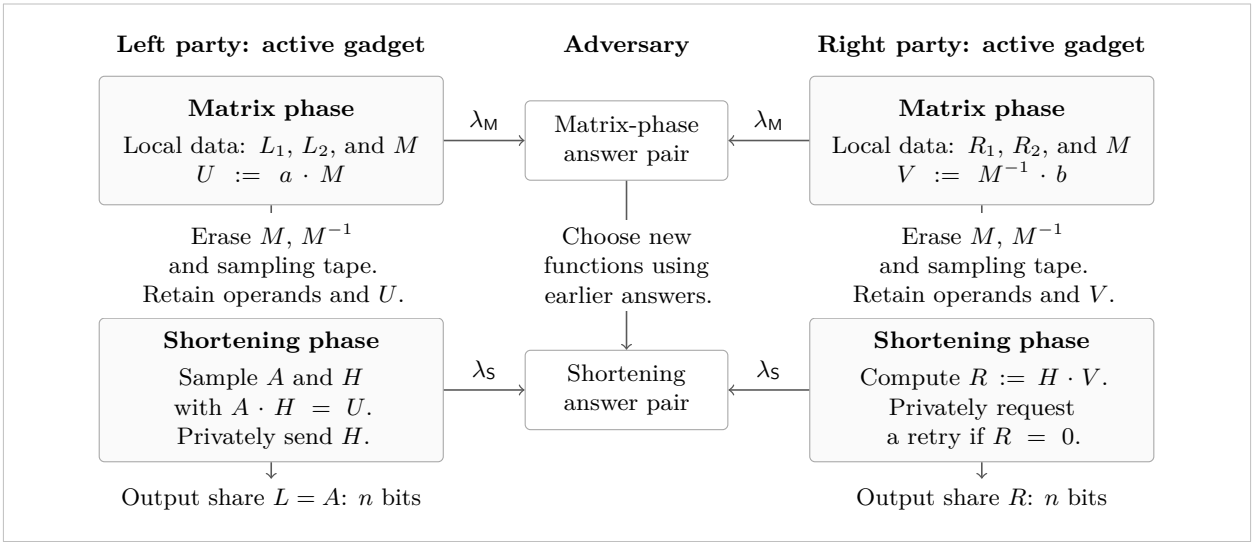


Figure 6: Local leakage in one circuit gadget. The matrix is erased between the two phases, while the adversary retains its answers. All retries belong to the shortening phase.

Shortening by a random projection. The refreshed shares have length m, while the wires of the circuit carry shares of length n, so the gadget must compress them without disturbing the bit they encode. The gadget does so by projecting onto n random independent directions. Given the nonzero long shares U and V of length $m \geq n$, perform the following attempt.

1. The left party samples a nonzero row $A \leftarrow\!\$\ U_{\mathsf{L}}$ and then samples

   $$H \leftarrow\!\$\ \{H' \in \mathbb{F}_2^{n \times m} : \operatorname{rank}(H') = n,\ A \cdot H' = U\}.$$

   It privately sends H.

2. The right party computes $R := H \cdot V$ and privately reports whether $R = 0$. If $R = 0$, repeat with fresh A and H and the same U and V. Otherwise, output

$$L := A, \qquad R := H \cdot V.$$

Correctness follows from one identity:

$$L \cdot R = A \cdot H \cdot V = U \cdot V = 1 + (L_1 \cdot R_1) \cdot (L_2 \cdot R_2).$$

The rows of H are independent and their span contains U. The new left share gives the coefficients expressing U in these rows. The new right share lists their inner products with V.

For every fixed pair of shares U and V, the final output is exactly a fresh encoding of $U \cdot V$ (see Lem. B.2 in the App. B). Here "fixed" means that we may condition on any particular pair produced by the preceding refresh. The fresh shortening randomness still gives the same output distribution. If $U \cdot V = 1$, the first attempt always succeeds. If $U \cdot V = 0$, its failure probability is

$$\frac{2^{m-n} - 1}{2^{m-1} - 1} \leq 2^{1-n}.$$

For $n \geq 2$, at most two attempts are needed in expectation. The protocol never explicitly evaluates $U \cdot V$, but a failed attempt implies $U \cdot V = A \cdot (H \cdot V) = 0$, so the private failure acknowledgment lets both parties infer that the encoded bit is zero. The retry-error bound below accounts for this possibility. Exact freshness explains why shortening produces a suitable encoding for the next operation. It does not by itself control leakage from the shortening transcript. In particular, the message H reveals a subspace containing the long left share. Our proof first refreshes the long encoding built from $L_1 \otimes L_2$, erases the refresh matrix, and then analyzes the shortening transcript together with the retained operands. The refresh gives this analysis its starting point: for fixed operands and before any answer is observed, the refreshed pair (U, V) is uniform among all nonzero long encodings of the output bit, whereas the unrefreshed long shares would be determined by the operands. This uniformity allows us to view a successful first shortening attempt in reverse, starting from its short output and sampling the long shares that could have produced it (see Eq. (B.13) in the App. B).

Each shortening leakage function may read its own old operands, its long share, the full shortening sampling tape and private transcript held by that party, and its final output share. All retries remain in the same phase with the same two leakage functions and total budget. The adversary observes no additional answers or timing information. The erased matrix is unavailable, so the proof does not cover a leakage function that reads both M and the later projection matrix H.

Relation to refreshing and cost. For $m = n$, every full-row-rank H is invertible, and the same sampling rule is equivalent to choosing H uniformly in $\mathsf{InvMatr}_2^n$ and setting

$$A := U \cdot H^{-1}, \qquad R := H \cdot V.$$

This is the matrix refresh of Def. 2.4 with $M = H^{-1}$. It never retries and communicates n^2 bits. The storage protocol of Sec. 4 uses this square case directly, with no preceding refresh or additional shortening stage. A rectangular shortening attempt communicates $n \cdot m$ matrix bits and one private success bit. Thus the complete shortening procedure communicates at most $2 \cdot (n \cdot m + 1)$ bits in expectation. The NAND gadget has $m = n^2 + 1$ and communicates an expected $O(n^4)$ bits in total, dominated by its full $m \times m$ refresh matrix. All sampling and local operations run in expected polynomial time (see Sec. 5.5).

5.3 Extending the local approximation to a computation gadget

We now extend the local approximation of Sec. 4.3 from a refresh to a computation gadget, which will allow us to compose the operations in Sec. 5.4. Recall that a *table* is a real function of a left argument and a right argument (see Def. 2.5), and its *nuclear norm* is the least total size $\sum_j \|a_j\|_2 \cdot \|b_j\|_2$ of a representation $\sum_j a_j \cdot b_j^{\mathsf{T}}$ as a sum of products (see Eq. (A.3) in the App. A). For a chosen representation, the left and right *energies* Φ and Ψ are the sums of the squares of the factors (see Def. 2.16). A *formal incoming history* is any sequence of earlier answers, whether or not it can occur (see Sec. 3). The *leakage probability function* prob of an answer is its probability given the encodings involved, and a round or gate is *signed* when prob is replaced by its signed approximation G (see Defs. 4.2 and 4.14).

5.3.1 Retaining the operand shares

The refreshing proof retains the same shares to which the random matrix is applied. A computation gadget instead retains its two operand shares while refreshing their tensor encoding. The table must keep those operands because the leakage functions may read them and because the gadget shares them with the input-port refreshes that produce them, so the composition proof compares executions in which their labels may differ. Since the tensor map is injective, an operand pair carries the same information as its old long share, which suggests applying Lem. 4.6 in the long dimension. That lemma, however, bounds an error averaged over all nonzero old shares, whereas the tensors of operand shares occupy only a thin subset of the long-share space. A small average over the whole space need not be small on that subset. We therefore extend the approximation lemma of Lem. 4.6 to distinguish the *old local data*, over which the estimates are averaged, from the *share to be refreshed*, which is a function of those data. The old data no longer range over the full space of shares: they range over an arbitrary finite set that is mapped injectively into that space, which is what makes the lemma apply to tensors. The analytic argument remains the one from Sec. 4.3.3, because the Gram and four-cycle estimates require only the corresponding counts for pairs of old inputs, and a uniform invertible matrix sends any two distinct nonzero vectors to a uniform pair of distinct nonzero vectors, whether or not they are tensors. Beyond injectivity, the tensor structure therefore enters the estimates only through the number of old inputs, which determines the admissible thresholds.

As in Sec. 2.2, for a share length $n' \geq 2$ let $U_{\mathsf{L}}^{(n')}$ and $U_{\mathsf{R}}^{(n')}$ be the nonzero row and column vectors of length n' over $\mathbb{F}_2$, and let $\mu_s^{(n')}$ be the uniform distribution on pairs with inner product s. At $n' = n$ we omit the superscript. Independent uniform nonzero shares have inner product s with probability

$$p_s^{(n')} = \begin{cases} 2^{n'-1} / \left(2^{n'} - 1\right), & s = 1, \\ \left(2^{n'-1} - 1\right) / \left(2^{n'} - 1\right), & s = 0. \end{cases}$$

In particular, $p_s^{(n)} = p_s$, and $p_s^{(n')} \geq 1/3$.

Let $\mathcal{A}$ be a finite nonempty set indexing the possible old local data of either party, equipped with the uniform distribution. In our applications, we identify a row and its transposed column with the same bit string, so both parties use the same set $\mathcal{A}$. A party's role determines the orientation of its shares. Fix $m \geq 2$ and injective maps

$$a : \mathcal{A} \longrightarrow U_{\mathsf{L}}^{(m)}, \qquad b : \mathcal{A} \longrightarrow U_{\mathsf{R}}^{(m)}.$$

Thus every $a(L^\circ)$ is a nonzero row and every $b(R^\circ)$ is a nonzero column. The maps need not be linear. Here $L^\circ \in \mathcal{A}$ and $R^\circ \in \mathcal{A}$ are the old local data of the left and right parties, respectively, while $a(L^\circ)$ and $b(R^\circ)$ are the shares of length m to which the matrix refresh is applied:

- *Refreshing:* $L^\circ = L$, $R^\circ = R$, $a(L^\circ) = L$, $b(R^\circ) = R$, and $m = n$.
- *NAND computation:* $L^\circ = (L_1, L_2)$, $R^\circ = (R_1, R_2)$, $a(L^\circ) = (L_1 \otimes L_2, 1)$, $b(R^\circ) = \binom{R_1 \otimes R_2}{1}$, and $m = n^2 + 1$.

Both choices are injective: this is immediate for refreshing, and App. B.2 explains it for the tensor maps. Injectivity ensures that distinct old inputs give distinct nonzero vectors, which over $\mathbb{F}_2$ are linearly independent in pairs. The extension of the matrix estimates in the App. B.1 uses this fact. Put

$$d := |\mathcal{A}|, \qquad N_m := 2^m, \qquad q_m := N_m - 1, \qquad \mathcal{X} := \mathcal{A} \times U_{\mathsf{L}}^{(m)}, \qquad \mathcal{Y} := \mathcal{A} \times U_{\mathsf{R}}^{(m)}.$$

Injectivity gives $1 \le d \le q_m$. Let ν_{joint} be the uniform reference distribution on $\mathcal{X} \times \mathcal{Y}$. Under this distribution, L°, R°, U, and V are independent and uniform in their respective spaces. The refresh equations are not imposed. All unqualified norms of tables on $\mathcal{X} \times \mathcal{Y}$ use this distribution. In the actual refresh experiment, sample the coordinates as follows. Let L° and R° be independent uniform elements of $\mathcal{A}$, choose an independent matrix $M \leftarrow\!\$\ \mathsf{InvMatr}_2^m$, and set

$$U := a(L^\circ) \cdot M, \qquad V := M^{-1} \cdot b(R^\circ).$$

Write $x := (L^\circ, U)$, $y := (R^\circ, V)$, and $\omega := (x, y) = ((L^\circ, U), (R^\circ, V))$ for the grouped coordinates, which keep both the old local data and the refreshed shares, as needed when composing gadgets. We also write $\|F\|_\infty := \max_{x,y} |F(x, y)|$ for the largest absolute entry of a table. In averages indexed by L°, R°, U, and V, each named variable is sampled independently and uniformly from its indicated space, and M is uniform in $\mathsf{InvMatr}_2^m$. For any table $F : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ and fixed invertible matrix M, let $F^M : \mathcal{A} \times \mathcal{A} \to \mathbb{R}$ denote the table obtained by substituting the refresh equations:

$$F^M(L^\circ, R^\circ) := F\left((L^\circ, a(L^\circ) \cdot M), (R^\circ, M^{-1} \cdot b(R^\circ))\right).$$

The inner product $a(L^\circ) \cdot b(R^\circ) \in \mathbb{F}_2$ is the single bit encoded by the shares being refreshed. For refreshing, it equals $L \cdot R$. For a NAND gate, it equals $1 + (L_1 \cdot R_1) \cdot (L_2 \cdot R_2)$, its prescribed output bit.

Definition 5.3 (Consistent retained data and consistency projection)**.** *Call $\omega \in \mathcal{X} \times \mathcal{Y}$* consistent *when $U \cdot V = a(L^\circ) \cdot b(R^\circ)$. For every real table $F : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$, its* consistency projection *removes the entries at inconsistent tuples:*

$$(\mathsf{project}(F))\,((L^\circ, U), (R^\circ, V)) := \begin{cases} F\,((L^\circ, U), (R^\circ, V)), & U \cdot V = a(L^\circ) \cdot b(R^\circ), \\ 0, & U \cdot V \neq a(L^\circ) \cdot b(R^\circ). \end{cases}$$

This projection has the same properties as in Secs. 4.3.1 and 4.3.3: applying it twice changes nothing, and $(\mathsf{project}(F))^M = F^M$ for every invertible M. The refresh experiment, that is, the distribution of ω when L°, R°, and M are sampled as above, has density

$$\varphi\,((L^\circ, U), (R^\circ, V)) = \begin{cases} 1/p^{(m)}_{a(L^\circ) \cdot b(R^\circ)}, & U \cdot V = a(L^\circ) \cdot b(R^\circ), \\ 0, & U \cdot V \neq a(L^\circ) \cdot b(R^\circ) \end{cases}$$

relative to ν_{joint}: its probability at each tuple is φ times the reference probability. Indeed, for each fixed L° and R°, invertible matrices act transitively on all nonzero encodings of $a(L^\circ) \cdot b(R^\circ)$, so by the argument of Lem. 2.19 in dimension m the refreshed pair (U, V) is uniform on them. Since $p_s^{(m)} \ge 1/3$, this density is at most three.

For later use, define the sign table

$$J(x, y) := \begin{cases} 1/2, & (x, y) \text{ is consistent}, \\ -1/2, & \text{otherwise}. \end{cases}$$

Then $\mathsf{project}(F) = F/2 + J \cdot F$, where $J \cdot F$ is entrywise multiplication.

5.3.2 The extended approximation lemma

We now apply the singular-coefficient truncation of Eq. (4.7), retaining the old local data as part of each grouped coordinate. The resulting table must remain independent of the plaintext labels, so that later comparisons can change the encoding distributions without changing the approximation. Fix any formal incoming history and the adversary's random tape. Let $f_{\mathsf{L},z}(L^\circ, M)$ and $f_{\mathsf{R},z}(R^\circ, M)$ be the Boolean masks for one answer z (see Sec. 3). On consistent tuples, let $\mathsf{prob}_z\left((L^\circ, U), (R^\circ, V)\right)$ be their product averaged over the uniform matrices satisfying $a(L^\circ) \cdot M = U$ and $M^{-1} \cdot b(R^\circ) = V$. Extend prob_z by zero elsewhere. Thus $\mathsf{project}(\mathsf{prob}_z) = \mathsf{prob}_z$ and $0 \le \mathsf{prob}_z \le 1$. For any real table W, conditioning on the old and refreshed shares and then averaging gives the *answer-mask identity*

$$\langle \mathsf{prob}_z, W \rangle = \mathbb{E}_{L^\circ, R^\circ, M}\Big[p^{(m)}_{a(L^\circ)\cdot b(R^\circ)} \cdot f_{\mathsf{L},z}(L^\circ, M) \cdot f_{\mathsf{R},z}(R^\circ, M) \cdot W^M(L^\circ, R^\circ)\Big]. \tag{5.9}$$

Here L° and R° are independent uniform elements of $\mathcal{A}$, and $M \leftarrow\!\$\ \mathsf{InvMatr}_2^m$ is independent of them. Indeed, the density of the refresh tuples is $1/p^{(m)}_{a(L^\circ)\cdot b(R^\circ)}$ on their support, so the factor $p^{(m)}$ changes the refresh average back to the reference average. We also record that

$$\begin{aligned} p_0^{(m)} &= \frac{q_m - 1}{2q_m}, \\ p_1^{(m)} &= \frac{q_m + 1}{2q_m}, \\ |2p_s^{(m)} - 1| &= 1/q_m \qquad \text{for every } s \in \mathbb{F}_2. \end{aligned} \tag{5.10}$$

Lemma 5.4 (Approximation of an answer with retained local data)**.** *There exists an absolute constant $C > 0$ such that the following holds for every formal incoming history, adversary tape, answer z, and threshold $\beta \in [d^{-1/3}, 1]$. Take a probability-normalized orthonormal expansion (see Lem. 2.10) $\mathsf{prob}_z = \sum_j \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}$ and put*

$$I := \{j : \sigma_j > \beta\}, \qquad \mathsf{prob}_z^{>\beta} := \sum_{j \in I} \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}. \tag{5.11}$$

Then

$$\left\|\mathsf{prob}_z^{>\beta}\right\|_* \le 2, \tag{5.12}$$

$$\left\|\mathsf{prob}_z^{>\beta}\right\|_2 \le 1. \tag{5.13}$$

The balanced factors $\sqrt{\sigma_j} \cdot u_j$ and $\sqrt{\sigma_j} \cdot v_j$ of $\mathsf{prob}_z^{>\beta}$ (see Def. 2.17) have energies Φ_β and Ψ_β satisfying, for every grouped row $x \in \mathcal{X}$ and column $y \in \mathcal{Y}$,

$$\Phi_\beta(x) \le 1/\beta, \tag{5.14}$$

$$\Psi_\beta(y) \le 1/\beta, \tag{5.15}$$

$$\mathbb{E}_{x \leftarrow\!\$ \mathcal{X}}[\Phi_\beta(x)] = \left\|\mathsf{prob}_z^{>\beta}\right\|_* \le 2, \tag{5.16}$$

$$\mathbb{E}_{y \leftarrow\!\$ \mathcal{Y}}[\Psi_\beta(y)] = \left\|\mathsf{prob}_z^{>\beta}\right\|_* \le 2. \tag{5.17}$$

Finally,

$$\left\|\mathsf{prob}_z - 2 \cdot \mathsf{project}\Big(\mathsf{prob}_z^{>\beta}\Big)\right\|_2 \le C \cdot \beta^{1/3}. \tag{5.18}$$

The constant C is independent of the dimension, threshold, answer, formal history, and admissible masks. The construction depends on the masks and public maps, not on any plaintext labels.

App. B.1 proves this extension. By Eqs. (5.14) to (5.17), the approximation $2 \cdot \mathsf{prob}_z^{>\beta}$ has balanced energies $2 \cdot \Phi_\beta$ and $2 \cdot \Psi_\beta$, bounded pointwise by $2/\beta$ and in mean by 4. The factor two compensates for the consistency condition, which keeps roughly half of the independent tuples. The approximation $2 \cdot \mathsf{prob}_z^{>\beta}$ is a global table and may be nonzero on inconsistent tuples. Only the residual $\mathsf{prob}_z - 2 \cdot \mathsf{project}\left(\mathsf{prob}_z^{>\beta}\right)$ is zero there. On consistent tuples this residual equals $\mathsf{prob}_z - 2 \cdot \mathsf{prob}_z^{>\beta}$, so Eq. (5.18) also bounds its root-mean-square value under the actual refresh distribution by $\sqrt{3} \cdot C \cdot \beta^{1/3}$. The same bound holds for its mean absolute value by the Cauchy–Schwarz inequality (see Lem. 2.7). The nuclear bound Eq. (5.12) relies on the answer-mask identity Eq. (5.9), so it applies to leakage probability functions and not to arbitrary bounded consistent tables. If no coefficient is kept, then $\mathsf{prob}_z^{>\beta} = 0$, with all energies zero.

Proof outline. The projection and factor-energy bounds are unchanged from the refreshing proof. Three estimates must account for the number d of old inputs, which may differ from the number q_m of nonzero long shares. Injectivity makes the images of two distinct old inputs a uniform pair of distinct nonzero vectors under the random matrix. This gives the Gram-variance bound of Lem. 4.7, which controls the nuclear norm. Injectivity also allows the consistency sign to be treated as a submatrix of the full binary sign matrix. Finally, in the four-cycle calculation, coinciding old inputs have probability $1/d$. With these changes, the proof of Lem. 4.6 in Sec. 4.3.3 gives Eq. (5.12) and, for $t := \left\|\mathsf{prob}_z - 2 \cdot \mathsf{project}\left(\mathsf{prob}_z^{>\beta}\right)\right\|_2$,

$$t^2 \le C \cdot \beta^{1/2} \cdot t^{1/2} + C/(d \cdot \beta). \tag{5.19}$$

Since $d \cdot \beta^3 \ge 1$, Eq. (5.19) yields $t \le C \cdot \beta^{1/3}$ exactly as for refreshing. App. B.1 gives the changed estimates and the complete deductions. □

5.3.3 Transporting the approximation through shortening

The counterpart of Lem. 4.13 for a computation gadget retains three encodings: the two operands and the output. The approximation of Lem. 5.4 covers the matrix phase in dimension $n^2 + 1$: it retains the operands and accounts for the thin tensor support through the size d of the old data. Its table still has the refreshed long shares among its coordinates, however, and two steps remain. The table must instead depend on the short output encoding, because neighboring operations share that encoding, whereas the long shares are temporary and no other operation reads them. The answer must also include the shortening phase, whose leakage functions may read the message H and may depend on the matrix-phase answer. We therefore carry the matrix-phase approximation through the shortening step, for each pair of matrix-phase and shortening answers, and so obtain an approximation of the complete local view of the gadget.

The compiled execution has an exact *canonical* description: sample all wire encodings first, independently, and then sample gate answers sequentially from leakage probability functions conditioned on each gate's three encodings. This unconditional description supplies the independent wire variables in the composition argument. Conditioning on observed answers can create dependencies and is not used to justify independence. The description is exact because of the freshness of Lem. B.2 in the App. B: even conditional on its operand encodings and all internal state preceding the gadget, a completed gadget has a uniformly random encoding of its output bit. This conditioning does not include the fresh randomness drawn within the gadget. The answer at that gadget may depend on its output encoding, and the conditional leakage probability function records this dependence. Lem. B.5 in the App. B proves equality of the joint encoding-and-answer distributions. Tables below

group the three left shares into one coordinate and the three right shares into the other: write $U_{\mathsf{L}}^{S_i}$ and $U_{\mathsf{R}}^{S_i}$ for the sets of triples of nonzero left shares and of nonzero right shares indexed by the wires of S_i. Their norms use the corresponding product uniform measures.

Definition 5.5 (Local labelings and their norms)**.** *Put $q := 2^n - 1$ and fix a computation gadget i with scope S_i. A* local labeling *s assigns a bit to each of the three wires of S_i. It is* valid *when these bits satisfy the equation of gate i. For a local labeling s and a table F on S_i, that is, a real function of X_{S_i}, put*

$$\|F\|_{2,s} := \left(\mathbb{E}_{X_{S_i} \leftarrow\$ \bigotimes_{w \in S_i} \mu_{s(w)}} \left[|F(X_{S_i})|^2\right]\right)^{1/2}.$$

Unqualified norms still use independent uniform left and right shares.

Lemma 5.6 (A gate with its inputs and output retained)**.** *There exists an exact canonical experiment with conditional leakage probability functions $\mathsf{prob}_{i,h,z}(X_{S_i})$, defined on valid local labelings, with $\mathsf{prob}_{i,h,z} \geq 0$ and $\sum_z \mathsf{prob}_{i,h,z} = 1$. Its joint distribution of generated wire encodings and answers equals that of the actual execution. There exists an absolute constant $C > 0$ such that, for every threshold $\beta \in [q^{-2/3}, 1]$, computation operation i, formal incoming history $h \in \mathcal{Z}^{i-1}$, and answer $z \in \mathcal{Z}$, there exists a real table $G_{i,h,z}^{\beta}$, common to all wire labelings, with a product representation whose left and right energies Φ and Ψ (the sums of squared factors; see Def. 2.16) satisfy, for all grouped shares $x \in U_{\mathsf{L}}^{S_i}$ and $y \in U_{\mathsf{R}}^{S_i}$,*

$$\left\|G_{i,h,z}^{\beta}\right\|_* \leq 4, \tag{5.20}$$

$$\Phi(x) \leq 2/\beta, \tag{5.21}$$

$$\Psi(y) \leq 2/\beta, \tag{5.22}$$

$$\mathbb{E}_{x \leftarrow\$ U_{\mathsf{L}}^{S_i}}[\Phi(x)] \leq 4, \tag{5.23}$$

$$\mathbb{E}_{y \leftarrow\$ U_{\mathsf{R}}^{S_i}}[\Psi(y)] \leq 4, \tag{5.24}$$

$$\left\|\mathsf{prob}_{i,h,z} - G_{i,h,z}^{\beta}\right\|_{2,s} \leq C \cdot \beta^{1/3} \quad \textit{for every valid local labeling } s, \tag{5.25}$$

$$\left\|G_{i,h,z}^{\beta}\right\|_{2,s} \leq C_0 \quad \textit{for every local labeling } s. \tag{5.26}$$

Here $C_0 := 4 \cdot \sqrt{27}$, and C is independent of the dimension, threshold, answer, formal history, and admissible leakage functions.

The energies refer to the chosen product representation, which shortening can leave unbalanced, so Eqs. (5.23) and (5.24) state the two mean-energy bounds separately. The same approximation families are fixed for all formal histories before any wire labels are changed. Unlike true leakage probabilities, these tables may be negative and need not sum to one over the answers.

Proof outline. The matrix-phase argument applies the local approximation of Lem. 5.4 to the injective maps $(L_1, L_2) \mapsto (L_1 \otimes L_2, 1)$ and $(R_1, R_2) \mapsto (R_1 \otimes R_2, 1)$. Over $\mathbb{F}_2$, a nonzero rank-one tensor determines both nonzero factors, so the set $\mathcal{A}$ of old local data has size $d = q^2$. Consequently the admissible threshold range is $d^{-1/3} = q^{-2/3} \leq \beta \leq 1$. The long approximation is twice the part of the answer table, extended by zero to the inconsistent tuples, whose singular coefficients exceed β. It has nuclear norm at most 4 and energies at most $2/\beta$ (see Cor. B.1 in the App. B), and valid-label L_2 error at most $C \cdot \beta^{1/3}$ (see Eq. (B.11) in the App. B).

To reach the short output encoding, we describe a successful first shortening attempt in reverse (see Eq. (B.13) in the App. B): given the output (A, R), the matrix H is uniform of full row rank, V is

uniform in the fiber $\{v : H \cdot v = R\}$, and $U = A \cdot H$. These reverse sampling statements precede conditioning on either phase's leakage answer. Those answers enter through the leakage probability function and shortening masks. For fixed H, the left factor uses A and the retained left operands, while the right factor averages over the fiber determined by R and uses the retained right operands. The shortening masks use only these respective local data and the shortening tape. Neither may read the erased refresh matrix. Averaging over the fiber and inserting the masks therefore turns each product of the long table into one product for each H, yielding a product representation of the short table. A uniform short encoding followed by these reverse steps also gives exactly a uniform long encoding of the same bit, so the L_2 error is still averaged under the distribution for which it was bounded. Jensen's inequality and masks bounded by one control the L_2 error and pointwise energies. The uniform reverse marginals preserve each separate mean-energy bound (see Eqs. (B.20) to (B.23) and (B.25) in the App. B). The actual retry procedure adds at most $4/2^n$ per answer pointwise, which is absorbed by $C \cdot \beta^{1/3}$ throughout the stated range. Under any three wire labels, the canonical density relative to the independent-share reference measure is at most 27. Hence Eq. (5.26) follows from $\left\|G^\beta\right\|_{2,s} \le \sqrt{27} \cdot \left\|G^\beta\right\|_2 \le 4 \cdot \sqrt{27}$. Exact freshness gives the canonical execution (see Lem. B.5 in the App. B). App. B proves these statements with the required erasure and retry conditions. □

5.4 Composition with hidden output shares

So far we have approximated the leakage of each computation gadget and each input-port refresh separately. We now combine these local approximations into a composition bound for the entire execution. The argument compares the complete leakage transcripts of two executions in which the private, internal, and hidden output wires may carry different bits. The adversary sees only leakage from their encodings. The bits may also be chosen during the execution as functions of the preceding leakage answers. This covers public inputs that the adversary chooses between the invocations of a stateful circuit (see Sec. 6.3). Signed tables appear only inside the proof: the actual computation and the simulator continue to sample from ordinary probability distributions.

For continual refreshing we changed one label at a time along a chain, keeping at most two rounds signed (see Sec. 4.7). For the compiled circuit, we compare each execution with one common signed term that does not depend on the changing labels. Obtaining this term requires two replacements. At operations covered by the local approximation, we replace the true leakage probability function, which is defined only on tuples satisfying the operation's equation, by its approximation, whose table is common to all labelings. Dependence on the changing labels then remains only in the encoding distributions, so on those wires we also replace the fixed-bit encoding distribution by independent uniform shares. The main idea is therefore to expand two kinds of differences together: the error of a local approximation, and the difference between a fixed-bit encoding and independent uniform shares. After marking a set of these differences, we sum over all the unmarked differences, which restores the true leakage probabilities and encoding distributions away from the marked differences. The only remaining signed factors are then the marked differences themselves and the approximations at operations touching marked wires, so the local bounds accumulate linearly in the circuit size.

In this subsection, write $\mathcal{X} := U_{\mathsf{L}} \times U_{\mathsf{R}}$ for the encoding space. For any finite wire set W, a tuple $x = (x_w)_{w \in W} \in \mathcal{X}^W$ contains one share pair $x_w = (\ell_w, r_w)$ per wire. A table on grouped left and right shares is evaluated at x by grouping its left coordinates and its right coordinates. For intuition, consider a single operation that touches a single wire, with true factor $\mathsf{prob} =$

$G + e$ and encoding distribution μ_s. Then

$$\mathbb{E}_{X \leftarrow\$ \mu_s}[\mathsf{prob}(X)] = \mathbb{E}_{X \leftarrow\$ \nu}[G(X)] + \sum_{x \in \mathcal{X}} G(x) \cdot (\mu_s(x) - \nu(x)) + \mathbb{E}_{X \leftarrow\$ \mu_s}[e(X)].$$

The first term is a common reference term, independent of s. The second is a discrepancy term: a signed sum whose absolute value is bounded as a whole, so that its positive and negative weights can cancel. The third is the approximation error averaged under μ_s, the distribution for which its local bound holds. The expansion below organizes such terms for all operations at once.

5.4.1 The composition statement

We first state the composition bound in an abstract form that covers both the stateless compiler and its stateful extension in Sec. 6.3. Fix the adversary's tape and its auxiliary input (see Sec. 5.1). The protocol has S operations in a fixed order, each with a nonempty wire scope S_i of size at most three and an answer alphabet of size at most K. Each wire belongs to at most three scopes. Wires that lie in no scope may be omitted, since they do not affect the leakage transcript, so at most $3S$ wires are relevant. Let U be the full ambient set of wire locations in this experiment after that omission, including the public wires.

Let $\mathcal{I} \subseteq \{1, \ldots, S\}$ be the set of operations to which the local approximation applies. In the compiler these are the computation gadgets and the input-port refreshes. Every other operation has its scope entirely among public wires, whose labels are the same in the executions being compared. In the compiler these are the public-input samplers, which remain true factors throughout the proof. Let $\mathcal{V} \subseteq U$ be the set of remaining wires, including private input wires, internal wires, and hidden output wires. We call these the *changing wires*, because the comparison does not assume that their labels agree.

A valid labeling s assigns a bit to every wire and satisfies every operation's equation. We also allow labels that are chosen during the execution, as in Sec. 6.3, where the adversary chooses each public input after seeing earlier leakage. A *history-dependent labeling* s assigns to every wire w a bit $s_h(w)$ that is a function of the formal history h of the operations preceding the operation that produces w. The private input wires, which the trusted encoder supplies, have constant labels. For a complete formal history z, the *induced labeling* s_z assigns to w the bit $s_h(w)$ for the corresponding prefix h of z. The labeling s is *valid* if s_z is valid for every z, and two history-dependent labelings *agree on the public wires* if their induced labelings do so for every z. A labeling in the previous sense is the special case whose labels do not depend on the history. The canonical experiment runs the operations in order: before operation i, with incoming history h_{i-1}, it samples the encodings of the wires that operation i produces independently, $X_w \leftarrow\$ \mu_{s_{h_{i-1}}(w)}$, and then draws the answer z_i from the true conditional probabilities $\mathsf{prob}_{i,h_{i-1},z_i}(X_{S_i})$. The private input encodings are sampled first. These probabilities are nonnegative and sum to one for every valid local tuple and every formal incoming history. Denote the resulting transcript distribution by P_s. Because the label of a wire is fixed before the wire is produced, the encoding weights accumulated along a complete transcript $z = (z_1, \ldots, z_S)$ are exactly $\bigotimes_{w \in U} \mu_{s_z(w)}$, so with $h_{i-1} := (z_1, \ldots, z_{i-1})$,

$$P_s(z) = \mathbb{E}_{X_U \leftarrow\$ \bigotimes_{w \in U} \mu_{s_z(w)}} \left[\prod_{i=1}^{S} \mathsf{prob}_{i,h_{i-1},z_i}(X_{S_i}) \right]. \tag{5.27}$$

For labels that do not depend on the history, this is the experiment that samples all encodings before any answer is drawn. The wire encodings are independent in this unconditional expectation.

Conditioning on a transcript can create dependencies. The two shares of a fixed-bit encoding are generally correlated, whereas ν samples the two shares of a wire independently. The true tables, like the approximation families below, are fixed for all formal histories before labelings are compared. They depend on the labeling only through the wire encodings supplied to them.

Theorem 5.7 (Composition with hidden output shares)**.** *There exists an absolute constant C with the following property. Let $n \geq 2$ and $q := 2^n - 1$. Suppose the canonical experiment above is exact, and every $i \in \mathcal{I}$ has the local approximation and separate mean-energy bounds of Lems. 4.13 and 5.6 at $\beta = q^{-1/3}$. Then any two valid history-dependent labelings s and s' that agree on the public wires satisfy*

$$\delta(P_s, P_{s'}) \leq C \cdot S \cdot K^9 \cdot q^{-1/27}. \tag{5.28}$$

The right-hand side may be replaced by its minimum with one, and the same bound holds when the adversary's independent random tape is included in the view.

The threshold $q^{-1/3}$ is allowed for a dimension-n refresh by Lem. 4.13, and for a computation gadget by Lem. 5.6, whose allowed range starts at $q^{-2/3}$. The proof below uses only the valid-label L_2 errors and the separate mean-energy bounds of these approximations. They need be neither positive nor normalized over the answers.

5.4.2 Trimming and products of local errors

A wire can occur in three operations, so a product of local errors is controlled by Hölder's inequality with exponent three (see Claim 5.9), which requires L_3 errors. Trimming each side's factor energy separately turns the available L_2 error into the required L_3 error. Trimming also gives the uniform energy cap used to bound the product of tables at operations touching a marked wire (see Eq. (5.39)).

All wire and answer spaces in this section are finite, so the encoding and answer sums below are finite sums. The adversary's possibly infinite tape is fixed, and unbounded sampler records have already been averaged into the true local probabilities. For a local real table F, a local labeling s, and real $p \geq 1$, write

$$\|F\|_{p,s} := \left(\mathbb{E}_{X_{S_i} \leftarrow\$ \bigotimes_{w \in S_i} \mu_{s(w)}} [|F(X_{S_i})|^p]\right)^{1/p}.$$

Claim 5.8 (Trimming the factor energies)**.** *Let $0 < \beta \leq 1$ and $C_{\mathrm{err}} \geq 0$. On a fixed local scope, let* prob *be a table taking values in $[0,1]$ on every valid local tuple. Suppose G has a chosen product representation with energies Φ and Ψ (see Def. 2.16) such that $\mathbb{E}_x[\Phi(x)] \leq 4$ and $\mathbb{E}_y[\Psi(y)] \leq 4$ under the independent uniform grouped-share reference measures, and $\|\mathsf{prob} - G\|_{2,s} \leq C_{\mathrm{err}} \cdot \beta^{1/3}$ for every valid local labeling s. Set $B := \beta^{-1/3}$ and define*

$$\overline{G}(x,y) := \begin{cases} G(x,y), & \Phi(x) \leq B \text{ and } \Psi(y) \leq B, \\ 0, & \text{otherwise.} \end{cases} \tag{5.29}$$

This table has a product representation whose two pointwise energies are at most B. Moreover, for a constant C_1 depending only on C_{err} and every valid local labeling s,

$$\left\|\mathsf{prob} - \overline{G}\right\|_{3,s} \leq C_1 \cdot \beta^{1/9}. \tag{5.30}$$

Proof. Set every left factor to zero on rows with $\Phi(x) > B$ and every right factor to zero on columns with $\Psi(y) > B$, keeping the other factors. This gives Eq. (5.29). The new energies are at most B, and hence $|\overline{G}| \le B$ by Eq. (2.10).

Fix a valid local labeling s, and write $\mathbb{E}_s$ for the expectation over $X_{S_i} \leftarrow\!\$ \bigotimes_{w \in S_i} \mu_{s(w)}$. Under this distribution the grouped left shares are uniform, as are the grouped right shares. Markov's inequality therefore bounds the probability of the event "$\Phi(x) > B$ or $\Psi(y) > B$" by $8/B$. On this event $|\mathsf{prob} - \overline{G}| = \mathsf{prob} \le 1$. On its complement, $\overline{G} = G$ and $|G| \le B$, so

$$\begin{aligned}\mathbb{E}_s\Big[|\mathsf{prob} - \overline{G}|^3\Big] &\le (1+B) \cdot \mathbb{E}_s\Big[|\mathsf{prob} - G|^2\Big] + 8/B \\ &\le C_{\mathrm{err}}^2 \cdot (1+B) \cdot \beta^{2/3} + 8/B \le C \cdot \beta^{1/3},\end{aligned}$$

where C depends only on C_{err}. Taking the cube root proves Eq. (5.30). □

Claim 5.9 (Hölder's inequality for scopes of incidence three)**.** *Let $(X_w)_{w \in U}$ be independent random variables on finite spaces, indexed by a finite set U. Let I be a finite index set and, for each $i \in I$, let f_i be a nonnegative function of the variables in a scope $A_i \subseteq U$. Suppose each variable occurs in at most three scopes. Then, with expectations over all the variables,*

$$\mathbb{E}\left[\prod_{i \in I} f_i(X_{A_i})\right] \le \prod_{i \in I} \Big(\mathbb{E}\Big[f_i(X_{A_i})^3\Big]\Big)^{1/3}. \tag{5.31}$$

Proof. Induct on the number of variables. The case of no variables is an equality. At most three factors depend on a given variable X_w. Holding the other variables fixed, apply Hölder's inequality with exponent three to these factors, filling unused positions with the constant one. Replace each affected factor f_i by $(\mathbb{E}_{X_w}[f_i^3])^{1/3}$, now a function of the remaining variables in its scope. The incidence bound is unchanged. Applying the induction hypothesis to the remaining variables gives Eq. (5.31), because the cube of each replaced factor is $\mathbb{E}_{X_w}[f_i^3]$. □

Apply Claim 5.8 separately to each operation, formal history, and answer, fixing all the resulting tables once. From now on set

$$\beta := q^{-1/3}, \tag{5.32}$$

$$B := q^{1/9},$$

$$\varepsilon := C_1 \cdot q^{-1/27}, \tag{5.33}$$

$$e_{i,h,z} := \mathsf{prob}_{i,h,z} - \overline{G}_{i,h,z}.$$

Every $e_{i,h,z}$ is used only on valid local tuples, where its L_3 norm is at most ε by Eq. (5.30). If a value outside that domain is needed to make it a table, choose any extension. The proof never evaluates an error factor there.

5.4.3 A signed difference for each encoding

The trimmed approximations handle the leakage probabilities, and the second expansion concerns the encoding distributions. Recall from Def. 2.18 that ν is the uniform distribution on $U_{\mathsf{L}} \times U_{\mathsf{R}}$: it samples the two shares independently and does not encode a fixed bit. Define the signed weight function

$$D_b(x) := \mu_b(x) - \nu(x), \qquad b \in \{0,1\}, \quad x \in \mathcal{X}.$$

The weights $D_b(x)$ can be negative and are never sampled. In the expansion of Sec. 5.4.4, the term with ν on every changing wire and no approximation error does not depend on the private input. Every other term contains a difference D_b or an approximation error.

Claim 5.10 (Tensor discrepancy for encoding differences)**.** *Put $\gamma := 3/\sqrt{q}$. For every set W of wires, all bits $(b_w)_{w\in W}$, and every real table F on the grouped left and right shares of W,*

$$\left| \sum_{x\in\mathcal{X}^W} F(x) \cdot \prod_{w\in W} D_{b_w}(x_w) \right| \le \gamma^{|W|} \cdot \|F\|_* , \tag{5.34}$$

where the nuclear norm uses independent uniform grouped shares.

Proof. By Eqs. (2.11) and (2.12), $p_0 = (q-1)/(2q)$ and $p_1 = (q+1)/(2q)$. For $x = (\ell, r) \in \mathcal{X}$, the reference probability $\nu(x)$ is positive. The density of D_b relative to ν is therefore well-defined and equals

$$\varphi_b(x) := \frac{D_b(x)}{\nu(x)} = \frac{(-1)^b \cdot (-1)^{\ell\cdot r} + 1 - 2p_b}{2p_b}.$$

The signed sum in Eq. (5.34) is equivalently the ordinary expectation

$$\mathbb{E}_{X_W \leftarrow\$ \bigotimes_{w\in W} \nu} \left[F(X_W) \cdot \prod_{w\in W} \varphi_{b_w}(X_w) \right],$$

where the X_w are sampled independently from ν. The function being averaged can be negative. By Eq. (A.6) in the App. A.3, the largest singular coefficient of the sign table $(-1)^{\ell\cdot r}$ is at most $\sqrt{q+1}/q$. The constant table 1 has largest singular coefficient one, $p_b \ge 1/3$, and $|1 - 2p_b| = 1/q$. Consequently

$$\sigma_{\max}(\varphi_b) \le \frac{3}{2}\left(\frac{\sqrt{q+1}}{q} + \frac{1}{q}\right) \le \frac{3}{\sqrt{q}} = \gamma,$$

where the last inequality uses $q \ge 3$.

Orthonormal expansions of the individual densities tensor to an orthonormal expansion on the grouped coordinates, whose coefficients are products of the individual coefficients. The largest coefficient of $\prod_{w\in W} \varphi_{b_w}$ is therefore at most $\gamma^{|W|}$. The weighted sum in Eq. (5.34) is the pairing of F with this product density. Applying Eq. (2.7) to each term in an orthonormal expansion of F and summing its singular coefficients gives Eq. (5.34). □

5.4.4 Expanding and summing marked terms

We use two expansions: leakage probabilities are written as approximations plus errors, and fixed-bit encoding distributions as the independent uniform reference distribution plus signed differences. These expansions must be coordinated to use the local error bounds. An approximation error contains the true leakage probability function, which is defined only on tuples whose bits satisfy the equation of its operation, and Eq. (5.30) bounds the error only when these bits come from the fixed-bit encoding distributions of a valid labeling. Independent uniform shares on a wire of its scope can produce tuples outside this domain, where the true function is undefined and no bound is available. To use a local error bound, we must therefore preserve the original fixed-bit encoding distributions on the entire scope of its error factor. We choose the error operations first,

keep those wire distributions unchanged, and expand encoding differences only on the remaining changing wires, so the set of wires we may expand depends on the chosen error operations. The following sets record which wires must be kept unchanged and which operations can inspect a selected wire. For $A \subseteq \mathcal{I}$ and a set W of wires, let $\mathcal{V}_A$ be the set of wires of the operations in A, and let $\mathcal{N}(W)$ be the set of operations touching W:

$$\mathcal{V}_A := \bigcup_{i \in A} S_i, \qquad \mathcal{N}(W) := \{i : S_i \cap W \neq \varnothing\}.$$

Fix a complete formal history z. Abbreviate $\mathsf{prob}_{i,h_{i-1},z_i}$ to prob_i, and use analogous abbreviations for $\overline{G}_i$ and e_i. Also abbreviate the induced label $s_z(w)$ to $s(w)$. This fixes the tables and the labels without conditioning the encoding distributions on an observed transcript. The identities below hold for each z separately. Operation indices range over $\{1, \dots, S\}$, including the public-input samplers. In Eq. (5.27), expand $\mathsf{prob}_i = \overline{G}_i + e_i$ for $i \in \mathcal{I}$. In a term whose error operations are A, keep the original encoding weights on $\mathcal{V}_A$ and expand $\mu_{s(w)} = \nu + D_{s(w)}$ only on $\mathcal{V} \setminus \mathcal{V}_A$. This gives a finite identity

$$P_s(z) = \sum_{A \subseteq \mathcal{I}} \sum_{W \subseteq \mathcal{V} \setminus \mathcal{V}_A} T^s_{A,W}(z). \tag{5.35}$$

Explicitly, $T^s_{A,W}$ uses e_i at operations in A, $\overline{G}_i$ at operations in $\mathcal{I} \setminus A$, and the true factor prob_i at operations outside $\mathcal{I}$. Its coordinate weights are $D_{s(w)}$ on W, ν on $\mathcal{V} \setminus (\mathcal{V}_A \cup W)$, and $\mu_{s(w)}$ on every other wire of U. Thus every error factor keeps the original independent fixed-bit encoding distributions on its entire scope, where its L_3 bound holds. Error factors with overlapping scopes need not be independent, so their product is averaged under the joint encoding distribution.

Bounding the identity in Eq. (5.35) term by term would discard the normalization that the proof needs. A single term $T^s_{A,W}$ has an approximation $\overline{G}_i$ at every operation of $\mathcal{I} \setminus A$, and these tables may be negative and need not sum to one over the answers. Bounding its absolute mass over complete histories directly would therefore enumerate the answers of all those operations, at a cost of up to a factor K for each. To recover the normalization of the true probabilities, we instead group the expanded terms before bounding their absolute mass. For any chosen marks, we add every term that contains those differences. Summing the additional differences will restore true probability factors away from the marked errors and the operations touching marked wires. The bound on the absolute mass of each such sum depends on the numbers of marks rather than on the size of the circuit, and inclusion–exclusion in Sec. 5.4.5 recovers all the nonempty terms from these sums. For marked sets $A_\star \subseteq \mathcal{I}$ and $W_\star \subseteq \mathcal{V}$, let

$$F^s_{A_\star,W_\star} := \sum_{\substack{A \supseteq A_\star,\ W \supseteq W_\star \\ W \subseteq \mathcal{V} \setminus \mathcal{V}_A}} T^s_{A,W}. \tag{5.36}$$

This is the signed sum of all terms containing the marked differences, including any additional unmarked differences. If $W_\star \cap \mathcal{V}_{A_\star} \neq \varnothing$, the sum is empty and equals zero.

Claim 5.11 (Summing all unmarked differences)**.** *If $W_\star \cap \mathcal{V}_{A_\star} = \varnothing$, then*

$$\begin{aligned} F^s_{A_\star,W_\star}(z) = \sum_{x \in \mathcal{X}^U} &\left(\prod_{i \in A_\star} e_i(x_{S_i}) \right) \cdot \left(\prod_{i \in \mathcal{N}(W_\star)} \overline{G}_i(x_{S_i}) \right) \\ &\cdot \left(\prod_{i \notin A_\star \cup \mathcal{N}(W_\star)} \mathsf{prob}_i(x_{S_i}) \right) \\ &\cdot \left(\prod_{w \in W_\star} D_{s(w)}(x_w) \right) \cdot \left(\prod_{w \in U \setminus W_\star} \mu_{s(w)}(x_w) \right). \end{aligned} \tag{5.37}$$

The last product includes all fixed public wires, and the third product includes every operation outside $A_\star \cup \mathcal{N}(W_\star)$, including public-input samplers. Every error and true factor on the right is used only on valid local tuples with nonzero original encoding weight. Empty products equal one.

Proof. A term with $A \supseteq A_\star$ and $W \supseteq W_\star$ occurs in Eq. (5.36) only if $W_\star \cap \mathcal{V}_A = \varnothing$, that is, only if $A \cap \mathcal{N}(W_\star) = \varnothing$. For a fixed such A, sum first over all additional wire differences outside $W_\star$. Every unmarked changing-wire weight becomes $\nu + D_{s(w)} = \mu_{s(w)}$, and wires in $\mathcal{V}_A$ and fixed public wires already have this weight. The resulting coordinate weights no longer depend on A. Now sum over the additional error operations. At each operation in $\mathcal{I} \setminus (A_\star \cup \mathcal{N}(W_\star))$, the sum is $\overline{G}_i + e_i = \mathsf{prob}_i$. The operations in $\mathcal{N}(W_\star)$ cannot be error operations, so they keep $\overline{G}_i$, and they all lie in $\mathcal{I}$ because all other operations have fixed public scopes. This proves Eq. (5.37).

No true factor touches $W_\star$. Every wire in the scope of a true factor thus keeps its original fixed-bit encoding weight $\mu_{s(w)}$, so the labeling is valid wherever that factor is used. □

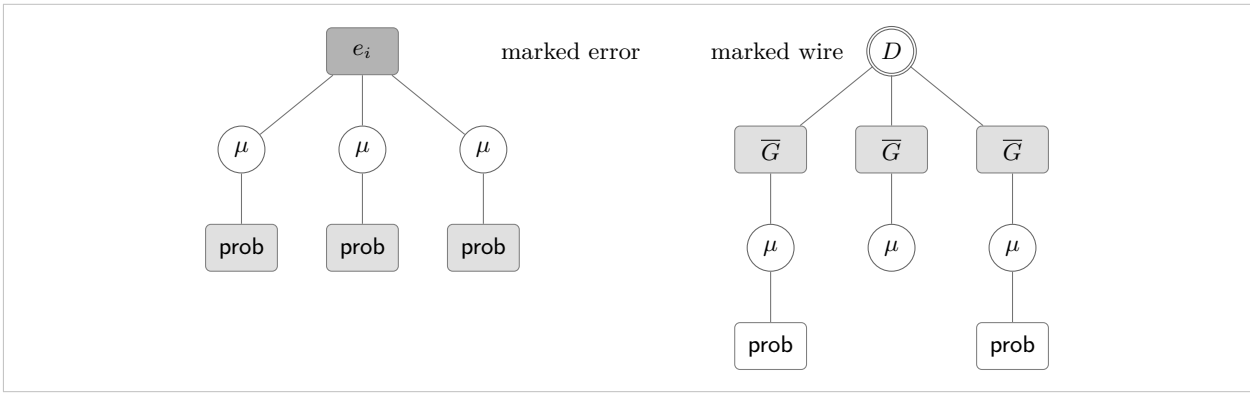


Figure 7: The structure of one marked term. Shaded operations are guards, whose answers are enumerated. Discrepancy removes the tables $\overline{G}$ around the marked wire, Hölder's inequality bounds the marked errors, the true factors at guards are bounded by one, and the remaining true factors sum to one.

Claim 5.12 (Bound for jointly marked differences)**.** *For all $A_\star \subseteq \mathcal{I}$ and $W_\star \subseteq \mathcal{V}$,*

$$\left\| F^s_{A_\star, W_\star} \right\|_1 = \sum_z |F^s_{A_\star, W_\star}(z)| \le (K^9 \cdot \varepsilon)^{|A_\star|} \cdot (K^3 \cdot B^3 \cdot \gamma)^{|W_\star|}. \tag{5.38}$$

This bound is uniform in the valid labeling and in all formal histories.

Proof. We may suppose that $W_\star \cap \mathcal{V}_{A_\star} = \varnothing$, since otherwise $F^s_{A_\star, W_\star} = 0$. Fix a complete history and all encodings outside $W_\star$. The product of the approximation factors in Eq. (5.37) is a table in the grouped left and right shares of $W_\star$:

$$\Pi_\star := \prod_{i \in \mathcal{N}(W_\star)} \overline{G}_i.$$

Multiply the chosen product representations of its $|\mathcal{N}(W_\star)| \le 3|W_\star|$ factors. The energy of the resulting left factors is the product of their left energies, and hence is at most $B^{3|W_\star|}$ pointwise. The same holds on the right. By Eq. (2.5) and the Cauchy–Schwarz inequality (see Lem. 2.7) over the factor indices,

$$\|\Pi_\star\|_* \le \sqrt{\mathbb{E}_{L_{W_\star} \leftarrow \$ U_{\mathsf{L}}^{W_\star}} [\Phi_{\Pi_\star}(L_{W_\star})]} \cdot \sqrt{\mathbb{E}_{R_{W_\star} \leftarrow \$ U_{\mathsf{R}}^{W_\star}} [\Psi_{\Pi_\star}(R_{W_\star})]} \le B^{3|W_\star|}.$$

Consequently, pointwise in all outside encodings, the tensor discrepancy Eq. (5.34) gives

$$\left| \sum_{x \in \mathcal{X}^{W_\star}} \Pi_\star(x) \cdot \prod_{w \in W_\star} D_{s(w)}(x_w) \right| \le B^{3|W_\star|} \cdot \gamma^{|W_\star|}. \tag{5.39}$$

Every other factor in Eq. (5.37) is independent of $W_\star$. The absolute value in Eq. (5.39) encloses the whole signed sum, preserving cancellation. This bound removes every signed approximation factor before any true factor is discarded.

We still need to sum the adaptive answers while averaging the remaining error factors jointly. These factors may share wire variables, and each depends on its own incoming answer prefix. The normalization of true probabilities cannot be used at every operation. A marked error and an operation touching a marked wire have no true factor, and their tables need not sum to one over the answers. The discrepancy bound above removes an approximation table, but its answer still has to be summed. A true factor that reads a wire of a marked error does sum to one, but it may depend on encodings over which the error bound must average, so it cannot in general be moved outside that average. If such a factor precedes a marked error, its answer can also help select that error's table through the incoming history, preventing us from summing it out first in general. We therefore enumerate the answers of these operations, which form a bounded set, so that the other true factors can be summed in reverse time order without changing the encoding distributions used for the error bound. We treat earlier and later neighbors alike, unlike in Sec. 4.6. This is simpler and leads to the factor K^9 for each marked error. Let

$$\mathcal{G}_\star := \mathcal{N}(\mathcal{V}_{A_\star}) \cup \mathcal{N}(W_\star).$$

We call this set of operations the *guards*: they are the operations that share a wire with a marked error or touch a marked wire (see Fig. 7). It contains $A_\star$ and has size at most $9|A_\star| + 3|W_\star|$, because every scope has at most three wires and every wire lies in at most three scopes. Operations outside $\mathcal{G}_\star$ have scopes disjoint from $\mathcal{V}_{A_\star} \cup W_\star$.

Write $X_{\text{out}} := X_{U \setminus (\mathcal{V}_{A_\star} \cup W_\star)}$ for the outside encodings. Enumerate the answers of the guards, at a cost of at most $K^{9|A_\star|+3|W_\star|}$, and discard the remaining true factors among the guards using $0 \le \mathsf{prob}_i \le 1$. Fix one such answer tuple $z_\star$ on $\mathcal{G}_\star$. For a complete history z extending $z_\star$, put

$$\Pi_z(X_{\text{out}}) := \prod_{i \notin \mathcal{G}_\star} \mathsf{prob}_{i,h_{i-1},z_i}(X_{S_i}).$$

Since operations outside $\mathcal{G}_\star$ have scopes disjoint from $\mathcal{V}_{A_\star} \cup W_\star$, the factor Π_z depends only on X_{out}, whereas the error factors depend only on $X_{\mathcal{V}_{A_\star}}$. After Eq. (5.39) it therefore remains to bound

$$\sum_{z:\, z|_{\mathcal{G}_\star} = z_\star} \sum_{X_{\text{out}}} \Big(\prod_{w \notin \mathcal{V}_{A_\star} \cup W_\star} \mu_{s_z(w)}(X_w) \Big) \cdot \Pi_z(X_{\text{out}}) \cdot \mathbb{E}_{X_{\mathcal{V}_{A_\star}} \leftarrow\$ \bigotimes_{w \in \mathcal{V}_{A_\star}} \mu_{s_z(w)}} \left[\prod_{i \in A_\star} |e_{i,h_{i-1},z_i}(X_{S_i})| \right].$$

For each complete history z, Claim 5.9 and the valid-label L_3 error Eq. (5.30), applied to the valid labeling s_z, give

$$\mathbb{E}_{X_{\mathcal{V}_{A_\star}} \leftarrow\$ \bigotimes_{w \in \mathcal{V}_{A_\star}} \mu_{s_z(w)}} \left[\prod_{i \in A_\star} |e_{i,h_{i-1},z_i}(X_{S_i})| \right] \le \varepsilon^{|A_\star|}. \tag{5.40}$$

The encodings in this expectation are sampled independently from their original fixed-bit distributions, and the bound is uniform in z. It remains to show that

$$\sum_{z:\, z|_{\mathcal{G}_\star}=z_\star} \sum_{X_{\mathrm{out}}} \Big(\prod_{w\notin \mathcal{V}_{A_\star}\cup W_\star} \mu_{s_z(w)}(X_w) \Big) \cdot \Pi_z(X_{\mathrm{out}}) = 1. \tag{5.41}$$

Sum the answers of the operations outside $\mathcal{G}_\star$ and the outside encodings in reverse time order, taking the answer of an operation before the encodings of the wires that it produces. Each true factor sums to one over its answer for every formal prefix and every valid local tuple, and the tuples with nonzero weight are valid because s_z is valid. Each encoding weight sums to one whatever its label is. When a variable is summed, nothing else depends on it: all later factors and weights have already been summed, earlier factors depend only on earlier answers and encodings, and the label of a wire is fixed by the answers preceding the operation that produces it. The enumerated guard answers remain in the prefixes but have no remaining factors of their own. This proves Eq. (5.41). For labels that do not depend on the history, it says that Π_z sums to one over the histories for every fixed X_{out} of nonzero weight. Combining Eqs. (5.39) to (5.41) with the count of guard answers proves Eq. (5.38). The original independent encoding distributions are never conditioned on an observed history. □

5.4.5 The common term and the final comparison

It remains to treat the one term without any difference, which is common to the compared executions, and to combine all bounds. The term $T^s_{\varnothing,\varnothing}$ in Eq. (5.35) uses ν on every wire in $\mathcal{V}$. Its remaining coordinate weights at a complete history z are the encoding distributions of the public bits $s_z(w)$, and all its tables are common to the labelings. Write $T_\circ := T^s_{\varnothing,\varnothing}$ for this common term. Thus $T_\circ$ is the same signed weight function for all valid history-dependent labelings that agree on the public wires. It need not be nonnegative or have total mass one, and we do not sample it.

Put

$$\rho_A := K^9 \cdot \varepsilon,$$
$$\rho_W := K^3 \cdot B^3 \cdot \gamma.$$

Inclusion–exclusion sums all nonempty terms through the marked sums:

$$P_s - T_\circ = \sum_{\substack{A_\star\subseteq\mathcal{I},\ W_\star\subseteq\mathcal{V} \\ (A_\star,W_\star)\neq(\varnothing,\varnothing)}} (-1)^{|A_\star|+|W_\star|+1} \cdot F^s_{A_\star,W_\star}.$$

To verify this finite identity, fix a nonempty term $T^s_{A,W}$. Its coefficient on the right is the sum of $(-1)^{|Y|+1}$ over all nonempty subsets Y of the disjoint union of its operation and wire indices, and this sum is one. The empty term has coefficient zero.

Taking norms only after this regrouping and using Eq. (5.38) gives

$$\|P_s - T_\circ\|_1 \le (1+\rho_A)^{|\mathcal{I}|} \cdot (1+\rho_W)^{|\mathcal{V}|} - 1. \tag{5.42}$$

For the parameters in Eqs. (5.32) to (5.33), we have

$$\rho_A \le C_1 \cdot K^9 \cdot q^{-1/27}, \tag{5.43}$$
$$\rho_W \le 3 \cdot K^3 \cdot q^{-1/6}. \tag{5.44}$$

Proof of Thm. 5.7. Subtract the same signed weight function $T_\circ$ from both real distributions. The triangle inequality, the factor $1/2$ in statistical distance, and Eq. (5.42) imply

$$\delta(P_s, P_{s'}) \le (1+\rho_A)^{|\mathcal{I}|} \cdot (1+\rho_W)^{|\mathcal{V}|} - 1.$$

Let $\tau := |\mathcal{I}| \cdot \rho_A + |\mathcal{V}| \cdot \rho_W$. If $\tau \le 1$, the right-hand side is at most $\exp(\tau) - 1 \le 2\tau$. If $\tau > 1$, use instead $\delta(P_s, P_{s'}) \le 1 \le \tau$. Hence the distance is at most 2τ in all cases. Now $|\mathcal{I}| \le S$, $|\mathcal{V}| \le 3S$, $K \ge 1$, and $q^{-1/6} \le q^{-1/27}$. Substituting Eqs. (5.43) and (5.44) proves Eq. (5.28), after enlarging the absolute constant. The case $S = 0$ has an empty transcript. Finally, all estimates are uniform in the adversary's fixed tape, so taking expectation over its independent random tape gives the same bound for the view that includes the tape. □

5.5 Concrete simulation and expected work

It remains to justify the claim of polynomial expected primitive work in Thm. 5.2. We do so by giving concrete sampling algorithms and accounting for every rejected proposal and retry. Recall from its proof that the simulator executes the compiled protocol on zero private inputs, keeps the public input, discards its dummy output encodings, and reports the supplied authorized output $\mathbf{y}$. The dummy computation need not produce $\mathbf{y}$, since the trusted decoder hides its output shares. Thm. 5.7 supplies the statistical comparison. The simulator's algorithm is fixed independently of the private input and target error, and $\mathbf{y}$ is available before its first simulated leakage answer.

Primitive work. *Primitive work* counts binary arithmetic, single-bit sampling, and indexed reads, writes, and comparisons of circuit addresses. Addresses are machine words. We use the same concrete samplers for real and simulated executions, retaining every accepted or rejected proposal in the prescribed local phase tape.

Concrete samplers. To initialize a uniform encoding of s, sample two independent uniform vectors in $\mathbb{F}_2^n$ and reject the pair unless both vectors are nonzero and their inner product is s. The accepted pair is uniform on the desired encoding fiber, and for $n \ge 2$ each proposal is accepted with probability at least $1/8$. The private-input encoder is trusted, so its initialization proposals are excluded from leakage. In a public-input sampler, which is not trusted, the entire local proposal tape is included in that operation's permitted leakage and budget. To sample a uniform invertible $n' \times n'$ binary matrix, sample a uniform matrix and reject it if singular. The acceptance probability exceeds $1/4$ by Eq. (4.44). To sample the nonzero row A, sample a uniform vector and reject zero. To sample the shortening matrix given A and U, fix the first index j with $A_j \ne 0$, sample all $n-1$ rows H_i with $i \ne j$ independently and uniformly in $\mathbb{F}_2^m$, and set

$$H_j := U - \sum_{i \ne j} A_i \cdot H_i.$$

Here $A_j = 1$, so the filled matrix satisfies $A \cdot H = U$. Reject the entire free-row proposal unless H has full row rank. Filling row j is a bijection from the free-row choices to all matrices satisfying the affine constraint, so the accepted matrix is uniform among the required full-row-rank completions. The acceptance probability is

$$\prod_{i=1}^{n-1} \left(1 - 2^{i-m}\right) > 1/4.$$

Indeed, after an invertible row transformation whose first row is A, the matrix has full row rank exactly when the $n-1$ uniform rows extend the fixed nonzero row U to a linearly independent

set. All rejected and accepted shortening proposals, together with all outer attempts, remain in the appropriate party's local tape, as allowed by the leakage model. Each nonzero-row proposal and each outer shortening attempt succeeds with probability at least 1/2.

Expected work. Each completed sampler record is a finite sequence of proposals over finite alphabets, but its length is unbounded, so these records have countable support. The uniform success bounds imply almost-sure termination. For a nonnegative integer work counter J, its expectation is the nonnegative series $\mathbb{E}[J] = \sum_{j=0}^{\infty} j \cdot \mathbb{P}[J = j]$. Nonnegativity justifies rearranging these series when combining costs. Each trial, rank test, inversion, tensor product, and projection uses polynomially many binary operations. Every operation produces one fresh encoding, so together with the private input encodings at most $I_{\mathrm{priv}} + S$ encoding locations are needed before erasures. The circuit makes polynomially many top-level sampler calls. Each nested rejection loop has a uniformly bounded expected number of attempts, conditional on the state entering that loop. Applying these conditional bounds from the inner loops outward and summing the resulting nonnegative expected costs gives a polynomial bound W on total expected primitive work.

A *local-answer evaluation* consists of the adversary's selection of one leakage function and its evaluation on the local state. A computation gadget requires four local-answer evaluations, an input-port refresh two, and a public-input sampler two, so by Eqs. (5.3) to (5.5) their total number is $8t_0 + 2I_{\mathrm{pub}}$. If one local-answer evaluation costs at most R_{ev}, expected total work is at most $W + (8t_0 + 2I_{\mathrm{pub}}) \cdot R_{\mathrm{ev}}$. More generally, expected total work is at most W plus the extended nonnegative expectation of the cost of the actual additional computation. That computation may depend on the same private sampling tapes, and its expected cost may be infinite. Bounded answer length alone does not bound this evaluation cost, and we assume no finite expected evaluation cost for arbitrary functions on unbounded tapes.

Capping the running time. The same simulator has strictly polynomial primitive work if we cap the number of shortening attempts and the trials of every rejection sampler. First cap the outer attempts, which bounds the number of inner sampling calls, and then cap each inner sampler. The caps can be chosen polynomially in the stated parameters so that a union bound over the geometric tails makes the probability of any cap being reached at most $2^{-\kappa-1}$, where $2^{-\kappa}$ is the target error. An arbitrary fixed termination rule on that event adds at most this probability to statistical distance. For the remaining error, use Eq. (5.7) with target error $2^{-\kappa-1}$. With a uniform per-evaluation bound R_{ev}, the capped simulator also has polynomial total work in the stated parameters and R_{ev}. The expected-work guarantee and the guarantee for the capped simulator are both stated for one evaluation. Thm. 6.1 extends the expected-work guarantee to repeated evaluation on persistent state with an online simulator that runs the same operations. The capped variant is not formalized (see Sec. 6.1).

6 Formalization and extensions

With the storage and single-evaluation compiler guarantees in place, we can describe precisely what is checked in Lean. We then discuss extensions to adaptive leakage within a refresh, stateful circuits, and larger finite fields. We prove and formalize the extension to stateful circuits in Sec. 6.3. We sketch the extension to adaptivity within a refresh and discuss the extension to larger fields. Neither of these last two extensions is formalized.

6.1 Lean formalization

Our formalization, written in Lean 4 [36] with mathlib [42], covers the binary continual-refreshing bound (see Thm. 4.1), the NAND compiler and its signed composition theorem with history-dependent labels (see Thms. 5.2 and 5.7), the concrete expected-work simulator of Sec. 5.5, and continual stateful privacy against adaptively chosen public inputs (see Thm. 6.1). The source files are available in the `version-1.0` folder of the project repository. A separate note, *Annotated Lean Reference Specifications*, is available at `lean/annotated-specifications.pdf`.

Lean checks that a proof establishes its formal statement. Whether that statement faithfully expresses the intended mathematical claim always remains for human readers to judge. Simplicity of the model and statements is therefore particularly important. We deliberately use deterministic strategies, with the adaptivity, locality, phase ordering, and erasures specified in the paper. Randomized adversaries and randomized leakage functions are handled by the averaging reduction of Sec. 3.4 and Sec. 5.2.1: fixing the adversary's tape and the presampled auxiliary choices gives a deterministic strategy, and the uniform bound is then averaged over those choices. This reduction, including its treatment of possibly infinite tapes, is a mathematical argument in the paper and is not part of the checked statements. The protocol's own sampling records, including rejected proposals and retries, remain in the checked model. The source interface is NAND-only and permits arbitrary fanout, repeated operands, zero gates, input wires designated directly as outputs, repeated output positions, and unused inputs. The AND/OR/NOT conversion in Sec. 5.1.1 is mathematical preprocessing outside that checked interface.

For the compiler, the formal development constructs the actual input-port refresh chains and proves their ordinary correctness, the I_{priv} trusted-encoding count, $S = 3t_0 + I_{\mathrm{pub}}$, distinct gadget operands, and scope and incidence bounds of three. The formal development also proves that the canonical experiment is exact and supplies the local approximation bounds for matrix refreshing and rectangular shortening, including full tapes of rejected proposals. The formal proof of Thm. 5.7 follows Sec. 5.4: energy trimming, the Hölder bound for incidence three, the joint expansion, tensor discrepancy, enumeration of guard answers under adaptive leakage, and inclusion–exclusion over marked sets. Its explicit finite bound is $10^6 \cdot S \cdot K^9 \cdot q^{-1/27}$, where $q = 2^n - 1$ and $K = 2^{2\lambda}$. The formal local certificates allow separate mean energies of eight rather than the analytical bound of four in Lem. 5.6. The proof carries this larger allowance through trimming and absorbs it into the displayed absolute coefficient. The construction and the exponents are unchanged. The checked compiler theorem supplies the local certificates, exact canonical distribution, and incidence bounds for the actual compiled program, rather than assuming those properties. Output encodings are hidden. The checked trusted decoder releases only the output tuple in the clear.

The simulator is fixed from the public source and supplied strategy before the private input and target error are quantified. It receives the authorized output before answering leakage queries, runs the actual operations on zero private inputs, and discards its dummy output pairs. The visible marginal of its counted execution, which includes every rejection and retry, is exactly this dummy-input view with the authorized output appended. To keep the formal cost model simple, its counter records random-bit sampling, scalar arithmetic and tests, and explicitly charged preparation operations, including the corresponding costs of every rejected proposal and shortening retry. Routine memory access and address bookkeeping are left to the conventional implementation analysis. This keeps the formal development focused on the probabilistic and cryptographic arguments at the core of the security proof. Writing $P := I_{\mathrm{priv}}$, Lean proves that the expected value of this counter is at most

$$W = P + k + 16P \cdot (6n + 1) + 100S \cdot (n^2 + 2)^3,$$

where k counts designated output positions, including repetitions. A uniform charge R_{ev} for each adversary and leakage evaluation adds exactly $(8t_0 + 2I_{\mathrm{pub}}) \cdot R_{\mathrm{ev}}$. Joint accounting can instead add

the expected cost of tape-dependent additional computation without an independence assumption. For $\kappa \geq 1$, a checked conservative sufficient length is

$$n \geq 12000 \cdot (\kappa + \lambda + \ln(S+2)), \qquad n \geq 2.$$

Here ln is the natural logarithm used in Lean. The exact simulator marginal and work bound hold at every $n \geq 2$, independently of this sufficient security threshold.

Each of the three public model files, `Refreshing.lean`, `Compiler.lean`, and `EfficientSimulation.lean`, has one bundled main theorem stated with `sorry`, Lean's placeholder for an omitted proof. Separate proof environments supply complete proofs and do not depend on those admitted declarations. `Refreshing.lean` states its components directly in its main theorem, whereas `Compiler.lean` and `EfficientSimulation.lean` define named component propositions with their numerical bounds and then bundle them in their main theorems. The public model files also admit needed construction certificates with `sorry`. Their complete proofs are checked separately by the Lean `comparator` tool, which checks the statements, definitions, and proof dependencies together. Substantive proofs, proof-only definitions, and proof-only theorem statements are confined to `lean/Proofs/`. Elementary construction proofs may appear inline in the public definitions. The repository describes the audit commands of the comparator and the permitted foundational axioms. These results certify the stated claims within the probability model and the specified cost convention. The results do not certify extracted machine code. The strictly polynomial variant obtained by capping the same simulator is not formalized. The extensions below to adaptivity within a phase and larger fields remain outside the formalized theorem statements. The stateful development constructs the chronological unrolling, proves the retained-state and trusted-decoder semantics, and extends composition to causal history-dependent labels. Its simulator is a function of an ideal oracle, independent of the initial state, and its complete view includes all public input/output records. The formal oracle interface describes the chronological requests and authorized responses mathematically. The counted stateful simulator uses the same cost convention and has the exact physical dummy-run marginal and expected counter bound obtained by replacing P by the initial-state size and k by the total number of output positions in the displayed bounds. Adversary computations, public-history processing, and ideal-oracle evaluation are charged separately.

6.2 Adaptivity within a refresh

Thm. 4.1 is stated for leakage that is adaptive across refreshes and nonadaptive within a refresh: the two leakage functions of a refresh are fixed before either answer of that refresh is revealed. The proof can be extended to leakage that is also adaptive within a refresh. In this stronger model, the adversary queries the parties' leakage functions one after another during the same refresh, choosing each function after seeing the earlier answers of that refresh, as long as each party leaks at most λ bits in total. We sketch the modifications. The theorems above and their formalization keep leakage nonadaptive within each phase.

Answers and masks. Fix the adversary's random tape. The interaction terminates after finitely many queries. The queried party, the length of each answer, and whether to stop are determined by this tape and the preceding answers. Lengths, timing, and termination carry no additional observations. The resulting binary transcript tree has depth at most 2λ, and its leaves are prefix-free, so padding its answer strings to 2λ bits is injective. Let z be this padded complete answer, so that the answer alphabet still has at most $K := 2^{2\lambda}$ elements. Fix the incoming history, the matrix M, and an answer z. The part of z that precedes an answer determines which leakage function produced that answer. Set $f_{\mathsf{L},z}(\ell, M) := 1$ if, for every left answer in z, the function determined

in this way outputs that answer on input (ℓ, M), and set it to zero otherwise. Define $f_{\mathsf{R},z}(r, M)$ in the same way for the right party. By induction along the sequence of answers, the old state (ℓ, r) produces z exactly when $f_{\mathsf{L},z}(\ell, M) \cdot f_{\mathsf{R},z}(r, M) = 1$. Since every old state produces exactly one answer, the masks satisfy the partition identity in Eq. (3.3) as before.

The rest of the proof. The local approximation in Sec. 4.3 treats each answer separately and uses its two Boolean masks. The approximation therefore provides the same per-answer nuclear, L_2, and energy estimates for complete interactive transcripts. The history argument of Sec. 4.6 uses only the number of answers, at most K, and the fact that the true probability functions sum to one. For private randomness in leakage functions, presample the auxiliary values for each possible query, indexed by the incoming history, the matrix, and the preceding answers within the refresh, and average the deterministic bounds.

The compiler. Apply the same complete-transcript masks separately to the matrix and shortening phases, with total per-party budgets λ_{M} and λ_{S} and at most $K = 2^{2(\lambda_{\mathsf{M}}+\lambda_{\mathsf{S}})}$ complete answers. For shortening, each complete transcript still determines a left mask and a right mask. The per-answer transport in Lem. B.4 in the App. B uses only their bounds in $[0, 1]$, so it transports the chosen energies and L_2 error without requiring separate answer partitions on the two sides. The proof of Thm. 5.7 uses these per-answer estimates and the normalization of the true complete-transcript probabilities. Erasure before shortening remains essential, and the shortening budget covers every retry. Auxiliary randomness is presampled as in Sec. 5.2.1, indexed by the gate and the preceding answers. Erased matrix state is unavailable during shortening. These observations describe the extension. We do not give a separate full proof for interactive leakage.

6.3 Stateful circuits

We now extend Thm. 5.2 to stateful circuits that are invoked repeatedly, with the input of each invocation chosen by the adversary after it has seen the leakage of the earlier ones. This is the continual-leakage setting for computation: the total leakage grows without bound in the number of invocations. A stateful circuit has registers whose contents persist between invocations. Fix state, input, and output lengths m_{state}, m_{in}, and m_{out}. Starting from an initial state $\mathsf{st}_0 \in \mathbb{F}_2^{m_{\text{state}}}$, its transition circuit computes on every invocation $t \geq 0$

$$(\mathsf{st}_{t+1}, \mathbf{y}_t) := \Gamma(\mathsf{st}_t, \mathbf{x}_t),$$

where $\Gamma : \mathbb{F}_2^{m_{\text{state}}} \times \mathbb{F}_2^{m_{\text{in}}} \to \mathbb{F}_2^{m_{\text{state}}} \times \mathbb{F}_2^{m_{\text{out}}}$ is a NAND source circuit as in Sec. 5.1.1, with t_0 gates and I_{const} public constants, st_t is the current private state, $\mathbf{x}_t \in \mathbb{F}_2^{m_{\text{in}}}$ is the public input of invocation t, and $\mathbf{y}_t \in \mathbb{F}_2^{m_{\text{out}}}$ is its public output. The registers supply st_{t+1} to the next invocation. Every feedback cycle passes through a register. See Ishai, Sahai, and Wagner [28, Section 2] for this model and its privacy definition against probing attacks.

The compiled stateful protocol. Fix the number $T \geq 1$ of invocations. The trusted encoder encodes the bits of st_0 independently, as in Sec. 5.1. These are the only trusted encodings of the entire execution. Invocation t starts once the adversary has chosen $\mathbf{x}_t$. It runs the operations of the execution schedule of Sec. 5.1 for Γ, with the current state encodings as its private operand encodings: the public-input samplers for the bits of $\mathbf{x}_t$, the input-port refreshes, and the NAND gadgets. At the end of the invocation, the trusted decoder reads the encodings of the output wires and releases $\mathbf{y}_t$. As before, it releases no shares and produces no leakage. The encodings of the wires that carry st_{t+1} are retained and serve as source wires of invocation $t + 1$: each is refreshed by its input-port chain immediately before every use, and an old copy is erased once all

of its uses, including any output position, have finished. All other encodings of invocation t are erased as in the single-evaluation protocol. Retained encodings are inactive between invocations and inaccessible to every leakage query. The public constants are sampled once, at the start of the execution, and reused through their input-port chains.

Unrolling the T invocations gives one NAND source circuit $\Gamma^{(T)}$ with private input st_0, public inputs $\mathbf{x}_0, \ldots, \mathbf{x}_{T-1}$ and the constants, and designated outputs $\mathbf{y}_0, \ldots, \mathbf{y}_{T-1}$, in which the state wires of invocation t feed the gates of invocation $t+1$. The compiled stateful protocol is the compiled protocol of Sec. 5.1 for $\Gamma^{(T)}$, except that the samplers for $\mathbf{x}_t$ run at the start of invocation t instead of at the start of the execution, and the output wires of invocation t are decoded at the end of that invocation instead of at the end of the execution. Neither change affects the scopes: every operation reads or produces at most three wires, and every wire belongs to at most three scopes, by the argument given after the execution schedule in Sec. 5.1. By Eq. (5.6), the number of operations is

$$S = 3T \cdot t_0 + T \cdot m_{\text{in}} + I_{\text{const}}. \tag{6.1}$$

Leakage and security. Leakage is as in Sec. 5.1 for every operation of every invocation: each operation has its stated budget, each query reads only its party's data for the active operation, both functions of a phase are fixed before either answer is returned, and communication is private. The adversary is adaptive across the whole execution. At the start of invocation t, it chooses $\mathbf{x}_t$ as a function of its auxiliary input, its tape, all leakage answers so far, and $\mathbf{y}_0, \ldots, \mathbf{y}_{t-1}$. During the invocation it chooses the leakage functions of each phase as a function of the same data together with $\mathbf{x}_t$. Its *view* consists of all leakage answers, the inputs $\mathbf{x}_0, \ldots, \mathbf{x}_{T-1}$, and the outputs $\mathbf{y}_0, \ldots, \mathbf{y}_{T-1}$. The *ideal stateful circuit* with initial state st_0 answers the queries $\mathbf{x}_0, \mathbf{x}_1, \ldots$ in turn with the outputs $\mathbf{y}_0, \mathbf{y}_1, \ldots$ of the transition circuit. A *simulator* interacts with the adversary in place of the compiled protocol and has oracle access to the ideal stateful circuit. It does not receive st_0.

Theorem 6.1 (Continual leakage for stateful circuits)**.** *There exist absolute constants $C \geq 1$ and $C' \geq 1$ with the following property. For every transition circuit, dimension $n \geq 2$, pair of nonnegative integer phase budgets, number of invocations $T \geq 1$, and admissible adaptive adversary, there exists a simulator, independent of the initial state, such that for every st_0 the adversary's views in the compiled stateful protocol and in the interaction with the simulator are at statistical distance at most*

$$\min\left\{1,\ C \cdot S \cdot K^9 \cdot (2^n - 1)^{-1/27}\right\} \leq C' \cdot S \cdot 2^{18\lambda - n/27}, \tag{6.2}$$

with S from Eq. (6.1) and $K = 2^{2\lambda}$. The simulator answers every query when it is asked and uses $\mathbf{y}_t$ only at the end of invocation t. Its expected primitive work is polynomial in the circuit and input sizes, T, and n, with $8T \cdot t_0 + 2(T \cdot m_{\text{in}} + I_{\text{const}})$ local-answer evaluations, adversary and leakage computation being charged separately as in Sec. 5.5. For every fixed $\alpha < 1/486$, choosing $\lambda \leq \alpha \cdot n$ gives error $S \cdot 2^{-\Omega(n)}$, so share length $n = O(\kappa + \lambda + \log(S+2))$ suffices for target distance $2^{-\kappa}$: the share length grows only logarithmically in the number of invocations, while the total leakage $2\lambda S$ grows linearly.

Proof. The simulator. The simulator encodes the all-zero initial state and, on receiving $\mathbf{x}_t$, runs invocation t of the compiled protocol on its dummy state encodings against the adversary, answering each leakage query from this execution. At the end of invocation t, it queries the ideal stateful circuit with $\mathbf{x}_t$ and returns its authorized response $\mathbf{y}_t$, without revealing the dummy output values or encodings. It retains any encodings needed for the next state or subsequent uses. An encoding may be erased only after its last required use has finished. Its operations are those of the compiled protocol for $\Gamma^{(T)}$, so the expected-work bound and the count of local-answer evaluations follow from Sec. 5.5 with the operation counts of Eq. (6.1).

The view is a function of the transcript. Fix the adversary's tape, its auxiliary input, and st_0. In both experiments, $\mathbf{y}_t$ is the output of the ideal stateful circuit on st_0 and $\mathbf{x}_0, \ldots, \mathbf{x}_t$: in the real experiment because every refresh and gadget is correct, so that the decoder releases the values of the output wires in the clear, and in the simulated experiment by construction. By induction on t, the inputs $\mathbf{x}_t$ and outputs $\mathbf{y}_t$ are therefore the same deterministic functions of the leakage answers preceding invocation t in both experiments, and the view is the same deterministic function of the leakage transcript in both. It suffices to bound the statistical distance between the two transcript distributions.

Two history-dependent labelings. With st_0 fixed, the leakage functions of every phase and the input of every invocation are functions of the preceding formal history. Thus the true tables $\mathsf{prob}_{i,h,z}$ and the labels of the public wires are fixed for all formal histories, as Sec. 5.4.1 requires. These tables and public labels are the same in both experiments. The real execution carries the history-dependent labeling s given by the plaintext values of $\Gamma^{(T)}$ on st_0 and these public labels, and the simulated execution carries the labeling s' given by the plaintext values on $0^{m_{\mathrm{state}}}$ and the same public labels. Both are valid for every history, and they agree on the public wires. The label of a wire of invocation t depends only on st_0 and $\mathbf{x}_0, \ldots, \mathbf{x}_t$, hence only on the answers preceding invocation t, which precede the operation producing the wire. Decoding is not an operation and produces no answer, so it does not enter the transcript.

Composition. The canonical experiment of the compiled stateful protocol is exact by Lem. B.5 in the App. B and the remark following it. Every computation gadget and input-port refresh has the local approximation and mean-energy bounds of Lems. 4.13 and 5.6. These bounds hold for every formal history and do not depend on the schedule. Together with the scope and incidence bounds noted above, Thm. 5.7 applies to s and s' and gives Eq. (6.2) by Eq. (5.28), including for a view that contains the adversary's tape. As in the proof of Thm. 5.2, the binary form and the parameter choice follow. □

The theorem keeps the assumptions of Sec. 5.1: a trusted encoder for the initial state, a trusted decoder at the end of every invocation, private channels, erasures, and leakage that is nonadaptive within a phase. Its causal composition bound, chronological compiler, decoder correctness, oracle simulation, and expected counter bound are formalized in Lean under the cost convention of Sec. 6.1.

6.4 General fields

The matrix action and inner-product encoding are defined over any finite field. Extending the quantitative security bounds requires adapting the local estimates and source compilation. We do not prove that extension here. All quantitative theorems and their Lean specifications in this paper use the binary field $\mathbb{F}_2$.

Acknowledgments

The work of Stefan Dziembowski was supported by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (project PROCONTRA, grant agreement No. 885666). The work of Sebastian Faust was supported by the European Research Council (ERC) under the European Union's Horizon Europe research and innovation programme (project CRYPTOLAYER, grant agreement No. 101044770).

References

[1] M. Ajtai. "Secure computation with information leaking to an adversary". In: *Proceedings of the 43rd ACM Symposium on Theory of Computing, STOC 2011, San Jose, CA, USA, 6-8 June 2011*. Ed. by L. Fortnow and S. P. Vadhan. ACM, 2011, pp. 715–724.

[2] A. Akavia, S. Goldwasser, and V. Vaikuntanathan. "Simultaneous Hardcore Bits and Cryptography against Memory Attacks". In: *Theory of Cryptography, 6th Theory of Cryptography Conference, TCC 2009, San Francisco, CA, USA, March 15-17, 2009. Proceedings*. Ed. by O. Reingold. Vol. 5444. Lecture Notes in Computer Science. Springer, 2009, pp. 474–495.

[3] J. Alwen, Y. Dodis, and D. Wichs. "Leakage-Resilient Public-Key Cryptography in the Bounded-Retrieval Model". In: *Advances in Cryptology - CRYPTO 2009, 29th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2009. Proceedings*. Ed. by S. Halevi. Vol. 5677. Lecture Notes in Computer Science. Springer, 2009, pp. 36–54.

[4] M. Andrychowicz. "Efficient Refreshing Protocol for Leakage-Resilient Storage Based on the Inner-Product Extractor". In: *CoRR* abs/1209.4820 (2012).

[5] M. Andrychowicz, S. Dziembowski, and S. Faust. "Circuit Compilers with $O(1/\log(n))$ Leakage Rate". In: *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II*. Ed. by M. Fischlin and J. Coron. Vol. 9666. Lecture Notes in Computer Science. Springer, 2016, pp. 586–615.

[6] S. Axler. *Linear Algebra Done Right*. 4th ed. Undergraduate Texts in Mathematics. Cham: Springer International Publishing, 2023.

[7] B. Barak, O. Goldreich, R. Impagliazzo, S. Rudich, A. Sahai, S. P. Vadhan, and K. Yang. "On the (im)possibility of obfuscating programs". In: *J. ACM* 59.2 (2012), 6:1–6:48.

[8] F. Benhamouda, A. Degwekar, Y. Ishai, and T. Rabin. "On the Local Leakage Resilience of Linear Secret Sharing Schemes". In: *J. Cryptol.* 34.2 (2021), p. 10.

[9] N. Bitansky, D. Dachman-Soled, and H. Lin. "Leakage-tolerant computation with input-independent preprocessing". In: *Advances in Cryptology – CRYPTO 2014, Part II*. Ed. by J. A. Garay and R. Gennaro. Vol. 8617. Lecture Notes in Computer Science. Springer, 2014, pp. 146–163.

[10] J. Bourgain. "More on the sum-product phenomenon in prime fields and its applications". In: *International Journal of Number Theory* 01.01 (2005), pp. 1–32.

[11] Z. Brakerski, Y. T. Kalai, J. Katz, and V. Vaikuntanathan. "Overcoming the Hole in the Bucket: Public-Key Cryptography Resilient to Continual Memory Leakage". In: *51st Annual IEEE Symposium on Foundations of Computer Science, FOCS 2010, Las Vegas, Nevada, USA, October 23-26, 2010*. IEEE Computer Society, 2010, pp. 501–510.

[12] G. Brian, S. Dziembowski, and S. Faust. "From Random Probing to Noisy Leakages Without Field-Size Dependence". In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part IV*. Ed. by M. Joye and G. Leander. Vol. 14654. Lecture Notes in Computer Science. Springer, 2024, pp. 345–374.

[13] S. Chari, C. S. Jutla, J. R. Rao, and P. Rohatgi. "Towards Sound Approaches to Counteract Power-Analysis Attacks". In: *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*. Ed. by M. J. Wiener. Vol. 1666. Lecture Notes in Computer Science. Springer, 1999, pp. 398–412.

[14] E. Chattopadhyay and D. Zuckerman. "Explicit two-source extractors and resilient functions". In: *Annals of Mathematics* 189.3 (2019), pp. 653–705.

[15] B. Chor and O. Goldreich. "Unbiased Bits from Sources of Weak Randomness and Probabilistic Communication Complexity". In: *SIAM J. Comput.* 17.2 (1988), pp. 230–261.

[16] D. Dachman-Soled, F.-H. Liu, and H.-S. Zhou. "Leakage-resilient circuits revisited – Optimal number of computing components without leak-free hardware". In: *Advances in Cryptology – EUROCRYPT 2015, Part II*. Ed. by E. Oswald and M. Fischlin. Vol. 9057. Lecture Notes in Computer Science. Springer, 2015, pp. 131–158.

[17] F. Davì, S. Dziembowski, and D. Venturi. "Leakage-Resilient Storage". In: *Security and Cryptography for Networks, 7th International Conference, SCN 2010, Amalfi, Italy, September 13-15, 2010. Proceedings*. Ed. by J. A. Garay and R. D. Prisco. Vol. 6280. Lecture Notes in Computer Science. Springer, 2010, pp. 121–137.

[18] Y. Dodis, K. Haralambiev, A. López-Alt, and D. Wichs. "Cryptography against Continuous Memory Attacks". In: *51st Annual IEEE Symposium on Foundations of Computer Science, FOCS 2010, Las Vegas, Nevada, USA, October 23-26, 2010*. IEEE Computer Society, 2010, pp. 511–520.

[19] A. Duc, S. Dziembowski, and S. Faust. "Unifying Leakage Models: From Probing Attacks to Noisy Leakage". In: *Advances in Cryptology - EUROCRYPT 2014 - 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings*. Ed. by P. Q. Nguyen and E. Oswald. Vol. 8441. Lecture Notes in Computer Science. Springer, 2014, pp. 423–440.

[20] S. Dziembowski and S. Faust. "Leakage-Resilient Cryptography from the Inner-Product Extractor". In: *Advances in Cryptology - ASIACRYPT 2011 - 17th International Conference on the Theory and Application of Cryptology and Information Security, Seoul, South Korea, December 4-8, 2011. Proceedings*. Ed. by D. H. Lee and X. Wang. Vol. 7073. Lecture Notes in Computer Science. Full version: Cryptology ePrint Archive, Report 2011/519, `https://eprint.iacr.org/2011/519`. Springer, 2011, pp. 702–721.

[21] S. Dziembowski and S. Faust. "Leakage-Resilient Circuits without Computational Assumptions". In: *Theory of Cryptography - 9th Theory of Cryptography Conference, TCC 2012, Taormina, Sicily, Italy, March 19-21, 2012. Proceedings*. Ed. by R. Cramer. Vol. 7194. Lecture Notes in Computer Science. Springer, 2012, pp. 230–247.

[22] S. Dziembowski, S. Faust, and K. Żebrowski. "Simple Refreshing in the Noisy Leakage Model". In: *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part III*. Ed. by S. D. Galbraith and S. Moriai. Vol. 11923. Lecture Notes in Computer Science. Springer, 2019, pp. 315–344.

[23] S. Dziembowski and K. Pietrzak. "Leakage-Resilient Cryptography". In: *49th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2008, Philadelphia, PA, USA, October 25-28, 2008*. IEEE Computer Society, 2008, pp. 293–302.

[24] S. Faust, T. Rabin, L. Reyzin, E. Tromer, and V. Vaikuntanathan. "Protecting Circuits from Leakage: the Computationally-Bounded and Noisy Cases". In: *Advances in Cryptology - EUROCRYPT 2010, 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Monaco / French Riviera, May 30 - June 3, 2010. Proceedings*. Ed. by H. Gilbert. Vol. 6110. Lecture Notes in Computer Science. Springer, 2010, pp. 135–156.

[25] S. Goldwasser and G. N. Rothblum. "Securing Computation against Continuous Leakage". In: *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*. Ed. by T. Rabin. Vol. 6223. Lecture Notes in Computer Science. Springer, 2010, pp. 59–79.

[26] S. Goldwasser and G. N. Rothblum. "How to Compute in the Presence of Leakage". In: *53rd Annual IEEE Symposium on Foundations of Computer Science, FOCS 2012, New Brunswick, NJ, USA, October 20-23, 2012*. IEEE Computer Society, 2012, pp. 31–40.

[27] S. Goldwasser and G. N. Rothblum. "How to Compute in the Presence of Leakage". In: *SIAM Journal on Computing* 44.5 (2015), pp. 1480–1549.

[28] Y. Ishai, A. Sahai, and D. A. Wagner. "Private Circuits: Securing Hardware against Probing Attacks". In: *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*. Ed. by D. Boneh. Vol. 2729. Lecture Notes in Computer Science. Springer, 2003, pp. 463–481.

[29] Y. Ishai and Y. Song. "Leakage-Tolerant Circuits". In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part IV*. Ed. by M. Joye and G. Leander. Vol. 14654. Lecture Notes in Computer Science. Springer, 2024, pp. 196–225.

[30] S. Jukna. *Extremal combinatorics: With applications in computer science*. 2nd ed. Texts in Theoretical Computer Science. An EATCS Series. Berlin, Heidelberg: Springer, 2011.

[31] A. Juma and Y. Vahlis. "Protecting Cryptographic Keys against Continual Leakage". In: *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*. Ed. by T. Rabin. Vol. 6223. Lecture Notes in Computer Science. Springer, 2010, pp. 41–58.

[32] Y. T. Kalai and L. Reyzin. "A survey of leakage-resilient cryptography". In: *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*. Ed. by O. Goldreich. ACM, 2019, pp. 727–794.

[33] A. B. Lewko, M. Lewko, and B. Waters. "How to leak on key updates". In: *Proceedings of the 43rd ACM Symposium on Theory of Computing, STOC 2011, San Jose, CA, USA, 6-8 June 2011*. Ed. by L. Fortnow and S. P. Vadhan. ACM, 2011, pp. 725–734.

[34] S. Micali and L. Reyzin. "Physically Observable Cryptography (Extended Abstract)". In: *Theory of Cryptography, First Theory of Cryptography Conference, TCC 2004, Cambridge, MA, USA, February 19-21, 2004, Proceedings*. Ed. by M. Naor. Vol. 2951. Lecture Notes in Computer Science. Springer, 2004, pp. 278–296.

[35] E. Miles and E. Viola. "Shielding circuits with groups". In: *Symposium on Theory of Computing Conference, STOC'13, Palo Alto, CA, USA, June 1-4, 2013*. Ed. by D. Boneh, T. Roughgarden, and J. Feigenbaum. ACM, 2013, pp. 251–260.

[36] L. de Moura and S. Ullrich. "The Lean 4 Theorem Prover and Programming Language". In: *Automated Deduction - CADE 28 - 28th International Conference on Automated Deduction, Virtual Event, July 12-15, 2021, Proceedings*. Ed. by A. Platzer and G. Sutcliffe. Vol. 12699. Lecture Notes in Computer Science. Springer, 2021, pp. 625–635.

[37] M. Naor and G. Segev. "Public-Key Cryptosystems Resilient to Key Leakage". In: *Advances in Cryptology - CRYPTO 2009, 29th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2009. Proceedings*. Ed. by S. Halevi. Vol. 5677. Lecture Notes in Computer Science. Springer, 2009, pp. 18–35.

[38] K. Pietrzak. "A Leakage-Resilient Mode of Operation". In: *Advances in Cryptology - EUROCRYPT 2009, 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings*. Ed. by A. Joux. Vol. 5479. Lecture Notes in Computer Science. Springer, 2009, pp. 462–482.

[39] E. Prouff and M. Rivain. "Masking against Side-Channel Attacks: A Formal Security Proof". In: *Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings*. Ed. by T. Johansson and P. Q. Nguyen. Vol. 7881. Lecture Notes in Computer Science. Springer, 2013, pp. 142–159.

[40] G. N. Rothblum. "How to Compute under AC^0 Leakage without Secure Hardware". In: *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*. Ed. by R. Safavi-Naini and R. Canetti. Vol. 7417. Lecture Notes in Computer Science. Springer, 2012, pp. 552–569.

[41] N. Srebro, J. D. M. Rennie, and T. S. Jaakkola. "Maximum-Margin Matrix Factorization". In: *Advances in Neural Information Processing Systems 17 [Neural Information Processing Systems, NIPS 2004, December 13-18, 2004, Vancouver, British Columbia, Canada]*. 2004, pp. 1329–1336. URL: https://proceedings.neurips.cc/paper/2004/hash/e0688d13958a19e087e123148555e4b4-Abstract.html.

[42] The mathlib Community. "The Lean mathematical library". In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, New Orleans, LA, USA, January 20-21, 2020*. Ed. by J. Blanchette and C. Hritcu. ACM, 2020, pp. 367–381.

A Proofs for the preliminaries

We provide proofs and supporting arguments for the statements, definitions, and examples in Sec. 2. All table and function norms use the uniform measures specified there. For a linear map A, its kernel is $\ker(A) = \{c : A \cdot c = 0\}$.

Statistical distance. For probability distributions P and Q on a finite set V, the positive and negative parts of $P-Q$ have equal total mass. Thus $\frac{1}{2}\|P-Q\|_1 = \sum_{v:P(v)>Q(v)}(P(v)-Q(v))$, which is also the largest absolute difference between the acceptance probabilities of a Boolean test. The maximum is attained by the test that outputs one exactly where $P(v) > Q(v)$. Let R be the common random tape, with distribution τ, and let P_r and Q_r be the finite view distributions for each fixed tape r. The statistical distance of the joint tape-and-view distributions is $\mathbb{E}_{R\leftarrow\$\tau}[\delta(P_R, Q_R)]$, also when R is a possibly infinite bit tape. Indeed, the maximizing test outputs one exactly where $P_R(v) > Q_R(v)$.

A.1 Refresh correctness and encoding counts

We check that a refresh preserves the encoded bit and count the encodings of each bit.

Preservation of the encoded secret. Since $(L_{i-1} \cdot M_i) \cdot \left(M_i^{-1} \cdot R_{i-1}\right) = L_{i-1} \cdot R_{i-1}$, every refresh preserves the secret, and invertibility keeps both shares nonzero.

Counting encodings. For every nonzero row ℓ, exactly half of the N binary columns satisfy $\ell \cdot r = 0$ and half satisfy $\ell \cdot r = 1$. Indeed, $r \mapsto \ell \cdot r$ is a surjective linear functional on $\mathbb{F}_2^n$. Removing the zero column leaves $N/2 - 1$ partners for secret zero and $N/2$ for secret one. The same count holds with rows and columns exchanged. Thus every nonzero share has the same number m_s of nonzero partners for a given secret s, and

$$m_0 = (N-2)/2, \qquad m_1 = N/2, \qquad |\Omega_s| = q \cdot m_s, \qquad p_s = m_s/q.$$

In particular, both μ_0 and μ_1 have uniform left and right marginals, and conditioning ν on Ω_s gives μ_s. Since $N \geq 4$, the class probabilities satisfy

$$p_1 \geq p_0 = \frac{N-2}{2(N-1)} \geq \frac{1}{3}.$$

Lem. 2.19 (Freshness; restated)**.** *For every $s \in \mathbb{F}_2$ and every $(\ell, r) \in \Omega_s$, if $M \leftarrow\!\$\ \mathsf{InvMatr}_2^n$, then the refreshed pair $(\ell \cdot M, M^{-1} \cdot r)$ has distribution μ_s.*

Proof of Lem. 2.19. We show that the action $(\ell, r) \mapsto (\ell \cdot M, M^{-1} \cdot r)$ is transitive on Ω_s. For two states $(\ell, r) \in \Omega_s$ and $(\ell', r') \in \Omega_s$, it suffices to find an invertible M with $M \cdot r' = r$ and $\ell \cdot M = \ell'$. If $s \neq 0$, map a basis of $\ker(\ell')$ to a basis of $\ker(\ell)$, and map r' to r. Appending the specified vectors to the kernel bases gives bases of the full column space because $\ell' \cdot r' = \ell \cdot r = s \neq 0$. If $s = 0$, extend r' and r to bases of $\ker(\ell')$ and $\ker(\ell)$, respectively, and append vectors v' and v with $\ell' \cdot v' = \ell \cdot v = 1$. Map the first adapted basis to the second. In both cases the resulting map is invertible, sends r' to r, and makes $\ell \cdot M$ agree with ℓ' on a basis. Since the action is transitive, for a fixed state the sets of matrices that take it to each state of Ω_s have equal cardinality: multiplication by a matrix taking one target state to another gives a bijection between the corresponding sets. Thus a uniform matrix gives a uniform state of Ω_s. □

The same freshness statement holds conditionally on the preceding execution because the next matrix is independent of that execution. Conditioning on a leakage answer can instead change the matrix distribution, so freshness need not hold after that conditioning.

A.2 Linear algebra for tables

Using the orthonormal expansions recalled in Lem. 2.10, we establish the minimum-cost characterization of their coefficient sum. This characterization bounds the nuclear norm of arbitrary product representations, including the products of local approximations in Lem. 4.16 and Claim 5.12, and it gives the minimum-cost representation in the proof of Lem. 2.20.

A.2.1 Product tables

The following elementary identities for tables of the form $a \cdot b^{\mathsf{T}}$ are used throughout.

Lemma A.1 (Inner products and norms of product tables)**.** *For functions a and c from U_{L} to $\mathbb{R}$, and b and e from U_{R} to $\mathbb{R}$,*

$$\left\langle a \cdot b^{\mathsf{T}}, c \cdot e^{\mathsf{T}} \right\rangle = \langle a, c \rangle \cdot \langle b, e \rangle , \tag{A.1}$$

$$\left\| a \cdot b^{\mathsf{T}} \right\|_2 = \|a\|_2 \cdot \|b\|_2 . \tag{A.2}$$

In particular, every unit product table has L_2 norm one.

Proof of Lem. A.1. Independence of ℓ and r gives

$$\mathbb{E}_{\ell,r}[a(\ell) \cdot b(r) \cdot c(\ell) \cdot e(r)] = \mathbb{E}_{\ell}[a(\ell) \cdot c(\ell)] \cdot \mathbb{E}_{r}[b(r) \cdot e(r)].$$

Taking $c = a$ and $e = b$ in Eq. (A.1) and then square roots proves Eq. (A.2). □

A.2.2 Minimum-cost factorizations

We now prove that the nuclear norm is the smallest cost of a product representation, which is the form in which the security proofs use it.

Definition A.2 (Product cost)**.** *For a finite product representation $F = \sum_j a_j \cdot b_j^{\mathsf{T}}$, its* product cost *is $\sum_j \|a_j\|_2 \cdot \|b_j\|_2$.*

Lemma A.3 (Minimum-cost characterization of the nuclear norm)**.** *The nuclear norm of every real table $F : U_{\mathsf{L}} \times U_{\mathsf{R}} \to \mathbb{R}$ is its smallest product cost:*

$$\|F\|_* = \min_{F = \sum_k a_k \cdot b_k^{\mathsf{T}}} \sum_k \|a_k\|_2 \cdot \|b_k\|_2 . \tag{A.3}$$

The minimum ranges over all finite product representations, including the empty representation for the zero table. It is attained by $a_j = \sqrt{\sigma_j} \cdot u_j$ and $b_j = \sqrt{\sigma_j} \cdot v_j$ from an orthonormal expansion of F.

Proof of Lem. A.3. We verify the standard factorization characterization [41, Sec. 3, Lem. 1] in our normalization. Fix an orthonormal expansion of F from Lem. 2.10 and put $W := \sum_j u_j \cdot v_j^{\mathsf{T}}$. Lem. A.1 gives $\langle F, W \rangle = \sum_j \sigma_j = \|F\|_*$. For any left function a, Bessel's inequality (see Eq. (2.2)) gives

$$\sum_j \langle a, u_j \rangle^2 \le \|a\|_2^2 . \tag{A.4}$$

The same inequality holds for the right family. Consequently the Cauchy–Schwarz inequality (see Lem. 2.7) gives, for any product table,

$$\left| \left\langle a \cdot b^{\mathsf{T}}, W \right\rangle \right| = \left| \sum_j \langle a, u_j \rangle \cdot \langle b, v_j \rangle \right| \le \|a\|_2 \cdot \|b\|_2 .$$

Every finite representation therefore satisfies

$$\|F\|_* = \langle F, W \rangle \le \sum_k \|a_k\|_2 \cdot \|b_k\|_2 .$$

The stated factors have $\|a_j\|_2^2 = \|b_j\|_2^2 = \sigma_j$, so they attain the minimum in Eq. (A.3). The empty representation gives the same conclusion when $F = 0$. □

Lem. 2.13 (Basic nuclear-norm bounds; restated)**.** *The function $\|\cdot\|_*$ is a norm on the real tables on $U_{\mathsf{L}} \times U_{\mathsf{R}}$. For every such table F,*

$$\|F\|_2 \le \|F\|_* . \tag{2.4}$$

For every integer $m \ge 0$ and all functions $a_j : U_{\mathsf{L}} \to \mathbb{R}$ and $b_j : U_{\mathsf{R}} \to \mathbb{R}$, indexed by $j = 1, \dots, m$,

$$\left\| \sum_j a_j \cdot b_j^{\mathsf{T}} \right\|_* \le \sum_j \|a_j\|_2 \cdot \|b_j\|_2 . \tag{2.5}$$

For every pair of functions $a : U_{\mathsf{L}} \to \mathbb{R}$ and $b : U_{\mathsf{R}} \to \mathbb{R}$,

$$\left\| a \cdot b^{\mathsf{T}} \right\|_* = \|a\|_2 \cdot \|b\|_2 . \tag{2.6}$$

Proof of Lem. 2.13. The product tables $u_j \cdot v_j^{\mathsf{T}}$ in an orthonormal expansion are themselves orthonormal by Lem. A.1. Pythagoras' theorem (see Eq. (2.1)) therefore gives $\|F\|_2^2 = \sum_j \sigma_j^2$, and hence

$$\|F\|_2 = \left(\sum_j \sigma_j^2 \right)^{1/2} \le \sum_j \sigma_j = \|F\|_* .$$

The inequality uses $\sigma_j \ge 0$ and also holds for the empty expansion of the zero table. Eq. (A.3) gives the arbitrary-representation bound Eq. (2.5) and a representation attaining the minimum. For a single product, Eqs. (A.2) and (A.3) give

$$\|a\|_2 \cdot \|b\|_2 = \left\| a \cdot b^{\mathsf{T}} \right\|_2 \le \left\| a \cdot b^{\mathsf{T}} \right\|_* \le \|a\|_2 \cdot \|b\|_2 ,$$

proving the product-table formula. Concatenating representations attaining the minimum for two tables proves the triangle inequality for $\|\cdot\|_*$. For real $\tau \ne 0$, replacing σ_j and u_j in an orthonormal expansion of F by $|\tau| \cdot \sigma_j$ and $\operatorname{sign}(\tau) \cdot u_j$ gives an orthonormal expansion of τF, so $\|\tau F\|_* = |\tau| \cdot \|F\|_*$. The case $\tau = 0$ is trivial. The lower bound makes the norm positive on nonzero tables. □

A.2.3 Largest singular coefficient

Lem. 2.15 (Largest singular coefficient; restated)**.** *For every real table $F : U_{\mathsf{L}} \times U_{\mathsf{R}} \to \mathbb{R}$ and all functions $u : U_{\mathsf{L}} \to \mathbb{R}$ and $v : U_{\mathsf{R}} \to \mathbb{R}$,*

$$\left| \left\langle F, u \cdot v^{\mathsf{T}} \right\rangle \right| \le \sigma_{\max}(F) \cdot \|u\|_2 \cdot \|v\|_2 . \tag{2.7}$$

For every nonzero F, there exist unit functions u and v attaining equality in Eq. (2.7). For every real table F,

$$\sigma_{\max}(F) \le \|F\|_2 . \tag{2.8}$$

Proof of Lem. 2.15. Write $F = \sum_j \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}$. For $F = 0$ the claims hold immediately. Otherwise, orthonormality and the Cauchy–Schwarz inequality (see Lem. 2.7) give

$$\begin{aligned} \left| \left\langle F, u \cdot v^{\mathsf{T}} \right\rangle \right| &\le \sigma_{\max}(F) \cdot \left(\sum_j \langle u, u_j \rangle^2 \right)^{1/2} \cdot \left(\sum_j \langle v, v_j \rangle^2 \right)^{1/2} \\ &\le \sigma_{\max}(F) \cdot \|u\|_2 \cdot \|v\|_2 . \end{aligned}$$

The last inequality applies Eq. (A.4) to each orthonormal family. Choosing $u = u_j$ and $v = v_j$ at a largest coefficient attains equality. Finally, $\sigma_{\max}(F) \le \left(\sum_j \sigma_j^2 \right)^{1/2} = \|F\|_2$. □

A.2.4 Pointwise energy bounds

For every finite representation $F = \sum_j a_j \cdot b_j^{\mathsf{T}}$, with factor energies Φ_F and Ψ_F from Def. 2.16, the pointwise bound is

$$|F(\ell, r)| \le \sqrt{\Phi_F(\ell) \cdot \Psi_F(r)} \qquad (\ell \in U_{\mathsf{L}},\ r \in U_{\mathsf{R}}). \tag{2.10}$$

Proof of Eq. (2.10). For any chosen finite factorization $F = \sum_j a_j \cdot b_j^{\mathsf{T}}$ and every ℓ and r, the Cauchy–Schwarz inequality (see Lem. 2.7) in the factor index gives

$$|F(\ell, r)| = \left| \sum_j a_j(\ell) \cdot b_j(r) \right| \le \sqrt{\sum_j a_j(\ell)^2} \cdot \sqrt{\sum_j b_j(r)^2} = \sqrt{\Phi_F(\ell) \cdot \Psi_F(r)}.$$

□

A.3 Binary signs and discrepancy

Recall that $N := 2^n$, $q = N - 1$, and $n \ge 2$. The discrepancy estimate follows by counting the signs of a nonzero binary linear form. The local approximation proof uses the same cancellation of binary signs to control the consistency condition (see Sec. 4.3.3).

Lemma A.4 (The binary inner-product sign table)**.** *On nonzero rows and columns, define*

$$\Upsilon(\ell, r) = \begin{cases} 1, & \ell \cdot r = 0, \\ -1, & \ell \cdot r = 1, \end{cases} \qquad (\mathrm{Av}_\Upsilon(v))(\ell) = \mathbb{E}_{r \leftarrow \$ U_{\mathsf{R}}}[\Upsilon(\ell, r) \cdot v(r)].$$

For every real function $v : U_{\mathsf{R}} \to \mathbb{R}$,

$$\|\mathrm{Av}_\Upsilon(v)\|_2^2 = \frac{N}{(N-1)^2} \cdot \|v\|_2^2 - \frac{1}{N-1} \cdot \left(\mathbb{E}_{r \leftarrow \$ U_{\mathsf{R}}}[v(r)]\right)^2. \tag{A.5}$$

Consequently, for all real functions u and v on the respective share spaces,

$$\left| \left\langle \Upsilon, u \cdot v^{\mathsf{T}} \right\rangle \right| \le \frac{\sqrt{N}}{N-1} \cdot \|u\|_2 \cdot \|v\|_2. \tag{A.6}$$

Proof. For distinct nonzero columns r and r', the product $\Upsilon(\ell, r) \cdot \Upsilon(\ell, r')$ is 1 or -1 according to the value of the nonzero linear form $\ell \cdot (r + r')$. Among all N rows its two values occur equally often. Removing the zero row removes one positive sign. Thus its mean over nonzero ℓ is $-1/q$, whereas for $r = r'$ the mean is 1. Expanding the square gives

$$\begin{aligned} \|\mathrm{Av}_\Upsilon(v)\|_2^2 &= \frac{1}{q^2} \sum_r v(r)^2 - \frac{1}{q^3} \sum_{r \ne r'} v(r) \cdot v(r') \\ &= \frac{q+1}{q^2} \cdot \|v\|_2^2 - \frac{1}{q} \cdot \left(\mathbb{E}_{r \leftarrow \$ U_{\mathsf{R}}}[v(r)]\right)^2. \end{aligned}$$

Since $q + 1 = N$, this is Eq. (A.5). Dropping its nonnegative last term and applying the Cauchy–Schwarz inequality (see Lem. 2.7) to $\mathbb{E}_{\ell \leftarrow \$ U_{\mathsf{L}}}[u(\ell) \cdot (\mathrm{Av}_\Upsilon(v))(\ell)]$ proves Eq. (A.6). □

Symbol	Meaning	Sec.
Storage and refreshing (see Secs. 2 to 4)		
n, $N := 2^n$, $q := N - 1$	share length in bits; number of n-bit strings; number of nonzero shares	2.2
U_{L}, U_{R}	nonzero rows and nonzero columns in $\mathbb{F}_2^n$	2.2
Ω_s, μ_s	pairs of shares with inner product s; the uniform distribution on them	2.2
ν, p_s	independent uniform nonzero shares; probability that their inner product is s	2.4
(L, R), (L', R')	old and refreshed shares in a single refresh	4.3.1
T, M_i	number of refreshes; uniform invertible matrix of refresh i	2.2
λ, $K := 2^{2\lambda}$	bits leaked per party per refresh; number of complete answers	3
Z_i, H_i	answer of refresh i; history $(Z_1, \ldots, Z_i)$	3
$f_{\mathsf{L},z}$, $f_{\mathsf{R},z}$	left and right masks of the answer z	3
P_s^T	distribution of the complete transcript for secret s	4.1
δ, $\|P\|_1$	statistical distance; sum of absolute values of a finite signed weight function P	2.1
$\|F\|_2$, $\|F\|_*$	L_2 norm and nuclear norm of a table F	2.3
$\langle F, W \rangle$	pairing of two tables	2.3
Φ_F, Ψ_F	energies of a chosen product representation	2.3
φ, $\mathsf{project}(F)$	density of the refresh experiment; consistency projection of a table F	4.3.1
$X_i = (L_i, R_i)$	encoding after refresh i	2.2
$\mathsf{prob}_{i,h,z}$, $G_{i,h,z}$	leakage probability function; its signed approximation	4.2, 4.3
β, ε	singular-coefficient threshold; per-answer L_2 approximation error	4.3
$\sigma_{\max}(F)$, disc_n	largest singular coefficient; one-encoding discrepancy bound	2.3, 4.5
κ, C	security parameter for target distance $2^{-\kappa}$; absolute constant	4.8, 2.1
Circuit compiler and the approximation lemma with retained local data		
t_0, S	source NAND gates after conversion; all $S = 3t_0 + I_{\mathrm{pub}}$ operations of the compiler	5.1
I_{priv}, I_{pub}, k	private input wires; public or constant input wires; designated outputs	5.1.1
$\mathbf{x}$, $\mathbf{s}$, $\mathbf{y}$	public input; private input; authorized output	5.1.1
λ_{M}, λ_{S}	per-party matrix and shortening budgets; $\lambda = \lambda_{\mathsf{M}} + \lambda_{\mathsf{S}}$	5.2.1
S_i	scope of an operation: the encoding locations it reads or produces	5.1.1
$\overline{G}$, B, D_b	trimmed approximation; trimming threshold; encoding difference $\mu_b - \nu$	5.4
a, b, $\mathcal{A}$, $d = \|\mathcal{A}\|$	injective maps; set of old local data and its size	5.3.1
L°, R°	old local data of the left and the right party	5.3.1
m, $N_m = 2^m$, $q_m = N_m - 1$	long-share length; numbers of all and nonzero long shares	5.2, 5.3.1
U, V; A, R	long refreshed shares; short output shares	5.2
H	shortening matrix with $A \cdot H = U$ and $R = H \cdot V$	5.2

Table 2: Main notation.

A.3.1 Two-secret discrepancy

We express the difference between expectations under μ_0 and μ_1 using the sign table, then apply the Cauchy–Schwarz inequality (see Lem. 2.7). A minimum-cost product factorization (see Eq. (A.3)) extends the result to arbitrary real tables.

Lem. 2.20 (Two-secret discrepancy; restated)**.** *For every real table F,*

$$\left|\mathbb{E}_{(\ell,r)\leftarrow\$\mu_0}[F(\ell,r)] - \mathbb{E}_{(\ell,r)\leftarrow\$\mu_1}[F(\ell,r)]\right| \le \frac{3}{\sqrt{N-2}} \cdot \|F\|_* \,. \tag{2.14}$$

In particular, for real functions $u : U_{\mathsf{L}} \to \mathbb{R}$ and $v : U_{\mathsf{R}} \to \mathbb{R}$,

$$\left|\mathbb{E}_{(\ell,r)\leftarrow\$\mu_0}[u(\ell)\cdot v(r)] - \mathbb{E}_{(\ell,r)\leftarrow\$\mu_1}[u(\ell)\cdot v(r)]\right| \le \frac{3}{\sqrt{N-2}} \cdot \|u\|_2 \cdot \|v\|_2 \,. \tag{2.15}$$

Proof of Lem. 2.20. First consider a product table $u(\ell)\cdot v(r)$. Subtracting the uniform mean from v leaves its discrepancy unchanged, because the two secret distributions have the same left marginal, and does not increase its L_2 norm. We may therefore assume $\mathbb{E}_{r\leftarrow\$U_{\mathsf{R}}}[v(r)] = 0$. For a fixed left share ℓ, let

$$\mathrm{gap}(\ell) = \mathbb{E}_{R\leftarrow\$\{r\in U_{\mathsf{R}}:\ell\cdot r=0\}}[v(R)] - \mathbb{E}_{R\leftarrow\$\{r\in U_{\mathsf{R}}:\ell\cdot r=1\}}[v(R)].$$

The two classes have uniform probabilities p_0 and p_1. Their weighted means sum to zero, while their difference weighted with signs is $(\mathrm{Av}_\Upsilon(v))(\ell)$. Since $p_0 + p_1 = 1$, solving these two equations gives

$$\mathrm{gap}(\ell) = \frac{(\mathrm{Av}_\Upsilon(v))(\ell)}{2\cdot p_0\cdot p_1}. \tag{A.7}$$

Eqs. (A.5) and (A.7) and Eqs. (2.11) and (2.12) now imply

$$\|\mathrm{gap}\|_2 \le \frac{2(N-1)}{(N-2)\sqrt{N}} \cdot \|v\|_2 \le \frac{3}{\sqrt{N-2}} \cdot \|v\|_2 \,. \tag{A.8}$$

For the last inequality, use $N-1 \le N$, $N/(N-2) \le 2$, and $2\sqrt{2} \le 3$. Averaging over the common uniform left marginal and applying the Cauchy–Schwarz inequality (see Lem. 2.7) with Eq. (A.8) gives

$$\begin{aligned}&\left|\mathbb{E}_{(\ell,r)\leftarrow\$\mu_0}[u(\ell)\cdot v(r)] - \mathbb{E}_{(\ell,r)\leftarrow\$\mu_1}[u(\ell)\cdot v(r)]\right| \\ &\quad = \left|\mathbb{E}_{\ell\leftarrow\$U_{\mathsf{L}}}[u(\ell)\cdot \mathrm{gap}(\ell)]\right| \le \frac{3}{\sqrt{N-2}} \cdot \|u\|_2 \cdot \|v\|_2 \,.\end{aligned}$$

Finally, choose a minimum-cost representation $F = \sum_j a_j \cdot b_j^{\mathsf{T}}$ using Eq. (A.3), and apply Eq. (2.15) to each term. Since $\sum_j \|a_j\|_2 \cdot \|b_j\|_2 = \|F\|_*$, summing proves Eq. (2.14). □

B The compiler approximation and shortening

We first extend the refreshing approximation, proved in Sec. 4.3.3, to the retained local data of Sec. 5.3.1. We then specialize that extension to tensor encodings and show that rectangular shortening transports the approximation. Together, these three steps prove Lem. 5.6. Recall that prob denotes a true leakage probability function, G its signed approximation, and Φ and Ψ the factor energies of a chosen product representation (see Def. 2.16). The retained local data are the operand shares that the leakage may read in addition to the shares being refreshed. All constants are absolute, and all unqualified table norms use the specified product of uniform measures.

B.1 Extending the approximation to retained local data

Recall the setup of Sec. 5.3.1: the old local data L° and R° are sampled independently and uniformly from a set $\mathcal{A}$ of size d, and the injective maps a and b take them to nonzero m-bit shares. The grouped coordinates are $x = (L^\circ, U)$ and $y = (R^\circ, V)$, with uniform product reference measure. The consistency projection keeps precisely the tuples with $U \cdot V = a(L^\circ) \cdot b(R^\circ)$. The table prob_z is the conditional probability of one leakage answer z, extended by zero off these tuples.

Lem. 5.4 (Approximation of an answer with retained local data; restated)**.** *There exists an absolute constant $C > 0$ such that the following holds for every formal incoming history, adversary tape, answer z, and threshold $\beta \in [d^{-1/3}, 1]$. Take a probability-normalized orthonormal expansion (see Lem. 2.10) $\mathsf{prob}_z = \sum_j \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}$ and put*

$$I := \{j : \sigma_j > \beta\}, \qquad \mathsf{prob}_z^{>\beta} := \sum_{j \in I} \sigma_j \cdot u_j \cdot v_j^{\mathsf{T}}. \tag{5.11}$$

Then

$$\left\|\mathsf{prob}_z^{>\beta}\right\|_* \le 2, \tag{5.12}$$

$$\left\|\mathsf{prob}_z^{>\beta}\right\|_2 \le 1. \tag{5.13}$$

The balanced factors $\sqrt{\sigma_j} \cdot u_j$ and $\sqrt{\sigma_j} \cdot v_j$ of $\mathsf{prob}_z^{>\beta}$ (see Def. 2.17) have energies Φ_β and Ψ_β satisfying, for every grouped row $x \in \mathcal{X}$ and column $y \in \mathcal{Y}$,

$$\Phi_\beta(x) \le 1/\beta, \tag{5.14}$$

$$\Psi_\beta(y) \le 1/\beta, \tag{5.15}$$

$$\mathbb{E}_{x \leftarrow\$ \mathcal{X}}[\Phi_\beta(x)] = \left\|\mathsf{prob}_z^{>\beta}\right\|_* \le 2, \tag{5.16}$$

$$\mathbb{E}_{y \leftarrow\$ \mathcal{Y}}[\Psi_\beta(y)] = \left\|\mathsf{prob}_z^{>\beta}\right\|_* \le 2. \tag{5.17}$$

Finally,

$$\left\|\mathsf{prob}_z - 2 \cdot \mathsf{project}\left(\mathsf{prob}_z^{>\beta}\right)\right\|_2 \le C \cdot \beta^{1/3}. \tag{5.18}$$

The constant C is independent of the dimension, threshold, answer, formal history, and admissible masks. The construction depends on the masks and public maps, not on any plaintext labels.

Proof of Lem. 5.4. We extend the proof of Lem. 4.6, keeping track of the two sizes $d = |\mathcal{A}|$ and $q_m = 2^m - 1$. Throughout, $1 \le d \le q_m$ and $d \cdot \beta^3 \ge 1$, where $q_m = N_m - 1$ and $N_m = 2^m$. All averages use the uniform measures of Sec. 5.3.1, unless full vector spaces are specified.

Projection bounds. The row and column projection argument in Eqs. (4.18) to (4.20) applies to any bounded table, so it applies to prob_z on $\mathcal{X} \times \mathcal{Y}$. Put $e_u(x) := \sum_{j \in I} u_j(x)^2$ and define e_v

similarly. Every row and column of $\mathsf{prob}_z^{>\beta}$ has L_2 norm at most one, and

$$\begin{aligned} e_u &\le \beta^{-2}, \\ e_v &\le \beta^{-2}, \\ \Phi_\beta &\le \beta^{-1}, \\ \Psi_\beta &\le \beta^{-1}, \\ \mathbb{E}_{x \leftarrow\$ \mathcal{X}}\left[e_u(x)^2\right] &\le |I|/\beta^2, \qquad \text{(B.1)} \\ \mathbb{E}_{y \leftarrow\$ \mathcal{Y}}\left[e_v(y)^2\right] &\le |I|/\beta^2, \qquad \text{(B.2)} \\ \left\|\mathsf{prob}_z^{>\beta}\right\|_2 &\le 1. \qquad \text{(B.3)} \end{aligned}$$

These statements also hold for an empty retained family.

Gram variance on an injectively encoded set. For a function $g_{L^\circ}(U)$ valued in a finite-dimensional real inner-product space, put $\overline{g}_{L^\circ} := \mathbb{E}_U[g_{L^\circ}(U)]$, $\widehat{g}_{L^\circ}(U) := g_{L^\circ}(U) - \overline{g}_{L^\circ}$, and $Y_M := d^{-1}\sum_{L^\circ} g_{L^\circ}(a(L^\circ)\cdot M)$. Here $L^\circ \in \mathcal{A}$ and $U \in U_{\mathsf{L}}^{(m)}$. For $L^\circ \neq L^{\circ\prime}$, the distinct nonzero vectors $a(L^\circ)$ and $a(L^{\circ\prime})$ are linearly independent over $\mathbb{F}_2$. Their images under uniform invertible M therefore form a uniform ordered pair of distinct nonzero rows. Since each $\widehat{g}_{L^\circ}$ has mean zero, deleting the diagonal from the sum over all pairs gives

$$\mathbb{E}_M\left[\left\langle \widehat{g}_{L^\circ}(a(L^\circ)\cdot M), \widehat{g}_{L^{\circ\prime}}(a(L^{\circ\prime})\cdot M)\right\rangle\right] = -\frac{1}{q_m - 1}\mathbb{E}_U[\langle \widehat{g}_{L^\circ}(U), \widehat{g}_{L^{\circ\prime}}(U)\rangle].$$

Expanding the squared norm of the average and combining the diagonal and off-diagonal terms yields the counterpart of Lem. 4.7:

$$\begin{aligned} \mathbb{E}_M\left[\|Y_M - \mathbb{E}_{M'}[Y_{M'}]\|^2\right] &= \frac{q_m}{d^2\cdot(q_m-1)}\sum_{L^\circ}\mathbb{E}_U\left[\|\widehat{g}_{L^\circ}(U)\|^2\right] \\ &\quad - \frac{1}{d^2\cdot(q_m-1)}\mathbb{E}_U\left[\left\|\sum_{L^\circ}\widehat{g}_{L^\circ}(U)\right\|^2\right] \qquad \text{(B.4)} \\ &\le \frac{q_m}{d\cdot(q_m-1)}\mathbb{E}_{L^\circ,U}\left[\|g_{L^\circ}(U)\|^2\right]. \end{aligned}$$

For the inequality, discard the nonpositive term and use the fact that centering does not increase the second moment. The column version follows from injectivity of b and uniformity of M^{-1}.

Nuclear norm. If I is nonempty, set $\tau := 2\left\|\mathsf{prob}_z^{>\beta}\right\|_*$ and $W := \sum_{j\in I} u_j \cdot v_j^{\mathsf{T}}$. Then $\langle \mathsf{prob}_z, W\rangle = \tau/2$, $|I|\cdot\beta \le \tau/2$, and $\|W\|_\infty \le \beta^{-2}$. Define the Gram matrices of the substituted singular functions by

$$\mathrm{Gram}_{\mathsf{L},M} := \mathbb{E}_{L^\circ}\left[u(L^\circ, a(L^\circ)\cdot M)\cdot u(L^\circ, a(L^\circ)\cdot M)^{\mathsf{T}}\right],$$

where $u = (u_j)_{j\in I}$ is a coefficient column, and define $\mathrm{Gram}_{\mathsf{R},M}$ with $v(R^\circ, M^{-1}\cdot b(R^\circ))$. As in Lem. 4.8, their means are Id. Apply Eq. (B.4) to coefficient matrices with the ordinary square-sum norm $\|\cdot\|_{\mathrm{E}}$, the square root of the sum of the squared entries, without probability normalization. Since $\left\|u\cdot u^{\mathsf{T}}\right\|_{\mathrm{E}}^2 = e_u^2$, combining this with Eqs. (B.1) and (B.2) gives, on each side,

$$\mathbb{E}_M\left[\|\mathrm{Gram}_{\mathsf{L},M} - \mathrm{Id}\|_{\mathrm{E}}^2\right], \quad \mathbb{E}_M\left[\|\mathrm{Gram}_{\mathsf{R},M} - \mathrm{Id}\|_{\mathrm{E}}^2\right] \le \frac{q_m\cdot|I|}{d\cdot(q_m-1)\cdot\beta^2}.$$

The Cauchy–Schwarz inequality (see Lem. 2.7) applied to the two mask coefficient vectors, as in Lem. 4.8, consequently gives

$$\mathbb{E}_{L^\circ,R^\circ,M}\Big[f_{\mathsf{L},z}(L^\circ,M)\cdot f_{\mathsf{R},z}(R^\circ,M)\cdot W^M(L^\circ,R^\circ)\Big]\le 1+\sqrt{\frac{q_m\cdot|I|}{d\cdot(q_m-1)\cdot\beta^2}}.$$

Using Eqs. (5.9) and (5.10), we obtain

$$\tau\le 1+\sqrt{\frac{q_m}{2d\cdot(q_m-1)\cdot\beta^3}}\cdot\sqrt{\tau}+\frac{1}{q_m\cdot\beta^2}\le 2+\sqrt{\tau}.$$

The last inequality uses $d\cdot\beta^3\ge 1$, $d\le q_m$, $q_m\ge 3$, and $\beta\le 1$. Thus $\tau\le 4$. This also holds when I is empty. The balanced factors of $2\cdot\mathsf{prob}_z^{>\beta}$ have mean energies τ and pointwise energies at most $2/\beta$, while $\left\|2\cdot\mathsf{prob}_z^{>\beta}\right\|_2\le 2$ follows from Eq. (B.3).

Cancellation of the consistency sign. As in Lem. 4.9, the full binary sign matrix on pairs of m-bit vectors satisfies $\Upsilon^{\mathrm{full}}\cdot(\Upsilon^{\mathrm{full}})^{\mathsf{T}}=N_m^2\cdot\mathrm{Id}$. The table $2J$ is its submatrix indexed by $(a(L^\circ),U)$ and $(b(R^\circ),V)$. Injectivity makes these genuine subsets, so extending functions by zero and applying the Cauchy–Schwarz inequality (see Lem. 2.7) bounds the unnormalized pairing by N_m times the two Euclidean norms. Each grouped domain now has dq_m elements. After converting to probability normalization, for every $f:\mathcal{X}\to\mathbb{R}$ and $g:\mathcal{Y}\to\mathbb{R}$ we get

$$\left|\left\langle J,f\cdot g^{\mathsf{T}}\right\rangle\right|\le\frac{N_m}{2d\cdot q_m}\cdot\|f\|_2\cdot\|g\|_2\le\frac{1}{d}\cdot\|f\|_2\cdot\|g\|_2\,. \tag{B.5}$$

Use the balanced representation of $\mathsf{prob}_z^{>\beta}$, with energies Φ_β and Ψ_β. Apply Eq. (B.5) to each product with the test factors inserted, and use the Cauchy–Schwarz inequality (see Lem. 2.7) over the products. For unit L_2 functions f and g the result is $\left|\left\langle J\cdot\mathsf{prob}_z^{>\beta},f\cdot g^{\mathsf{T}}\right\rangle\right|\le 1/(d\cdot\beta)$. Using Eq. (2.7) for the largest coefficient and expanding the residual as in Eq. (4.39), we obtain, for $\Delta_z:=\mathsf{prob}_z-2\cdot\mathsf{project}\Big(\mathsf{prob}_z^{>\beta}\Big)$ and $t:=\|\Delta_z\|_2$,

$$\sigma_{\max}(\Delta_z)\le\beta+2/(d\cdot\beta)\le 3\beta, \tag{B.6}$$

$$\|\Delta_z\|_\infty\le 3/\beta,$$

$$t^2=\langle\mathsf{prob}_z,\Delta_z\rangle+4\cdot\left\langle\mathsf{prob}_z^{>\beta},J\cdot\mathsf{prob}_z^{>\beta}\right\rangle, \tag{B.7}$$

$$\left|\left\langle\mathsf{prob}_z^{>\beta},J\cdot\mathsf{prob}_z^{>\beta}\right\rangle\right|\le 2/(d\cdot\beta). \tag{B.8}$$

Every row and column of Δ_z has L_2 norm at most three, and Δ_z is supported on consistent tuples.

Four cycles and collisions. We give the version of Lem. 4.12 needed for this smaller old-input domain. For any consistent table Δ, put $t:=\|\Delta\|_2$, $B_\infty:=\|\Delta\|_\infty$, and let B_2 bound all row and column L_2 norms. Extend Δ by zero whenever U or V is zero, obtaining $\widehat{\Delta}$. The full-vector quantities satisfy $\widehat{t}^2=(q_m/N_m)^2\cdot t^2$ and $\mathrm{Cyc}\Big(\widehat{\Delta}\Big)=(q_m/N_m)^4\cdot\mathrm{Cyc}(\Delta)$. The row and column norms remain at most B_2. For a uniform unrestricted binary $m\times m$ matrix A_0, put

$$\widetilde{\Delta}_{A_0}(L^\circ,(R^\circ,V)):=\begin{cases}\sqrt{N_m}\cdot\widehat{\Delta}((L^\circ,a(L^\circ)\cdot A_0),(R^\circ,V)), & A_0\cdot V=b(R^\circ),\\ 0, & \text{otherwise.}\end{cases}$$

The row measure is uniform on $\mathcal{A}$ and the column measure is uniform on $\mathcal{A}\times\mathbb{F}_2^{m\times 1}$. As in the refreshing proof, $\mathrm{Cyc}\Big(\widetilde{\Delta}_M\Big)=\mathrm{Cyc}\Big(\Delta^M\Big)$ for invertible M. Comparison with unrestricted matrices

(see Eq. (4.45)) applies to this nonnegative complete four-cycle and gives

$$\mathbb{E}_M\Big[\mathrm{Cyc}\Big(\Delta^M\Big)\Big] \le 4\mathbb{E}_{A_0}\Big[\mathrm{Cyc}\Big(\widetilde{\Delta}_{A_0}\Big)\Big].$$

Expand the latter average and insert independent full rows U_1 and U_2. The prescribed-row-and-column count (see Lem. 4.11, in dimension m) applies to $a(L^\circ_i)\cdot A_0 = U_i$ and $A_0 \cdot V_j = b(R^\circ_j)$. The pairings required by that lemma are imposed by the support of $\widehat{\Delta}$. Injectivity of a makes row rank one equivalent to $L^\circ_1 = L^\circ_2$, which forces $U_1 = U_2$. Column rank one means $V_1 = V_2$, which forces $R^\circ_1 = R^\circ_2$ by injectivity of b. Thus the four rank contributions have exactly the form of Eq. (4.53), with the following coefficients:

(row rank, column rank)	$(2,2)$	$(1,2)$	$(2,1)$	$(1,1)$
coefficient	16	$4/d$	$4/d$	$2/d^2$.

For example, rank $(1,2)$ has weight $4N_m$ before the equality probabilities are included. $L^\circ_1 = L^\circ_2$ costs $1/d$ and $U_1 = U_2$ costs $1/N_m$. Rank $(2,1)$ has the same cost with V and R° exchanged. Rank $(1,1)$ has weight $2N_m^2$ and equality probability $1/(d^2N_m^2)$.

For completeness, the rank $(2,2)$ term is the full four-cycle restricted to $L^\circ_1 \neq L^\circ_2$ and $V_1 \neq V_2$, multiplied by 16. Inclusion–exclusion subtracts two nonnegative complete four-cycles of sliced tables. The intersection adds at most $16B_\infty^2\widehat{t}^2/(d\cdot N_m)$: its equality probability is $1/(d\cdot N_m)$, each entry retains its full product marginal, and $|e_1e_2e_3e_4| \le (B_\infty^2/4)\sum_i e_i^2$. Hence this term is at most $16\mathrm{Cyc}\Big(\widehat{\Delta}\Big) + 16B_\infty^2\widehat{t}^2/(d\cdot N_m)$. Each mixed-rank term is a nonnegative product of two squared entries. Dropping its remaining inequality restriction bounds it by $4B_2^2\widehat{t}^2/d$. The rank $(1,1)$ term is at most $2B_\infty^2\widehat{t}^2/d^2$. Multiplying by four, using $N_m > d$ and $q_m/N_m \le 1$, gives

$$\mathbb{E}_M\Big[\mathrm{Cyc}\Big(\Delta^M\Big)\Big] \le 64\mathrm{Cyc}(\Delta) + \frac{32B_2^2}{d}t^2 + \frac{72B_\infty^2}{d^2}t^2. \tag{B.9}$$

Completing the error bound. We apply Eq. (B.9) to our residual Δ_z, with $t := \|\Delta_z\|_2$, $B_2 = 3$, and $B_\infty \le 3/\beta$. By Eq. (4.42) and Eq. (B.6), $\mathrm{Cyc}(\Delta_z) \le \sigma_{\max}(\Delta_z)^2\cdot t^2 \le 9\beta^2\cdot t^2$. Since $d\cdot\beta^3 \ge 1$ and $\beta \le 1$, both d^{-1} and $d^{-2}\cdot\beta^{-2}$ are at most β^2. Eq. (B.9) therefore yields $\mathbb{E}_M\Big[\mathrm{Cyc}\Big(\Delta_z^M\Big)\Big] \le C\cdot\beta^2\cdot t^2$. Applying the test bound Eq. (4.43), followed by Jensen's inequality over M, gives

$$\Big|\mathbb{E}_{L^\circ,R^\circ,M}\Big[f_{\mathsf{L},z}(L^\circ,M)\cdot f_{\mathsf{R},z}(R^\circ,M)\cdot\Delta_z^M(L^\circ,R^\circ)\Big]\Big| \le C\cdot\beta^{1/2}\cdot t^{1/2}. \tag{B.10}$$

Replacing $p_s^{(m)}$ by $1/2$ in Eq. (5.9) incurs error at most $\|\Delta_z\|_\infty/(2q_m) \le 3/(2d\cdot\beta)$. Combining Eqs. (B.7), (B.8) and (B.10) gives Eq. (5.19):

$$t^2 \le C\cdot\beta^{1/2}\cdot t^{1/2} + C/(d\cdot\beta) \le C\cdot\beta^{1/2}\cdot t^{1/2} + C\cdot\beta^2. \tag{5.19}$$

If the second term on the right of Eq. (5.19) is at least $t^2/2$, then $t \le C\cdot\beta \le C\cdot\beta^{1/3}$. Otherwise, the first is at least $t^2/2$, and division by $t^{1/2}$ gives $t \le C\cdot\beta^{1/3}$. The case $t = 0$ is immediate. This includes $d = 1$, $\beta = 1$, and an empty retained family. □

B.2 Tensor encodings for the matrix phase of a circuit gate

The approximation lemma of Sec. 5.3.2 (see Lem. 5.4) also covers the matrix phase of a computation gadget. All estimates below are per answer. Their later use enumerates the relevant answers explicitly.

The matrix phase of a circuit gate. Fix a NAND computation gadget with two distinct operand encoding locations. Take $m = n^2 + 1$, $d = q^2$, and $\mathcal{A} = (\{0,1\}^n \setminus \{0^n\})^2$, with

$$L^\circ = (L_1, L_2), \quad R^\circ = (R_1, R_2), \qquad a(L^\circ) = (L_1 \otimes L_2, 1), \quad b(R^\circ) = \begin{pmatrix} R_1 \otimes R_2 \\ 1 \end{pmatrix}.$$

The tensor product lists all pairwise coordinate products, so $a(L^\circ) \cdot b(R^\circ) = 1 + (L_1 \cdot R_1) \cdot (L_2 \cdot R_2)$. Over $\mathbb{F}_2$, a nonzero rank-one tensor uniquely determines its two nonzero factors: any nonzero row or column determines one factor, and its entries determine the other. Appending a fixed coordinate preserves injectivity. For operand bits s_1 and s_2, put

$$\mathcal{D}^{\mathrm{long}}_{s_1,s_2} := \mu^{(n)}_{s_1} \otimes \mu^{(n)}_{s_2} \otimes \mu^{(m)}_{1+s_1 \cdot s_2}.$$

Corollary B.1 (Tensor instance)**.** *For every answer z and every formal incoming history, and for $q^{-2/3} \le \beta \le 1$, the construction of Lem. 5.4 gives a common table $2 \cdot \mathsf{prob}^{>\beta}_z$ with nuclear norm at most 4 and balanced energies $2 \cdot \Phi_\beta$ and $2 \cdot \Psi_\beta$, bounded pointwise by $2/\beta$ and in mean by 4. For every $(s_1, s_2) \in \mathbb{F}_2^2$,*

$$\left(\mathbb{E}_{\omega \leftarrow\$ \mathcal{D}^{\mathrm{long}}_{s_1,s_2}} \left[\left| \mathsf{prob}_z(\omega) - 2 \cdot \mathsf{prob}^{>\beta}_z(\omega) \right|^2 \right] \right)^{1/2} \le C \cdot \beta^{1/3}. \tag{B.11}$$

All constants are absolute and uniform in the masks and formal history.

Proof. Each fixed-bit encoding distribution has density at most three relative to independent uniform nonzero shares by Eq. (2.13), so the three-encoding distribution $\mathcal{D}^{\mathrm{long}}_{s_1,s_2}$ has density at most 27. On a valid local labeling, $\mathsf{prob}_z - 2 \cdot \mathsf{prob}^{>\beta}_z = \mathsf{prob}_z - 2 \cdot \mathsf{project}\left(\mathsf{prob}^{>\beta}_z\right)$. Apply Eq. (5.18) and take square roots of this density bound. The admissible threshold range is $d^{-1/3} \le \beta \le 1$ with $d = q^2$. □

The compiler shortens the long output back to length n. App. B.3 transports both the chosen factor energies and the L_2 error in Eq. (B.11).

B.3 Shortening by a random rectangular matrix

The shortener of Sec. 5.2 applies to any nonzero shares $U \in \mathbb{F}_2^{1\times m}$ and $V \in \mathbb{F}_2^{m\times 1}$, with $m \ge n \ge 2$. It samples a nonzero row A uniformly, samples a full-row-rank matrix $H \in \mathbb{F}_2^{n\times m}$ uniformly subject to $A \cdot H = U$, and sets $R := H \cdot V$. An attempt succeeds when $R \ne 0$. Otherwise, the parties repeat with fresh A and H and the same U and V. All attempts belong to one shortening phase and share its leakage budget.

For the circuit gadget, put $L^\circ := (L_1, L_2)$, $R^\circ := (R_1, R_2)$, and $m := n^2 + 1$. The NAND gadget forms the long shares

$$a(L^\circ) = (L_1 \otimes L_2, 1), \qquad b(R^\circ) = \begin{pmatrix} R_1 \otimes R_2 \\ 1 \end{pmatrix}.$$

Thus the long and short encodings both represent $s = 1 + (L_1 \cdot R_1) \cdot (L_2 \cdot R_2)$. This output bit can be either zero or one. Fixing the NAND gadget's final coordinate does not restrict the bit represented by the shortener. Recall that $N = 2^n$, and let $\mathcal{H}_{n,m}$ be the finite set of full-row-rank $n \times m$ binary matrices. The matrix from the preceding refresh has been erased before this phase.

The sampler in Sec. 5.5 produces the required uniform matrix H in expected polynomial time. The local sampling tape may be included in left leakage. Its conditional distribution will be averaged into the left mask below.

Lemma B.2 (Exact output distribution of shortening)**.** *Let $m \geq n \geq 2$, let $s \in \mathbb{F}_2$, and fix any nonzero long shares $U \in \mathbb{F}_2^{1\times m}$ and $V \in \mathbb{F}_2^{m\times 1}$ with $U \cdot V = s$. Let p_s^{succ} be the probability that one shortening attempt succeeds. Then*

$$1 - p_s^{\mathrm{succ}} = \begin{cases} (2^{m-n} - 1) \,/\, (2^{m-1} - 1)\,, & s = 0, \\ 0, & s = 1. \end{cases} \tag{B.12}$$

In particular, $p_s^{\mathrm{succ}} \geq 1/2$ and $1 - p_s^{\mathrm{succ}} \leq 2/N$. Conditional on success, the output has distribution $\mu_s^{(n)}$, independently of the particular fixed pair of shares U and V. The procedure that retries until success terminates almost surely, takes at most two attempts in expectation, and has exactly the same output distribution $\mu_s^{(n)}$.
If instead $(U, V) \leftarrow\!\$\ \mu_s^{(m)}$, then, conditional on first-attempt success and any fixed output $(A, R) \in \Omega_s$, the preceding variables have the following distribution:

$$H \leftarrow\!\$\ \mathcal{H}_{n,m}, \qquad V \leftarrow\!\$\ \{v : H \cdot v = R\}, \qquad U := A \cdot H. \tag{B.13}$$

The fiber in the middle has exactly 2^{m-n} elements, all nonzero. These statements concern encoding distributions before conditioning on leakage answers.

Proof. For fixed nonzero A, the number of matrices $H \in \mathcal{H}_{n,m}$ with $A \cdot H = U$ is the same for every nonzero U. Indeed, right multiplication by an invertible $m \times m$ matrix maps one such set bijectively to any other. This number is also independent of A, by invertible changes of the n rows. Consequently every admissible pair (A, H) in one attempt has the same probability.

The kernel of H is a uniformly random $(m-n)$-dimensional subspace of $\ker(U)$. To see this, the row space of H is a uniformly random n-dimensional subspace containing U: every such subspace has the same number of ordered bases, and each basis gives exactly one nonzero coordinate row A for U. Taking annihilators gives the assertion about $\ker(H)$. If $s = 1$, the vector V is outside $\ker(U)$, so it cannot lie in $\ker(H)$. If $s = 0$, symmetry among the $2^{m-1} - 1$ nonzero vectors of $\ker(U)$ gives

$$\mathbb{P}[H \cdot V = 0] = \frac{2^{m-n} - 1}{2^{m-1} - 1}.$$

This proves Eq. (B.12) and its bounds.

For exact output uniformity, an invertible $n \times n$ matrix B sends an admissible attempt (A, H) to $(A \cdot B, B^{-1} \cdot H)$, preserving its probability and changing its output to $(A \cdot B, B^{-1} \cdot R)$. The action on nonzero pairs of inner product s is transitive by the argument of Lem. 2.19. All successful outputs therefore have equal probability. Each retry uses an independent attempt with the same positive success probability, proving almost-sure termination, the expected-attempt bound, and exact freshness of the final output.

Finally, suppose the long pair is uniform. Fix a valid short output (A, R). For every $H \in \mathcal{H}_{n,m}$ there exist exactly 2^{m-n} choices of V with $H \cdot V = R$. Each is nonzero because $R \neq 0$. Putting $U = A \cdot H$ gives a nonzero share and $U \cdot V = A \cdot R = s$. Every such choice has the same forward probability: the long pair is uniform, A is uniform, and the number of eligible matrices for A and U is constant. These choices are precisely all the successful preimages of (A, R). Counting them proves Eq. (B.13). □

Conditional leakage probability functions. Fix a gate and an incoming history. For a matrix-phase answer ζ, let $\mathsf{prob}_\zeta\left((L^\circ,U),(R^\circ,V)\right)$ be its conditional probability given the old operands and the refreshed long pair. On valid long tuples, $\mathsf{prob}_\zeta \geq 0$ and $\sum_\zeta \mathsf{prob}_\zeta = 1$. For fixed ζ, the two shortening leakage functions are chosen before either shortening answer is revealed. Write that answer as $z = (z_\mathsf{L}, z_\mathsf{R})$. On a first-attempt success, their masks have the form

$$f_{\mathsf{L},\zeta z_\mathsf{L}}(L^\circ, A, H), \qquad f_{\mathsf{R},\zeta z_\mathsf{R}}(R^\circ, V, H).$$

Here the left mask averages over the local sampling tape conditional on A, H, and $U = A \cdot H$. Hence it takes values in $[0,1]$. The sampler's finite rejection records have countable support, and the bounded mask makes this conditional average an absolutely convergent weighted sum. The right mask is Boolean, or likewise averaged if the implementation uses additional independent right randomness. The successful acknowledgment is fixed on this branch. The two masks depend on separate local variables once H is fixed. Erasure is essential: the shortening views retain the old operands and long shares, but not the preceding matrix or its sampling tape. Thus the conditional averaging of the new sampling tape does not also depend on the other party's inputs or on hidden matrix-phase state.

Definition B.3 (Shortening operator)**.** *For every real table $G : (\mathcal{A}\times U_\mathsf{L}^{(m)})\times(\mathcal{A}\times U_\mathsf{R}^{(m)}) \to \mathbb{R}$, define a table $\mathcal{S}_{\zeta,z}(G)$ on $(\mathcal{A}\times U_\mathsf{L})\times(\mathcal{A}\times U_\mathsf{R})$ as follows. In the expectation below, sample $H \leftarrow\!\!\$\ \mathcal{H}_{n,m}$ and, conditional on H, sample V uniformly from $\{v : H\cdot v = R\}$:*

$$(\mathcal{S}_{\zeta,z}(G))((L^\circ,A),(R^\circ,R)) = \mathbb{E}_{H,V}\begin{bmatrix} G((L^\circ, A\cdot H),(R^\circ,V)) \\ \cdot\, f_{\mathsf{L},\zeta z_\mathsf{L}}(L^\circ,A,H)\cdot f_{\mathsf{R},\zeta z_\mathsf{R}}(R^\circ,V,H)\end{bmatrix}.$$

For nonzero A and R, every variable in this formula belongs to its prescribed ambient share space, whether or not the operand and output plaintexts satisfy the gate equation.

On valid local triples, the leakage probability function conditioned on success of the first attempt is

$$\widehat{\mathsf{prob}}_{\zeta,z} = \mathcal{S}_{\zeta,z}\left(\mathsf{prob}_\zeta\right). \tag{B.14}$$

This follows directly from the reverse description in Eq. (B.13). No density or conditioning factor is needed.

The operator $\mathcal{S}_{\zeta,z}$ is an average over H of products: for fixed H, the left mask depends on the left data and the right mask on the right data. It therefore carries a product representation of G to a product representation of $\mathcal{S}_{\zeta,z}(G)$, and Jensen's inequality keeps the energies and the L_2 error under control.

Lemma B.4 (Transport through shortening)**.** *Fix any gate, formal incoming history, and adversary tape. For every operand plaintext pair $(s_1,s_2)\in\mathbb{F}_2^2$, put $s := 1 + s_1\cdot s_2$ and*

$$\mathcal{D}^{\mathrm{long}}_{s_1,s_2} := \mu_{s_1}^{(n)}\otimes\mu_{s_2}^{(n)}\otimes\mu_s^{(m)}, \qquad \mathcal{D}^{\mathrm{short}}_{s_1,s_2} := \mu_{s_1}^{(n)}\otimes\mu_{s_2}^{(n)}\otimes\mu_s^{(n)}.$$

Fix a matrix answer ζ, a shortening answer z, a threshold $\beta\in[q^{-2/3},1]$, where $q = 2^n-1$, and a real number $\varepsilon\geq 0$. Suppose the real table $G_\zeta = \sum_t a_t\cdot b_t^\mathsf{T}$ has chosen factor energies Φ and Ψ such that, for every $x\in\mathcal{X}$ and $y\in\mathcal{Y}$,

$$\Phi(x) \leq 2/\beta, \tag{B.15}$$

$$\Psi(y) \leq 2/\beta, \tag{B.16}$$

$$\mathbb{E}_{x\leftarrow\!\!\$\mathcal{X}}[\Phi(x)] \leq 4, \tag{B.17}$$

$$\mathbb{E}_{y\leftarrow\!\!\$\mathcal{Y}}[\Psi(y)] \leq 4, \tag{B.18}$$

$$\left(\mathbb{E}_{\omega\leftarrow\!\!\$\mathcal{D}^{\mathrm{long}}_{s_1,s_2}}\left[\left|\mathsf{prob}_\zeta(\omega) - G_\zeta(\omega)\right|^2\right]\right)^{1/2} \leq \varepsilon \quad \textit{for every } (s_1,s_2)\in\mathbb{F}_2^2. \tag{B.19}$$

Then $\widetilde{G}_{\zeta,z} := \mathcal{S}_{\zeta,z}(G_\zeta)$ has a product representation whose energies satisfy, for every $(L^\circ, A) \in \mathcal{A} \times U_{\mathsf{L}}$ and $(R^\circ, R) \in \mathcal{A} \times U_{\mathsf{R}}$,

$$\Phi_{\mathrm{out}}(L^\circ, A) \le 2/\beta, \tag{B.20}$$

$$\Psi_{\mathrm{out}}(R^\circ, R) \le 2/\beta, \tag{B.21}$$

$$\mathbb{E}_{(L^\circ,A)\leftarrow\$\mathcal{A}\times U_{\mathsf{L}}}[\Phi_{\mathrm{out}}(L^\circ, A)] \le 4, \tag{B.22}$$

$$\mathbb{E}_{(R^\circ,R)\leftarrow\$\mathcal{A}\times U_{\mathsf{R}}}[\Psi_{\mathrm{out}}(R^\circ, R)] \le 4, \tag{B.23}$$

$$\left\|\widetilde{G}_{\zeta,z}\right\|_* \le 4. \tag{B.24}$$

Let $\mathsf{prob}^{\mathrm{out}}_{\zeta,z}$ be the actual complete-gadget leakage probability function of the answer (ζ, z), including all retries, conditional on the operand encodings and a short output encoding with the prescribed output bit. Then, for every operand plaintext pair $(s_1, s_2) \in \{0,1\}^2$,

$$\left(\mathbb{E}_{\omega\leftarrow\$\mathcal{D}^{\mathrm{short}}_{s_1,s_2}}\left[\left|\mathsf{prob}^{\mathrm{out}}_{\zeta,z}(\omega) - \widetilde{G}_{\zeta,z}(\omega)\right|^2\right]\right)^{1/2} \le \varepsilon + 4/2^n. \tag{B.25}$$

Under any assignment of the three wire labels, including labels violating the gate equation, the L_2 norm of $\widetilde{G}_{\zeta,z}$ is at most $C_0 := 4 \cdot \sqrt{27}$. The transported representation need not be balanced. Its two mean energy bounds are separate inequalities.

Proof. For fixed $H \in \mathcal{H}_{n,m}$, define

$$a^H_t(L^\circ, A) := a_t(L^\circ, A \cdot H) \cdot f_{\mathsf{L},\zeta z_{\mathsf{L}}}(L^\circ, A, H),$$

$$b^H_t(R^\circ, R) := \mathbb{E}_{V\leftarrow\$\{v:H\cdot v=R\}}[b_t(R^\circ, V) \cdot f_{\mathsf{R},\zeta z_{\mathsf{R}}}(R^\circ, V, H)].$$

Use the factors $|\mathcal{H}_{n,m}|^{-1/2} \cdot a^H_t$ and $|\mathcal{H}_{n,m}|^{-1/2} \cdot b^H_t$, indexed by (H, t). The sum of their products is exactly $\widetilde{G}_{\zeta,z}$. Since the masks lie in $[0,1]$, Jensen's inequality on the right fibers and Eqs. (B.15) and (B.16) give

$$\Phi_{\mathrm{out}}(L^\circ, A) \le \mathbb{E}_{H\leftarrow\$\mathcal{H}_{n,m}}[\Phi(L^\circ, A \cdot H)] \le 2/\beta, \tag{B.26}$$

$$\Psi_{\mathrm{out}}(R^\circ, R) \le \mathbb{E}_{H\leftarrow\$\mathcal{H}_{n,m}}\left[\mathbb{E}_{V\leftarrow\$\{v:H\cdot v=R\}}[\Psi(R^\circ, V)]\right] \le 2/\beta. \tag{B.27}$$

Average Eq. (B.26) over independent uniform L° and A. For each fixed nonzero A, right multiplication by invertible $m \times m$ matrices preserves the uniform distribution on $\mathcal{H}_{n,m}$ and acts transitively on nonzero long rows, so $A \cdot H$ is uniform among them and Eq. (B.17) gives Eq. (B.22). For Eq. (B.27), sample uniform nonzero R and then a uniform member V of its fiber. Averaged over uniform full-row-rank H, the distribution of this nonzero V is invariant under all invertible changes of the m coordinates and therefore uniform. The old data R° remain independent, so Eq. (B.18) gives Eq. (B.23). These averaging facts preserve the original energies before masks are imposed. Masks can only reduce the bounds. The Cauchy–Schwarz inequality (see Lem. 2.7) over (H, t) and the product-representation bound Eq. (2.5) now yield

$$\left\|\widetilde{G}_{\zeta,z}\right\|_* \le \sqrt{\mathbb{E}_{(L^\circ,A)}[\Phi_{\mathrm{out}}(L^\circ, A)] \cdot \mathbb{E}_{(R^\circ,R)}[\Psi_{\mathrm{out}}(R^\circ, R)]} \le 4,$$

which is Eq. (B.24). By Eq. (2.4), the uniform L_2 norm is at most the nuclear norm. Every fixed three-wire labeling has density at most 27 relative to independent uniform shares, proving the arbitrary-label bound C_0.

For the error, put $e_\zeta := \mathsf{prob}_\zeta - G_\zeta$ on the full long table. The operator $\mathcal{S}_{\zeta,z}$ is a positive average with total mask weight at most one, so the weighted Cauchy–Schwarz inequality (see Lem. 2.7) gives

$$|\mathcal{S}_{\zeta,z}(e_\zeta)|^2 \le \mathcal{S}_{\zeta,z}(|e_\zeta|^2).$$

Average over $\omega \leftarrow\$ \mathcal{D}^{\mathrm{short}}_{s_1,s_2}$ and use the reverse description in Eq. (B.13). Removing the masks only increases the nonnegative summand and leaves the long distribution $\mathcal{D}^{\mathrm{long}}_{s_1,s_2}$. Thus, using Eqs. (B.14) and (B.19),

$$\mathbb{E}_{\omega\leftarrow\$\mathcal{D}^{\mathrm{short}}_{s_1,s_2}}\left[\left|\widehat{\mathsf{prob}}_{\zeta,z}(\omega) - \widetilde{G}_{\zeta,z}(\omega)\right|^2\right] \le \mathbb{E}_{\omega\leftarrow\$\mathcal{D}^{\mathrm{long}}_{s_1,s_2}}\left[\left|\mathsf{prob}_\zeta(\omega) - G_\zeta(\omega)\right|^2\right] \le \varepsilon^2. \tag{B.28}$$

Conditioning on first-attempt success preserves this bound: this event has the same probability p^{succ}_s for every long pair encoding s, so it leaves the old operands, long pair, and matrix answer distributed as before.

Finally, retain the actual retries, whose leakage may inspect all attempts within the same phase budget. For fixed old operands, the final short encoding is uniform $\mu^{(n)}_s$ both on first-attempt success and on failure followed by fresh attempts. Consequently, conditional on any valid old and short encodings, the complete answer distribution has the pointwise mixture

$$\mathsf{prob}^{\mathrm{out}} = p^{\mathrm{succ}}_s \cdot \widehat{\mathsf{prob}} + (1 - p^{\mathrm{succ}}_s) \cdot \mathsf{prob}^{\mathrm{retry}}.$$

The components are probability distributions on the complete answer alphabet, including the matrix answer ζ. If failure has probability zero, its unused component may be arbitrary. The sum of absolute differences between two probability distributions is at most two. Hence, in particular, for each individual answer,

$$|\mathsf{prob}^{\mathrm{out}}_{\zeta,z} - \widehat{\mathsf{prob}}_{\zeta,z}| \le 2(1 - p^{\mathrm{succ}}_s) \le 4/2^n. \tag{B.29}$$

Combining Eqs. (B.28) and (B.29) with the L_2 triangle inequality proves Eq. (B.25). The proof is per answer throughout, so no factor of the answer-alphabet size appears. □

Lemma B.5 (Exact canonical execution)**.** *Fix the public input and all secret input values, hence every wire's plaintext value. The joint distribution of all generated wire encodings and leakage answers is exactly the distribution generated by the following canonical experiment: sample every distinct wire encoding independently from its corresponding μ_s, then generate successive adaptive answers from the conditional leakage probability functions of the actual gadgets given their incident wire encodings.*

Proof. Fix an incoming history and the complete input encodings of one operation. For a computation gadget, Lem. B.2 shows that its final output encoding has distribution $\mu^{(n)}_s$, independently of this information. This statement holds for every refreshed long pair and includes all retries. For an input-port refresh, the same freshness follows from Lem. 2.19, since its matrix is sampled independently of the preceding execution. A public-input sampler draws an independent encoding of its prescribed public bit. In every case, generate the actual transition by first sampling that fresh output and then sampling its complete answer conditional on the old and new encodings. This reproduces exactly the original transition, including any dependence of the answer on the new encoding.

Apply this identity successively in topological order, using the actual adaptive leakage functions selected by the incoming history. Since each new encoding is fresh independently of the entire preceding execution, the wire samples may equivalently be drawn before any leakage answers are generated. A location read by multiple operations has one sample, reused at every such read.

The initial input encodings already have this independent product distribution. The constructed experiment therefore reproduces the joint encoding-and-answer distribution. Marginalizing the hidden encodings gives the leakage transcript. Appending the trusted decoder's prescribed output gives the compiler view. □

The lemma extends to labels that are chosen during the execution, where the public bit of a sampler, and hence the plaintext values of the wires that later operations produce, may be functions of the leakage answers preceding the operation. This is the setting of the history-dependent labelings in Sec. 5.4.1 and of the stateful protocol in Sec. 6.3. The transition argument above is unchanged, since the fresh output of an operation is sampled from μ_b for the bit b that its incoming history prescribes. Applying it in topological order gives the canonical experiment of Sec. 5.4.1, which samples each wire encoding immediately before its producing operation and whose transcript probabilities are given by Eq. (5.27) with the induced labeling. Only the final step, which moves all wire samples before the first answer, requires the labels to be fixed in advance. The composition proof does not use that step.

We return to the gate notation of Sec. 5.3: S_i is the set of three wires incident to gate i, X_{S_i} is their tuple of encodings, $U_{\mathsf{L}}^{S_i}$ and $U_{\mathsf{R}}^{S_i}$ are the sets of triples of nonzero left and of nonzero right shares indexed by S_i, with their product uniform measures, and h is the incoming answer history. Complete gate answers belong to the alphabet $\mathcal{Z}$ of size K (see Sec. 5.2.1). A labeling is valid when its wire bits satisfy every gate operation.

For a local labeling s, write $\|F\|_{2,s} := \left(\mathbb{E}_{X_{S_i} \leftarrow\$ \bigotimes_{w\in S_i} \mu_{s(w)}}\left[|F(X_{S_i})|^2\right]\right)^{1/2}$.

Lem. 5.6 (A gate with its inputs and output retained; restated)**.** *There exists an exact canonical experiment with conditional leakage probability functions $\mathsf{prob}_{i,h,z}(X_{S_i})$, defined on valid local labelings, with $\mathsf{prob}_{i,h,z} \geq 0$ and $\sum_z \mathsf{prob}_{i,h,z} = 1$. Its joint distribution of generated wire encodings and answers equals that of the actual execution. There exists an absolute constant $C > 0$ such that, for every threshold $\beta \in [q^{-2/3}, 1]$, computation operation i, formal incoming history $h \in \mathcal{Z}^{i-1}$, and answer $z \in \mathcal{Z}$, there exists a real table $G^{\beta}_{i,h,z}$, common to all wire labelings, with a product representation whose left and right energies Φ and Ψ (the sums of squared factors; see Def. 2.16) satisfy, for all grouped shares $x \in U_{\mathsf{L}}^{S_i}$ and $y \in U_{\mathsf{R}}^{S_i}$,*

$$\left\|G^{\beta}_{i,h,z}\right\|_{*} \leq 4, \tag{5.20}$$

$$\Phi(x) \leq 2/\beta, \tag{5.21}$$

$$\Psi(y) \leq 2/\beta, \tag{5.22}$$

$$\mathbb{E}_{x\leftarrow\$ U_{\mathsf{L}}^{S_i}}[\Phi(x)] \leq 4, \tag{5.23}$$

$$\mathbb{E}_{y\leftarrow\$ U_{\mathsf{R}}^{S_i}}[\Psi(y)] \leq 4, \tag{5.24}$$

$$\left\|\mathsf{prob}_{i,h,z} - G^{\beta}_{i,h,z}\right\|_{2,s} \leq C\cdot\beta^{1/3} \quad \textit{for every valid local labeling } s, \tag{5.25}$$

$$\left\|G^{\beta}_{i,h,z}\right\|_{2,s} \leq C_0 \quad \textit{for every local labeling } s. \tag{5.26}$$

Here $C_0 := 4\cdot\sqrt{27}$, and C is independent of the dimension, threshold, answer, formal history, and admissible leakage functions.

Proof of Lem. 5.6. Use the true complete-answer functions $\mathsf{prob}^{\mathrm{out}}_{\zeta,z}$ from Lem. B.4. They are nonnegative and sum to one on every valid local labeling. Lem. B.5 gives the exact joint encoding-and-answer distribution, whose answer marginal is the leakage transcript.

Fix a gate, a formal incoming history, and $q^{-2/3} \le \beta \le 1$. Apply Cor. B.1 to each matrix-phase answer ζ, with $d = q^2$ and $m = n^2 + 1$. It supplies the global approximation $2 \cdot \mathsf{prob}_\zeta^{>\beta}$, where the superscript keeps the singular coefficients exceeding β as in Eq. (5.11), with nuclear norm at most four, balanced pointwise energies at most $2/\beta$, both mean energies at most four, and valid-label L_2 error $C \cdot \beta^{1/3}$ (see Eq. (B.11)). For each shortening answer apply Lem. B.4 to this representation. The transported table satisfies the separate energy bounds, the nuclear bound, and the arbitrary-label L_2 bound by Eqs. (B.20) to (B.24) and the last assertion of that lemma. By Eq. (B.25), its valid-label error is at most

$$C \cdot \beta^{1/3} + 4/2^n \le C' \cdot \beta^{1/3},$$

since $\beta \ge q^{-2/3}$ and $2^{-n} \le q^{-2/9} \le \beta^{1/3}$. Relabel the complete answer (ζ, z) as one answer of the gadget. All constructions depend only on the public maps, masks, threshold, and formal incoming history. The same tables serve every plaintext labeling. The constants are absolute and uniform in all these choices. The transported representation is the chosen representation in the interface and is not assumed to be balanced. □

C An error in the Goldwasser–Rothblum proof

We describe an error in Goldwasser and Rothblum's leakage-resilient computation proof [27]. Their construction uses ciphertext banks to protect arbitrary circuits for an unbounded number of executions in the model where only computation leaks. All claim, lemma, figure, and page references to their work below are to the journal version, *SIAM Journal on Computing* **44**(5), 1480–1549 (2015). The main issue is Claim 6.24, an auxiliary statement used in the security argument for continual refreshing. We give a counterexample to that claim and briefly record other issues in the printed argument and procedures. All the errors and gaps discussed in this appendix were identified with the help of ChatGPT.

We contacted Goldwasser and Rothblum, who confirmed the gap. They told us that they had known about it for some time and had a fix, which had not yet been published at the time of our correspondence.

C.1 Claim 6.24 and its hypotheses

We first recall Claim 6.24 and the notions it uses. All arithmetic in this appendix is over $\mathbb{F}_2$. The symbols below are those of [27] and are local to this appendix. Several of them, including K, κ, L, ℓ, m, and d, denote different objects here than in the rest of the paper. Write $K \in \mathbb{F}_2^\kappa$ for a column key and $M \in \mathbb{F}_2^{\kappa \times m}$ for a matrix. The relevant notion of *independence up to orthogonality* (IuO) is defined in [27, Definitions 6.18–6.19, p. 1523].

Definition C.1 (Independence up to orthogonality)**.** *A distribution $\mathcal{V}$ on $\mathbb{F}_2^\kappa \times \mathbb{F}_2^{\kappa \times m}$ is IuO if there exist distributions μ_K on $\mathbb{F}_2^\kappa$ and μ_M on $\mathbb{F}_2^{\kappa \times m}$ such that one obtains $\mathcal{V}$ by sampling $K \leftarrow\!\$\ \mu_K$ and then sampling $M \leftarrow\!\$\ \mu_M$ conditioned on $K^\mathsf{T} \cdot M = 0$. The conditioning event must have positive probability for every key in the support of μ_K.*

Its independently drawn variant is $\mu_K \otimes \mu_M$, with the orthogonality condition omitted. The underlying distributions need not in general be the actual marginals of $\mathcal{V}$. Our counterexample uses the actual marginals and therefore also applies to that more restrictive reading.

For a random variable Z taking values in a finite set $\mathcal{Z}$, let

$$H_\infty(Z) := -\log_2 \max_{z \in \mathcal{Z}} \mathbb{P}[Z = z].$$

Consider the positive integer parameters a, κ, and m of [27, Lemma 6.21], where $m \geq \kappa$ and a is the number of matrix pieces. The piece width is $\ell = \kappa/10$, and the per-piece leakage bound is $\lambda = \kappa/(20 \cdot a)$. The parameters are chosen so these are integers. A matrix piece is $M \cdot L$, where $L \in \mathbb{F}_2^{m \times \ell}$ has rank ℓ. Thus a piece consists of ℓ linear combinations of the columns of M. The adversary may obtain bounded leakage separately from the key and from the matrix piece. One conclusion of Claim 6.24 [27, pp. 1526–1527] is the following: if $\mathcal{V}$ is IuO, every matrix in its support has rank $\kappa - 1$, and

$$H_\infty(K, M) \geq \kappa + (\kappa - 1) \cdot 2\kappa - 0.15\kappa, \tag{C.1}$$

then the joint distributions of the key and leakage transcript W under $\mathcal{V}$ and under $\mu_K \otimes \mu_M$ satisfy the following bound. Here $\mathcal{L}_\mathcal{V}(K, W)$ denotes the joint distribution in the experiment with key–matrix distribution $\mathcal{V}$, and the same leakage strategy is used in both experiments:

$$\delta(\mathcal{L}_\mathcal{V}(K, W), \mathcal{L}_{\mu_K \otimes \mu_M}(K, W)) \leq 8\ell \cdot 2^{-0.05\kappa}. \tag{C.2}$$

In particular, the conclusion includes disclosure of the entire key after the bounded leakage. We take $m = 2\kappa$, so the entropy threshold is exactly the one printed in the claim.

C.2 A counterexample with one matrix entry

The claim says that, for a distribution meeting these hypotheses, the key and leakage transcript are nearly indistinguishable from those obtained with an independently drawn key and matrix. To refute the bound, we seek a relation that can be tested from the disclosed key and bounded matrix leakage and holds with noticeably different probabilities in the two experiments. We therefore build a distribution in which one matrix entry copies one key bit: a single leaked bit then exposes the relation once the key is disclosed, while the copy costs only a constant number of bits of joint min-entropy. The following construction is defined for every integer $\kappa \geq 3$, with $m = 2\kappa$. Below we specialize to parameters satisfying the claim's integrality and entropy requirements. Index vector coordinates and matrix rows by $0, \ldots, \kappa - 1$, and matrix columns by $1, \ldots, m$. Put

$$\mathcal{K} := \{k \in \mathbb{F}_2^\kappa : k[0] = 1\}.$$

For $k \in \mathcal{K}$, define

$$\mathcal{F}_k := \left\{ M \in \mathbb{F}_2^{\kappa \times m} : \operatorname{rank}(M) = \kappa - 1, \quad k^\mathsf{T} \cdot M = 0, \quad M[2, 1] = k[1] \right\}. \tag{C.3}$$

Sample K uniformly from $\mathcal{K}$ and, conditional on $K = k$, sample M uniformly from $\mathcal{F}_k$. Call the resulting joint distribution $\mathcal{D}$ and its actual marginals μ_K and μ_M. The entry $M[2, 1]$ is a marker: it copies the key bit $K[1]$. This equality is not forced by orthogonality alone. We verify that $\mathcal{D}$ nevertheless satisfies the hypotheses of Claim 6.24.

Rank and entropy. Put $d := \kappa - 1$. For every $k \in \mathcal{K}$, the linear functional $v \mapsto v[2]$ is nonzero on the d-dimensional hyperplane $k^\perp$. Otherwise e_2 would lie in $(k^\perp)^\perp = \operatorname{span}\{k\}$, contradicting $k[0] = 1$. Hence exactly 2^{d-1} choices for the first column meet the marker constraint. Choose the

remaining $m-1$ columns independently and uniformly in $k^\perp$. If they do not span $k^\perp$, some nonzero linear functional on that hyperplane vanishes on all of them. A union bound therefore gives

$$\mathbb{P}\Big[\text{the last } m-1 \text{ columns do not span } k^\perp\Big] \le (2^d-1)2^{-(m-1)} < 2^{-\kappa}.$$

If these columns span the hyperplane, the whole matrix has rank d, regardless of the first column. Consequently,

$$|\mathcal{F}_k| > 2^{dm-1}(1-2^{-\kappa}),$$

so these sets are nonempty and every supported pair has probability $2^{-(\kappa-1)}/|\mathcal{F}_k|$. It follows that

$$\begin{aligned} H_\infty(K,M) &> \kappa + dm - 2 + \log_2(1-2^{-\kappa}) \\ &> \kappa + dm - 3. \end{aligned} \tag{C.4}$$

Thus the marker costs only a constant number of bits of joint min-entropy.

Independence up to orthogonality. To sample the matrix marginal μ_M, first choose U uniformly from $\mathcal{K}$ and then choose M uniformly from $\mathcal{F}_U$. The matrix has rank $\kappa-1$, and the unique nonzero vector in its left kernel is U. For any independently chosen $k \in \mathcal{K}$, therefore,

$$k^{\mathsf{T}} \cdot M = 0 \quad \Longleftrightarrow \quad U = k.$$

This event has probability $2^{-(\kappa-1)}$, independent of k. Conditional on it, M is uniform on $\mathcal{F}_k$. Sampling from the actual marginals and imposing orthogonality therefore recovers $\mathcal{D}$ exactly. This proves the IuO property. Since the acceptance probability is constant, the example also works if one conditions the entire product distribution on orthogonality. The marker is compatible with this property because each matrix in the support determines its key, the unique nonzero vector in its left kernel. Conditioning the common matrix distribution on orthogonality to a chosen key selects that key's fiber. The relative weights inside the fiber already enforce the marker, so this conditioning does not remove it.

One leaked bit distinguishes the two distributions. Let L select the first ℓ columns of M, so that L has rank ℓ, and request only

$$W := M[2,1].$$

This is one bit of leakage from the allowed matrix piece $M \cdot L$. No key leakage is needed before the final disclosure. Under $\mathcal{D}$, one always has $W = K[1]$. Under $\mu_K \otimes \mu_M$, one has $W = U[1]$, where U is uniform on $\mathcal{K}$ and independent of K. Thus W is a fair bit independent of K in the second experiment. The test $W = K[1]$ accepts with probabilities one and $1/2$, respectively. Indeed, for every fixed key the conditional transcript distributions have distance $1/2$. Hence

$$\delta(\mathcal{L}_{\mathcal{D}}(K,W), \mathcal{L}_{\mu_K \otimes \mu_M}(K,W)) = \frac{1}{2}. \tag{C.5}$$

For a concrete choice, take

$$\kappa = 1000, \qquad m = 2000, \qquad a = 1, \qquad \ell = 100, \qquad \lambda = 50.$$

The entropy bound in Eq. (C.4) exceeds the threshold in Eq. (C.1), and the one-bit query is within the leakage allowance. But Eq. (C.2) would bound the distance by $800 \cdot 2^{-50} < 10^{-12}$, contradicting Eq. (C.5).

C.3 The failed inference and its scope

The proof on p. 1527 treats conditioning the key on the next observed matrix column as adding only its orthogonality equation. Independence up to orthogonality and high joint entropy do not justify this step. At the entropy threshold used here, a single exact relation can remain even though probability is spread over many key–matrix pairs. In Eq. (C.3), observing the first column also reveals $K[1] = M[2, 1]$, an additional constraint that survives the entropy requirement. Thus Claim 6.24 is false as stated, and its use in the proof of Lemma 6.21 needs a different justification. The counterexample does not show that an honestly initialized bank reaches this distribution, and it does not by itself refute the fresh-matrix statement of Lemma 6.21 or give an attack on the complete compiler.

Repairing the security argument is quite involved and requires techniques such as Fourier analysis. We do not present the repair here.

C.4 Other issues, briefly

Besides Claim 6.24, several other places in the printed argument and procedures need correction or further justification, and we record them briefly.

Posterior distributions and conditioning. The exact assertions in Lemma 5.3 (pp. 1503–1504) that the actual posterior key and ciphertext marginals are independent of the plaintexts, and that conditioning their product recovers the joint posterior, do not hold in general. For example, a rare ciphertext-side leakage event can impose a plaintext-dependent equation on the key. This concerns an exact identity and does not itself contradict an averaged negligible-error security guarantee. The collision-conditioning argument around Claim 6.28 (pp. 1529–1532) also needs justification, since conditioning on equal transcripts can reweight the starting key–matrix pair. The joint sampling and locality steps in Propositions 6.11 and 6.13 (pp. 1518–1519) need further justification as well. We do not claim that those propositions or Claim 6.28 are false.

Printed procedures and the permutation simulator. The unrestricted redraw in BankGenRand (Figure 3, p. 1509) can change the ciphertext coordinate required to equal one, leaving the encoding domain. Figure 6 (p. 1514) computes a refreshed key but returns the old key, which can change the decrypted bit if the return line is executed literally. The all-zero simulator used for Lemma 7.2 and Proposition 7.3 (pp. 1535–1537) is inconsistent with an observable linear relation satisfied by the real permutation input. Finally, the row constructed by SafeNAND (Figure 10, p. 1540) contains ciphertexts outside the domain of the refresh it invokes. The printed procedure can produce incorrect outputs and allows one bit of leakage to reveal its mask, contrary to its stated local simulation guarantee. These issues concern the procedures and guarantees as printed and are distinct from an end-to-end secret-recovery attack.